Contents

1 Introduction to Data Compression **2**

1.1 Learning Objectives . . . 2

1.2 Introduction and Motivation: Why Compress Data? . . . 2

1.2.1 Benefits of Data Compression . . . 3

1.3 Lossless vs. Lossy Compression . . . 3

1.3.1 Lossless Compression . . . 3

1.3.2 Lossy Compression . . . 4

1.3.3 Choosing Between Lossless and Lossy . . . 4

1.4 Compression Performance Metrics . . . 4

1.4.1 Size-Based Metrics . . . 4

1.4.2 Rate-Based Metrics . . . 4

1.5 Worked Example: Audio Compression Metrics . . . 5

1.6 Huffman Coding . . . 5

1.6.1 Step-by-Step Huffman Coding Example . . . 5

1.6.2 Key Properties of Huffman Coding . . . 7

1.7 End of Chapter Questions . . . 7

2 Theory of Compression — Limits and Optimality **11**

2.1 Types of Codes: From Ambiguous to Instantaneous . . . 11

2.2 Basic Terminology and Notation . . . 12

2.3 Information and Redundancy: The Core Concepts . . . 14

2.3.1 Information: A Formal Measure of Uncertainty Reduction . . . 14

2.3.2 Redundancy: The Enemy of Information and the Friend of Compression . . . 15

2.4 Entropy: The Fundamental Limit . . . 16

2.4.1 What is Entropy? Different Perspectives . . . 16

2.4.2 Calculating Entropy: Step by Step . . . 16

2.4.3 Entropy of English Text: A Practical Case Study . . . 17

2.4.4 Beyond First-Order Entropy: The Full Picture . . . 17

2.4.5 Entropy Rate . . . 18

2.4.6 The Entropy Theorem: Why It Matters . . . 18

2.4.7 Key Takeaways . . . 20

2.5 Entropy as a Lower Bound . . . 21

2.5.1 The Fundamental Inequality . . . 21

2.6 Kraft-McMillan Inequality: Core Theoretical Tool . . . 21

2.6.1 Statement and Interpretation . . . 21

2.7 Optimality of Huffman Codes (Theory Only) . . . 22

2.7.1 The Optimality Theorem . . . 22

2.7.2 Relation to Entropy . . . 22
2.7.3 Why Huffman is Optimal but Not Perfect . . . 23
2.8 Limitations of Symbol-by-Symbol Coding . . . 23
2.8.1 Three Fundamental Limitations . . . 23
2.9 Block Coding and Improved Efficiency . . . 24
2.9.1 The Block Coding Idea . . . 24
2.9.2 Key Mathematical Results . . . 24
2.9.3 Step-by-Step Example . . . 25
2.10 Trade-Offs in Block Coding . . . 25
2.10.1 The Engineering Challenges . . . 25
2.11 From Block Coding to Modern Compression . . . 26
2.11.1 Arithmetic Coding: Fractional-Bit Block Coding . . . 26
2.11.2 Context Modeling: Approximating Large Blocks . . . 26
2.12 What Theory Guarantees vs What Practice Achieves . . . 26
2.13 Summary and Key Takeaways . . . 26
2.13.1 The Big Picture . . . 27
2.13.2 Looking Forward . . . 27

3 Advanced Entropy Coding & Extensions 28
3.1 Introduction & Motivation . . . 28
3.2 Coding Taxonomy & Framework . . . 28
3.3 Shannon Coding (1948) . . . 30
3.4 Shannon–Fano Coding (1949) . . . 34
3.5 Canonical Huffman Codes . . . 37
3.6 Adaptive Huffman Coding . . . 44
3.7 Arithmetic Coding: The Paradigm Shift . . . 47
3.8 Comparison & Synthesis . . . 48
3.9 Forward Look . . . 49

4 Source Modeling and Statistical Dependence 53
4.1 Introduction: Beyond Coding . . . 53
4.2 Memoryless vs. Sources with Memory . . . 54
4.3 Conditional Entropy and Mutual Information . . . 55
4.4 Markov Sources . . . 56
4.5 Entropy Rate Revisited . . . 57
4.6 Context Modeling in Practice . . . 59
4.7 Case Study: Text Compression Modeling . . . 60
4.8 The Modeling–Coding Separation Principle . . . 61
4.9 Adaptive vs. Static Modeling . . . 62
4.10 Summary and Forward Look . . . 63

5 Dictionary-Based Compression: The Lempel–Ziv Revolution **65**
5.1 Motivation: Hitting the Limits of Statistical Coding 65
5.1.1 Recap: The Block Coding Dilemma 65
5.1.2 The Promise of Exploiting Long-Range Repetition 65
5.2 Paradigm Shift: From Statistics to Dictionaries 65
5.2.1 Core Philosophy of Dictionary Coding 66
5.2.2 Explicit vs. Implicit (Adaptive) Dictionaries 66
5.3 The Lempel-Ziv Algorithms 66
5.3.1 LZ77: The Sliding Window Algorithm 66
5.3.2 LZSS: Improving Practical Efficiency 67
5.3.3 LZ78: The Dictionary Growth Algorithm 67
5.3.4 LZW (Lempel-Ziv-Welch) 67
5.4 Theoretical Properties: Why Lempel–Ziv Works 67
5.4.1 The Universality Principle 67
5.4.2 Asymptotic Optimality 68
5.5 Bridging Paradigms: Dictionary vs. Entropy Coding 68
5.5.1 Complementary Approaches 68
5.5.2 The Hybrid Approach 68
5.6 Summary and Forward Look 68
5.6.1 Key Takeaways 68

6 DEFLATE Algorithm: The Standard in Practice **72**
6.1 Introduction: The Universal Compression Standard 72
6.1.1 Where DEFLATE Is Used: gzip, ZIP, PNG, HTTP 72
6.1.2 Historical Context: Phil Katz, ZIP Format, and Open Standards . 72
6.2 High-Level Architecture: A Two-Stage Pipeline 73
6.2.1 Stage 1: LZSS String Matching (Finding Redundancy) 73
6.2.2 Stage 2: Huffman Coding (Encoding the Reduced Data) 73
6.2.3 Why the Hybrid Approach Works: Transforming Redundancy . . 74
6.2.4 Design Philosophy: Fast Decoding Over Maximum Compression . 74
6.3 LZSS in DEFLATE: Detailed Implementation 74
6.3.1 Sliding Window: 32KB History and Lookahead Buffer 74
6.3.2 Hash Chains for Fast Match Finding 75
6.3.3 Lazy Matching and Greedy Heuristics 75
6.3.4 Output Format: (Length, Distance) Pairs vs. Literal Bytes 78
6.4 DEFLATE's Unique Huffman Coding Scheme 80
6.4.1 Two Alphabets: Literals (0-285) and Distances (0-29) 80
6.4.2 Canonical Huffman via Code Lengths: The Brilliant Compression 80
6.4.3 Run-Length Encoding of Code Lengths (19-Symbol Alphabet) . . 81
6.4.4 Why Canonical Codes Enable O(1) Decoding 81

6.5 Block Structure and Streaming Format 82
6.5.1 Three Block Types: Stored, Fixed Huffman, Dynamic Huffman . . 82
6.5.2 Bit-Level Layout: BFINAL, BTYPE, and Data 82
6.5.3 Streaming Support: Independent Blocks and Reset Points 83
6.6 Step-by-Step Compression Walkthrough 83
6.6.1 Input: "The cat sat on the mat" 83
6.6.2 LZSS Processing: Finding Matches 83
6.6.3 Building Canonical Huffman Tables 84
6.6.4 Bitstream Assembly: Final Compressed Output 84
6.7 Compression Levels and Performance Trade-offs 84
6.7.1 zlib's 9 Levels: From "fast" to "best" 84
6.7.2 Speed vs. Ratio: Why Level 6 is the Default 85
6.7.3 Memory Footprint: 32KB to 256KB Windows 85
6.8 Practical Applications and File Formats 85
6.8.1 gzip Format: Headers, Footers, and CRC32 85
6.8.2 ZIP Format: Multiple Files and DEFLATE 86
6.8.3 PNG: DEFLATE on Filtered Scanlines 86
6.8.4 HTTP Compression: Accept-Encoding Header 86
6.9 Limitations and When DEFLATE Fails 86
6.9.1 Poor Performance on Already-Compressed or Random Data . . . 86
6.9.2 The "Shakespeare Problem": Global vs. Local Repetition 86
6.9.3 Why Not Arithmetic Coding? Patents, Speed, and Complexity . . 87
6.9.4 Security Issues: CRIME, BREACH, and Compression Side-Channels 87
6.10 Modern Alternatives and Successors 87
6.10.1 LZMA and 7-Zip: Better Compression, Slower Speed 87
6.10.2 Zstandard (zstd): Facebook's Balanced Alternative 88
6.10.3 Brotli: Google's Web-Optimized Successor 88
6.10.4 Why DEFLATE Still Dominates: The Legacy Effect 88
6.11 Hands-On Analysis and Debugging 89
6.11.1 Using gzip -v -l and infgen for Inspection 89
6.11.2 Visualizing Matches and Huffman Trees 89
6.11.3 Benchmarking: Compression Ratio vs. Speed 89
6.12 Summary and Key Insights 89
6.12.1 Why DEFLATE Dominated for 30+ Years 89
6.12.2 Lessons for Compression Engineers: Practical Beats Perfect 90
6.12.3 Looking Forward: The Future of Compression Standards 90

7 Run-Length Encoding: Exploiting Equality Redundancy 96
7.1 Introduction and Motivation 96
7.1.1 The Simplest Form of Redundancy: Repetition 96

7.2 Fundamental Concepts and Theory . . . 97
7.2.1 Formal Definition of a Run . . . 97
7.2.2 Run-Length Distribution Theorem . . . 98
7.2.3 Compression Ratio Analysis . . . 98
7.3 Taxonomy of RLE Methods . . . 99
7.4 Count-Based RLE (Classic RLE) . . . 101
7.4.1 Basic Encoding Algorithm . . . 101
7.4.2 Basic Decoding Algorithm . . . 101
7.4.3 Count-After-Symbol vs. Count-Before-Symbol . . . 102
7.4.4 Fixed-Width Count RLE . . . 102
7.4.5 Handling Large Runs with Fixed-Width Counts . . . 104
7.4.6 Variable-Width Count RLE (Elias Gamma) . . . 105
7.5 Escape-Based RLE . . . 111
7.5.1 Core Concept . . . 111
7.5.2 Basic Escape Scheme Design . . . 111
7.5.3 Encoding Algorithm . . . 112
7.5.4 Decoding Algorithm . . . 113
7.5.5 Worked Example . . . 113
7.5.6 Choosing the Threshold . . . 115
7.5.7 Advantages and Disadvantages . . . 116
7.6 Block/Header-Based RLE (PackBits Style) . . . 117
7.6.1 PackBits Overview . . . 117
7.6.2 Encoding Algorithm . . . 118
7.6.3 Decoding Algorithm . . . 119
7.6.4 Worked Examples . . . 119
7.6.5 Optimal Use of PackBits . . . 121
7.6.6 PackBits in Real-World Formats . . . 122
7.7 Zero Run-Length Encoding (Zero RLE) . . . 123
7.7.1 Motivation: Sparse Data in Transform Coding . . . 123
7.7.2 Basic Zero RLE Algorithm . . . 123
7.7.3 Zero RLE in JPEG: A Detailed Case Study . . . 124
7.7.4 Handling Long Zero Runs . . . 126
7.7.5 Zero RLE in Other Transform Codecs . . . 126
7.7.6 Zero RLE Pipeline in Transform Coding . . . 127
7.8 Comparison of RLE Variants . . . 128
7.8.1 Decision Matrix . . . 128
7.8.2 When to Use Each Variant . . . 128
7.9 Limitations of Run-Length Encoding . . . 129
7.9.1 Why RLE Fails on Natural Language Text . . . 129
7.9.2 Sensitivity to Symbol Ordering . . . 130

7.10 RLE as a Preprocessing Technique 131
7.10.1 Combining RLE with Huffman Coding 131
7.10.2 The Crucial Role of RLE in BWT Pipeline 132
7.11 Summary 132

8 Transform-Based Compression: The Burrows–Wheeler Transform 134
8.1 The Three Paradigms of Lossless Compression 134
8.2 Motivation: Beyond Local Matching 134
8.2.1 The Limits of Sliding Windows 134
8.2.2 The Global Reordering Idea 135
8.3 The Burrows–Wheeler Transform 136
8.3.1 Historical Background 136
8.3.2 Intuition: Grouping Similar Contexts 136
8.3.3 Step-by-Step Example: `banana$` 136
8.3.4 Visualizing the Effect: Before and After 138
8.4 Formal Definition of the BWT 138
8.4.1 Cyclic Rotations and Lexicographic Sorting 138
8.4.2 The Last Column Construction 139
8.4.3 Properties of the Transform 139
8.5 Inverting the BWT 140
8.5.1 The First–Last (LF) Mapping 140
8.5.2 Algorithm for Computing LF-Mapping 141
8.5.3 Reconstruction Algorithm 145
8.5.4 Proof of Correctness (Sketch) 147
8.6 Why BWT Improves Compressibility 148
8.6.1 Run Formation and Structure Revelation 148
8.6.2 Connection to Context Modeling 149
8.7 The Full BWT Compression Pipeline 149
8.7.1 Stage 1: BWT Transform 149
8.7.2 Stage 2: Move-To-Front (MTF) Encoding 149
8.7.3 Stage 3: Run-Length Encoding (RLE) 151
8.7.4 Stage 4: Final Entropy Coding (Huffman or Arithmetic) 152
8.8 Why RLE is Essential After MTF 153
8.9 Efficient Implementation 154
8.9.1 Naive Rotation Matrix: Why It Is Impractical 154
8.9.2 Suffix Arrays for Efficient Construction 154
8.9.3 Time and Space Complexity 155
8.10 Comparison with Lempel–Ziv Methods 155
8.10.1 Local vs. Global Redundancy 155
8.10.2 Compression Ratio vs. Speed Trade-offs 156

8.11 Case Study: `bzip2` . . . 156
8.11.1 Architecture Overview . . . 156
8.11.2 Why BWT Beats DEFLATE on Text . . . 157
8.12 Limitations and Modern Extensions . . . 157
8.12.1 Block Size Constraints . . . 157
8.12.2 Memory Usage Considerations . . . 158
8.12.3 When BWT May Not Improve Compression . . . 158
8.12.4 Relation to FM-Index and Text Indexing . . . 159
8.13 Summary and Key Insights . . . 159
8.13.1 The Transform-Then-Compress Principle . . . 159
8.13.2 The BWT-RLE Connection . . . 160
8.13.3 When to Use BWT-Based Compression . . . 160
8.14 Hands-On Exercises . . . 160

9 Prediction by Partial Matching (PPM): The Statistical Pinnacle 163
9.1 Introduction: Beyond Finite-Context Models . . . 163
9.1.1 Recap of Markov Sources . . . 163
9.1.2 The Fundamental Insight . . . 164
9.2 The PPM Algorithm . . . 164
9.2.1 Core Components . . . 165
9.2.2 The Escape and Prediction Mechanism . . . 166
9.2.3 Escape Estimation: The PPMC Method . . . 166
9.2.4 The Exclusion Principle . . . 170
9.3 Step-by-Step Examples . . . 170
9.3.1 Example 1: Encoding `"banana"` . . . 170
9.3.2 Example 2: Encoding `"abracadabra"` (PPMC, $M = 2$) . . . 173
9.3.3 Example 3: Repetitive String `"abababababab"` ($M = 1$, PPMC) . . . 180
9.3.4 Example 4: Exclusion Applied to `"abracadabra"` (Symbol 5) . . . 181
9.4 PPM Implementation Challenges . . . 181
9.4.1 Memory Management . . . 181
9.4.2 Data Structures: The Context Trie . . . 182
9.4.3 The Zero-Frequency Problem and Modern Refinements . . . 182
9.5 PPM Variants and Successors . . . 183
9.5.1 PPM*: Unbounded Contexts . . . 183
9.5.2 PPMZ: Optimised Practical Variant . . . 184
9.5.3 PAQ: Context Mixing . . . 184
9.5.4 Limitations of PPM . . . 185
9.6 Why PPM Matters Today . . . 185
9.6.1 Theoretical Foundations . . . 185
9.6.2 Applications Beyond Compression . . . 186

9.6.3 Influence on Modern Compression Systems 186

10 Modern Entropy Coders — ANS and Range Coding 190

10.1 Introduction: Beyond Huffman's Limitations 190
10.2 Range Coding: Arithmetic With Integers 190
10.2.1 Core Concept 191
10.2.2 Renormalization with Byte Output 191
10.2.3 Advantages 192
10.2.4 Range Coding Example 192
10.3 Asymmetric Numeral Systems (ANS): The Breakthrough 193
10.3.1 Historical Context 193
10.3.2 The Core Insight 193
10.3.3 Intuitive Explanation: The "Stack" Property 194
10.3.4 Why ANS Is Faster 194
10.4 Variations of ANS 194
10.4.1 rANS (Range ANS) 194
10.4.2 tANS (Table ANS) 195
10.4.3 Comparison 195
10.5 Detailed rANS Example 195
10.5.1 Normalized Frequencies 195
10.5.2 Initial State 196
10.5.3 Encoding Equations 196
10.5.4 Renormalization 196
10.5.5 Encoding the Sequence `B A B A` 197
10.5.6 Final Compressed State 198
10.5.7 Decoding 198
10.5.8 Decoding Step-by-Step 199
10.5.9 Summary 200
10.6 ANS in Modern Compressors 200
10.6.1 Zstandard (Facebook / Meta) 200
10.6.2 Apple LZFSE 201
10.6.3 Google Draco 201
10.6.4 JPEG XL 201
10.7 Hands-On: Implementing a Simple ANS Coder 201
10.7.1 Educational rANS Implementation 202
10.7.2 Key Implementation Notes 204
10.8 Summary 204
10.9 Exercises 205

Data Compression
Lecture Notes

Dr. Faisal Aslam

1: Lecture 1: Introduction to Data Compression

1.1 Learning Objectives

By the end of this lecture, students will be able to:

- Understand the motivation and benefits of data compression
- Differentiate between lossless and lossy compression techniques
- Compute and interpret common compression performance metrics
- Apply the Huffman coding algorithm step by step
- Analyze real-world compression trade-offs

1.2 Introduction and Motivation: Why Compress Data?

Data compression is the process of representing information using fewer bits than its original form. It is a fundamental component of modern computing systems, enabling efficient storage, faster communication, and reduced operational costs.

Everyday applications of compression include:

- Streaming audio and video
- Image storage and sharing
- File archiving and backups
- Network communication and cloud services

Definition

Data Compression is the process of reducing the number of bits required to represent information, either:

- **Losslessly**: allowing exact reconstruction of the original data
- **Lossily**: allowing controlled loss of information to achieve higher compression

1.2.1 Benefits of Data Compression

Data compression provides three key benefits that are critical in modern computing:

1. **Reduce Storage Space**:
   - Allows more data to be stored in the same physical space
   - Enables archival of historical data that would otherwise be discarded
   - Reduces hardware requirements for storage systems
2. **Reduce Communication Time and Bandwidth**:
   - Enables faster file transfers and downloads
   - Makes high-quality streaming (4K/8K video) practical over limited bandwidth
   - Reduces latency in real-time applications like video conferencing and online gaming
   - Allows IoT devices to transmit data efficiently over wireless networks
3. **Save Money**:
   - Reduces cloud hosting costs (storage and egress fees)
   - Lowers communication costs for data transmission
   - Decreases capital expenditure on storage hardware
   - Reduces energy consumption for data centers and network infrastructure

1.3 Lossless vs. Lossy Compression

1.3.1 Lossless Compression

Lossless compression guarantees perfect reconstruction of the original data. It is essential when accuracy and data integrity are critical.

Typical applications:

- Text files and source code
- Executables and databases
- Medical, scientific, and legal data

1.3.2 Lossy Compression

Lossy compression achieves higher compression ratios by discarding information that is less perceptible or less important.

Typical applications:

- Audio (MP3, AAC)
- Images (JPEG)
- Video (H.264, H.265)

1.3.3 Choosing Between Lossless and Lossy

Factor	**Lossless Compression**	**Lossy Compression**
Reconstruction	Exact	Approximate
Data sensitivity	High	Moderate to low
Typical ratios	Low to moderate	High
Quality impact	None	Controlled degradation

Table 1: Lossless vs. Lossy Compression

1.4 Compression Performance Metrics

1.4.1 Size-Based Metrics

$$\text{Compression Ratio (CR)} = \frac{\text{Original Size}}{\text{Compressed Size}}$$
$$\text{Compression Factor} = \frac{\text{Compressed Size}}{\text{Original Size}}$$
$$\text{Space Savings (\%)} = \left(1 - \frac{\text{Compressed Size}}{\text{Original Size}}\right) \times 100\%$$

Interpretation:

- Larger compression ratios indicate better compression
- Smaller compression factors indicate better compression

1.4.2 Rate-Based Metrics

$$\text{Bits per Sample (bps)} = \frac{\text{Compressed Size (bits)}}{\text{Number of samples}}$$
$$\text{Bit-rate (bps)} = \frac{\text{Compressed Size (bits)}}{\text{Time (seconds)}}$$

These metrics are particularly important in audio and video compression systems.

1.5 Worked Example: Audio Compression Metrics

Example

Uncompressed Audio Properties

- Duration: 180 seconds
- Sampling rate: 44.1 kHz
- Bit depth: 16 bits
- Channels: 2 (stereo)

Original Size Calculation

$$\begin{aligned}\text{Total samples} &= 180 \times 44{,}100 \times 2 = 15{,}876{,}000\\ \text{Size (bits)} &= 15{,}876{,}000 \times 16 = 254{,}016{,}000\\ \text{Size (MB)} &= \frac{254{,}016{,}000}{8 \times 1{,}048{,}576} \approx 30.27\end{aligned}$$

Compression Results

Method	Size (MB)	CR	Savings	Bit-rate
FLAC (lossless)	18.16	1.67:1	40%	807 kbps
MP3 @ 320 kbps	6.75	4.49:1	77.7%	320 kbps
AAC @ 256 kbps	5.40	5.61:1	82.2%	256 kbps

1.6 Huffman Coding

Huffman coding is a widely used **lossless compression algorithm** that assigns variable-length binary codes to symbols based on their frequencies. More frequent symbols receive shorter codes.

1.6.1 Step-by-Step Huffman Coding Example

Example

Message: `MISSISSIPPI RIVER` (17 characters including space)

Symbol Frequencies

Symbol	Frequency
I	5
S	4
P	2
R	2
M	1
V	1
E	1
(space)	1

Tree Construction

1. Combine M(1) + V(1) $\rightarrow$ 2
2. Combine E(1) + (space)(1) $\rightarrow$ 2
3. Combine P(2) + R(2) $\rightarrow$ 4
4. Combine 2 + 2 $\rightarrow$ 4
5. Combine 4 + 4 $\rightarrow$ 8
6. Combine I(5) + S(4) $\rightarrow$ 9
7. Combine 8 + 9 $\rightarrow$ 17

One Possible Code Assignment

Symbol	Code	Length
I	00	2
S	01	2
P	100	3
R	101	3
M	1100	4
V	1101	4
E	1110	4
(space)	1111	4

Compressed Size

$$5(2) + 4(2) + 2(3) + 2(3) + 4(1) = 52 \text{ bits}$$

Original Size (ASCII) $= 17 \times 8 = 136$ bits
Compression Ratio $= 136/52 \approx 2.62 : 1$

1.6.2 Key Properties of Huffman Coding

- Produces prefix-free codes
- Enables instantaneous decoding
- Guarantees minimum average code length among prefix codes
- Widely used in practical compression systems

1.7 End of Chapter Questions

Exercise 1.0

Problem 1: Basic Compression Metrics
An uncompressed grayscale image has the following properties:

- Resolution: 1024×1024 pixels
- Bit depth: 8 bits per pixel

After compression, the image size is 320 KB.
Calculate:

(a) Original image size in KB

(b) Compression ratio

(c) Compression factor

(d) Space savings percentage

Exercise 1.1

Problem 2: Audio Bit-rate and Storage
A mono audio recording has the following parameters:

- Duration: 5 minutes
- Sampling rate: 48 kHz
- Bit depth: 16 bits

The file is compressed using a lossy codec to a constant bit-rate of 192 kbps.
Calculate:

(a) Size of the uncompressed audio file in MB

(b) Size of the compressed file in MB

(c) Compression ratio

(d) Bits per sample after compression

Exercise 1.2

Problem 3: Comparing Compression Options

A video clip has an uncompressed data rate of 120 Mbps. Three compression options are available:

Option	Compressed Bit-rate
A	6 Mbps
B	3 Mbps
C	1.5 Mbps

For each option, calculate:

(a) Compression ratio

(b) Data consumed for a 10-minute video (in MB)

Which option would you choose for:

(i) Live video streaming?

(ii) Archival storage?

Briefly justify your answers.

Exercise 1.3

Problem 4: Huffman Coding Construction

Given the following symbol frequencies:

Symbol	Frequency
A	10
B	8
C	6
D	5
E	4
F	3
G	2
H	2

(a) Construct the Huffman tree step by step

(b) Assign a binary code to each symbol

(c) Compute the total number of bits required to encode the message

(d) Calculate the average number of bits per symbol

Exercise 1.4

Problem 5: Fixed-Length vs. Huffman Coding

Using the symbol set from Problem 4:

(a) Determine the minimum fixed-length code required

(b) Compute the total number of bits using fixed-length coding

(c) Compare the result with Huffman coding

(d) Calculate the percentage reduction in total bits achieved by Huffman coding

Exercise 1.5

Problem 6: Text Compression Scenario

A text file contains 50,000 characters and is stored using 8-bit ASCII encoding. After compression using a lossless algorithm, the file size becomes 18 KB. Calculate:

(a) Original file size in KB

(b) Compression ratio

(c) Compression factor

(d) Space savings percentage

Explain why compression ratios for text files vary significantly depending on content.

Exercise 1.6

Problem 7: Practical Design Question

You are designing a compression system for a wearable health-monitoring device that:

- Records sensor data continuously
- Has limited storage capacity
- Requires exact data reconstruction
- Operates on a low-power processor

(a) Should the system use lossless or lossy compression? Explain.

(b) Which performance metrics are most important in this scenario?

(c) Would a variable-length coding scheme be appropriate? Why or why not?

2: Lecture 2: Theory of Compression — Limits and Optimality

2.1 Types of Codes: From Ambiguous to Instantaneous

Definition

Types of Codes

- **Non-singular Code**: Each source symbol maps to a distinct codeword

$$x_i \neq x_j \Rightarrow C(x_i) \neq C(x_j)$$

- **Uniquely Decodable Code**: Every finite sequence of codewords corresponds to exactly one sequence of source symbols

$$C(x_1)C(x_2)\cdots C(x_n) = C(y_1)C(y_2)\cdots C(y_m) \Rightarrow n = m \text{ and } x_i = y_i$$

- **Prefix Code (Instantaneous Code)**: No codeword is a prefix of another codeword

$$\forall i \neq j: \quad C(x_i) \text{ is not a prefix of } C(x_j)$$

Key Relationships:

$$\text{Prefix Codes} \subset \text{Uniquely Decodable Codes} \subset \text{Non-singular Codes}$$

Important

Why Prefix Codes are Special

- **Instantaneous decoding**: Can decode as soon as codeword ends (no lookahead needed)
- **Tree representation**: Always correspond to leaves of a binary tree
- **Kraft inequality**: Always satisfy $\sum 2^{-\ell_i} \leq 1$
- **Practical**: Used in Huffman coding, many real-world compressors

Example

Example: Comparing Different Code Types

For symbols $\{A, B, C, D\}$ with probabilities $\{0.5, 0.25, 0.125, 0.125\}$:

Code Type	A	B	C	D	Property
Non-singular	0	1	00	11	Distinct but ambiguous: "00" = AA or C?
Uniquely decodable	0	01	011	0111	Unique but need lookahead
Prefix code	0	10	110	111	Instant decoding: "0" = A, stop
Optimal prefix	0	10	110	111	Also Huffman optimal

Decoding examples:

- **Prefix code "010110"**: 0→A, 10→B, 110→C = "ABC" (instant)
- **Uniquely decodable "00111"**: Need to scan ahead to determine split
- **Non-singular "00"**: Ambiguous! Could be "AA" or "C"

Key insight: Prefix codes sacrifice some flexibility in codeword lengths (must satisfy Kraft inequality) for the benefit of instantaneous decoding.

2.2 Basic Terminology and Notation

Definition

Alphabet

An *alphabet* $\mathcal{X}$ is a finite set of possible symbols. Examples:

- Binary alphabet: $\mathcal{X} = \{0, 1\}$
- English letters: $\mathcal{X} = \{\texttt{A}, \ldots, \texttt{Z}\}$
- Bytes: $\mathcal{X} = \{0, 1, \ldots, 255\}$

Definition

Symbol

A *symbol* is a single element drawn from an alphabet. For example, the letter `E` is a symbol from the English alphabet.

Definition

Random Variable

A *random variable* X is a function that assigns a symbol or value to each outcome in a sample space:

$$X : \Omega \to \mathcal{X}$$

- Ω: Sample space (e.g., all possible states of a data source)

- $\mathcal{X}$: Set of possible values (alphabet, e.g., $\{0, 1\}$, ASCII characters)
- For each $\omega \in \Omega$, $X(\omega)$ is the value assigned to outcome ω

Example: Binary Source

- $\Omega = \{\text{emits } 0, \text{emits } 1\}$ (or could be more complex underlying physics)
- $\mathcal{X} = \{0, 1\}$
- $X(\text{emits } 0) = 0$, $X(\text{emits } 1) = 1$
- Probabilities: $P(X = 0) = P(\{\omega : X(\omega) = 0\}) = p$, $P(X = 1) = 1 - p$

Why this matters for compression:

- The entropy $H(X)$ depends on the probability distribution induced by X
- For $x \in \mathcal{X}$: $P(X = x) = P(\{\omega \in \Omega : X(\omega) = x\})$
- $H(X) = -\sum_{x \in \mathcal{X}} P(X = x) \log_2 P(X = x)$

Definition

Source

A *source* is a process that generates a sequence of symbols $(X_1, X_2, X_3, \dots)$ according to some probability law. In this lecture, we assume discrete sources unless stated otherwise.

Definition

Message (or Sequence)

A *message* is a finite sequence of symbols generated by the source:

$$x^n = (x_1, x_2, \dots, x_n)$$

Compression algorithms operate on messages, not on individual symbols.

Definition

Code and Codewords

A *code* assigns a binary string (codeword) to each symbol or message.

- Source symbols $\to$ codewords (e.g., Huffman coding)
- Messages $\to$ bitstreams (e.g., arithmetic coding)

Definition

Block Length

The *block length* n is the number of source symbols grouped together and encoded as a unit. Larger block lengths generally allow better compression but increase delay and complexity.

Definition

Model

A *model* estimates the probabilities of symbols or sequences. Better models lead to better compression by reducing uncertainty.

2.3 Information and Redundancy: The Core Concepts

2.3.1 Information: A Formal Measure of Uncertainty Reduction

In information theory, information is defined rigorously as a **quantitative measure of the reduction in uncertainty** that results from observing the outcome of a random event.

Definition 2.1. *Let X be a random event that occurs with probability $p = \Pr(X)$. The* ***information content*** *(or* self-information*) $I(X)$ provided by the occurrence of X is defined as:*

$$I(X) = \log_b \left(\frac{1}{p}\right) = -\log_b(p)$$

where:

- $b = 2$ *yields* ***bits*** *(binary digits)*
- $b = e$ *yields* ***nats*** *(natural units)*
- $b = 10$ *yields* ***hartleys*** *or* ***dits***

Example

Predictability vs. Information:

- In a city where it rains every day, the statement "It rained today" conveys almost no information because it was expected
- A file that contains only the bit '1' provides very little information
- A coin that always lands heads produces outcomes, but no information

  Key idea: Perfect predictability implies zero information gain.

Example

Daily Weather Forecast — Information Content:

- Sunny in Phoenix (probability 0.9): $I = -\log_2 0.9 \approx 0.15$ bits
- Snow in Phoenix (probability 0.001): $I = -\log_2 0.001 \approx 9.97$ bits
- Rain in Seattle (probability 0.3): $I = -\log_2 0.3 \approx 1.74$ bits

Interpretation: Rare events carry more information because they reduce uncertainty the most.

2.3.2 Redundancy: The Enemy of Information and the Friend of Compression

Redundancy refers to predictable or repeated structure in data. It is what allows data to be represented using fewer bits.

1. **Spatial Redundancy**: Neighboring data values are highly correlated

   Example

   In a photograph of a clear blue sky, most neighboring pixels have nearly identical color values.

   - **Naive**: Store the RGB value of each pixel independently
   - **Smarter**: Encode repeated pixel values using run-length encoding
   - **Even smarter**: Predict each pixel from its neighbors and encode only the small prediction error

2. **Statistical Redundancy**: Some symbols occur far more frequently than others

   Example

   English letter frequencies:

Letter	Frequency	Letter	Frequency
E	12.7%	Z	0.07%
T	9.1%	Q	0.10%
A	8.2%	J	0.15%

   Frequent letters get shorter codes in variable-length coding schemes.

3. **Knowledge Redundancy**: Information already known to both encoder and decoder

4. **Perceptual Redundancy**: Information that humans cannot perceive

2.4 Entropy: The Fundamental Limit

2.4.1 What is Entropy? Different Perspectives

Definition

Shannon Entropy of a discrete random variable X with possible values $\{x_1, x_2, \ldots, x_n\}$ having probabilities $\{p_1, p_2, \ldots, p_n\}$:

$$H(X) = -\sum_{i=1}^{n} p_i \log_2 p_i \quad \text{bits}$$

Two Complementary Interpretations:

1. **Average Information Content**: Expected value of information content across all symbols
2. **Uncertainty or Surprise**: Measures how uncertain we are about the next symbol

2.4.2 Calculating Entropy: Step by Step

Example

Binary Source Example - Detailed Calculation:
Consider a biased coin: P(Heads) = 0.8, P(Tails) = 0.2

Step 1: **Calculate individual information content**:

$$I_H = -\log_2(0.8) \approx 0.3219 \text{ bits}$$
$$I_T = -\log_2(0.2) \approx 2.3219 \text{ bits}$$

Step 2: **Calculate entropy as expected value**:

$$H = 0.8 \times 0.3219 + 0.2 \times 2.3219 = 0.7219 \text{ bits}$$

Step 3: **Verify using direct formula**:

$$H = -[0.8 \log_2(0.8) + 0.2 \log_2(0.2)] \approx 0.7219 \text{ bits}$$

Key Insights:

- **Extreme cases**:

- Fair coin (P=0.5): $H = 1.0$ bit (maximum uncertainty)
- Always heads (P=1.0): $H = 0$ bits (no uncertainty)
- 90% heads: $H \approx 0.469$ bits

2.4.3 Entropy of English Text: A Practical Case Study

Example

Calculating English Letter Entropy:
Based on letter frequencies in typical English text:

$$H \approx 4.18 \text{ bits/letter}$$

Layered Interpretation:

- **First-order entropy (letters independent)**: 4.18 bits/letter
- **Actual uncertainty is lower**: Letters have dependencies (Q→U)
- **Comparison with encoding schemes**:

Encoding Method	Bits/Letter
Naive (5 bits for 26 letters)	5.00
Huffman (letter-based)	4.30
Using digram frequencies	3.90
Using word frequencies	2.30
Optimal with full context	∼1.50

2.4.4 Beyond First-Order Entropy: The Full Picture

Higher-Order Entropies quantify uncertainty while accounting for increasing context:

- **Zero-order entropy** (H_0): $H_0 = \log_2 |\mathcal{X}|$
- **First-order entropy** (H_1): $H_1 = -\sum p(x) \log_2 p(x)$
- **Second-order entropy** (H_2): $H_2 = -\sum p(x, y) \log_2 p(x|y)$
- **Nth-order entropy** (H_N): $H_N = -\sum p(x_1, \ldots, x_N) \log_2 p(x_N \mid x_1, \ldots, x_{N-1})$

2.4.5 Entropy Rate

The **entropy rate** of a source is defined as the limiting uncertainty per symbol when arbitrarily long contexts are available:

$$H_\infty = \lim_{N\to\infty} H_N$$

2.4.6 The Entropy Theorem: Why It Matters

Definition

Expected Code Length

For a source with symbols $\{x_1, x_2, \ldots, x_n\}$ having probabilities $\{p_1, p_2, \ldots, p_n\}$, and a code that assigns codeword lengths $\{\ell_1, \ell_2, \ldots, \ell_n\}$, the **expected code length** L is:

$$L = \mathbb{E}[\ell(X)] = \sum_{i=1}^{n} p_i \ell_i \quad \text{(bits per symbol)}$$

This measures the average number of bits needed to encode one symbol from the source.

Definition

Compression Ratio and Efficiency

For a source with entropy $H(X)$ and code with expected length L:

- **Compression ratio**: $\rho = \frac{\text{original bits}}{\text{compressed bits}}$
- **Efficiency**: $\eta = \frac{H(X)}{L} \leq 1$
- **Redundancy**: $R = L - H(X) \geq 0$

Perfect compression occurs when $\eta = 1$ (100% efficient) and $R = 0$.

Important

Shannon's Source Coding Theorem (1948)

For a discrete memoryless source with entropy H and any $\epsilon > 0$:

1. **Converse (Impossibility Result)**:
   No lossless coding scheme can achieve expected code length $L < H$.

$$\boxed{L \geq H \quad \text{for any uniquely decodable code}}$$

2. **Achievability (Possibility Result)**:
   There exists a lossless coding scheme (specifically, block coding with suffi-

ciently large block size n) such that:

$$\boxed{H \leq L < H + \epsilon}$$

Equivalently: For any $\epsilon > 0$, $\exists n$ such that:

$$\frac{L_n}{n} < H + \epsilon$$

where L_n is the expected length for blocks of size n.

Interpretation:

- **Entropy is the fundamental limit**: H bits/symbol is the best we can ever do
- **We can get arbitrarily close**: With clever coding, we can approach this limit as closely as desired
- **The gap is achievable**: The "$+\epsilon$" represents practical overhead that can be made arbitrarily small

Example

Understanding the Theorem with Numbers

Consider a binary source with $p(0) = 0.9$, $p(1) = 0.1$:

$$H = -0.9 \log_2 0.9 - 0.1 \log_2 0.1 \approx 0.469 \text{ bits/symbol}$$

- **Naive coding**: Use 1 bit per symbol $\rightarrow L = 1.0$, efficiency $\eta = 0.469/1.0 = 46.9\%$
- **Huffman coding**: 0→0, 1→1 (same as naive!) $\rightarrow L = 1.0$, $\eta = 46.9\%$ *Why so bad?* Because we're coding symbols individually.
- **Block coding (n=2)**: Code pairs of symbols:

$$\begin{aligned} 00 &\rightarrow 0 \quad (\ell = 1, p = 0.81) \\ 01 &\rightarrow 10 \quad (\ell = 2, p = 0.09) \\ 10 &\rightarrow 110 \quad (\ell = 3, p = 0.09) \\ 11 &\rightarrow 111 \quad (\ell = 3, p = 0.01) \end{aligned}$$

$L_2 = 0.81 \times 1 + 0.09 \times 2 + 0.09 \times 3 + 0.01 \times 3 = 1.29$ bits/block Per symbol: $L = L_2/2 = 0.645$ bits/symbol, $\eta = 0.469/0.645 \approx 72.7\%$

- **Block coding (n=3)**: Would get even closer to 0.469
- **Theoretical limit**: As $n \to \infty$, $L \to 0.469$

Key insight: The theorem tells us:

1. We can never beat 0.469 bits/symbol (impossibility)
2. We can get as close as we want to 0.469 bits/symbol (achievability)

Important

What the Theorem Does NOT Say

- **It doesn't say how to construct the code** - just that one exists
- **It doesn't guarantee practical implementation** - block size n might need to be huge
- **It doesn't account for computational complexity** - the code might be too complex to implement
- **It assumes we know the true probabilities** - in practice, we estimate them

Yet, this theorem is revolutionary because it:

1. Establishes a **fundamental limit** (like the speed of light in physics)
2. Provides a **benchmark** for evaluating compression algorithms
3. Guides algorithm design toward this limit

2.4.7 Key Takeaways

- Entropy measures both **average information** and **uncertainty**
- Higher-order models reduce entropy by exploiting dependencies
- The entropy rate represents the ultimate compression limit
- Shannon's theorem precisely separates the *possible* from the *impossible*

2.5 Entropy as a Lower Bound

2.5.1 The Fundamental Inequality

Theorem 2.2 (Entropy Lower Bound)**.** *For any* ***uniquely decodable*** *code C for source X:*

$$L(C) \geq H(X)$$

where $L(C) = \mathbb{E}[\ell(X)] = \sum_i p_i \ell_i$ is the expected code length.

Example

Binary Source with $p(0) = 0.9$, $p(1) = 0.1$

$$H = -0.9 \log_2 0.9 - 0.1 \log_2 0.1 \approx 0.469 \text{ bits/symbol}$$

Why we can't achieve 0.4 bits/symbol:

1. For 100 symbols, typical sequences: $2^{100 \times 0.469} \approx 2^{46.9}$
2. To encode all uniquely, need at least $2^{46.9}$ codewords
3. At 0.4 bits/symbol, total bits = 40
4. # codewords $\leq 2^{40} < 2^{46.9} \rightarrow$ impossible!

2.6 Kraft-McMillan Inequality: Core Theoretical Tool

2.6.1 Statement and Interpretation

Theorem 2.3 (Kraft-McMillan Inequality (Binary Case))**.** *Let $\ell_1, \ell_2, \ldots, \ell_m$ be the lengths of codewords in a* ***prefix code****. Then:*

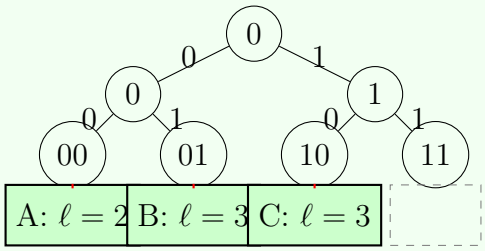
$$\sum_{i=1}^{m} 2^{-\ell_i} \leq 1$$

Conversely, if integers $\ell_1, \ldots, \ell_m$ satisfy this inequality, then there exists a binary prefix code with these lengths.

Example

Tree Visualization of Kraft Inequality

Consider a binary tree of depth $L = \max \ell_i$:



Calculating Kraft sum for $\{\ell_A = 2, \ell_B = 3, \ell_C = 3\}$:

$$\sum 2^{-\ell_i} = 2^{-2} + 2^{-3} + 2^{-3} = 0.25 + 0.125 + 0.125 = 0.5 \leq 1$$

Example

Testing Code Feasibility

1. **Valid lengths**: $\{1, 2, 3, 3\}$

$$\sum = 2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = 0.5 + 0.25 + 0.125 + 0.125 = 1.0 \quad \text{VALID}$$

2. **Invalid lengths**: $\{1, 1, 2\}$

$$\sum = 2^{-1} + 2^{-1} + 2^{-2} = 0.5 + 0.5 + 0.25 = 1.25 > 1 \quad \text{INVALID}$$

2.7 Optimality of Huffman Codes (Theory Only)

2.7.1 The Optimality Theorem

Theorem 2.4 (Huffman Optimality)**.** *Given a source with symbol probabilities $p_1, p_2, \ldots, p_m$, the Huffman algorithm produces a prefix code that* ***minimizes*** *the expected code length $L = \sum_{i=1}^{m} p_i \ell_i$ among all prefix codes.*

2.7.2 Relation to Entropy

For any Huffman code:

$$H(X) \leq L_{\text{Huffman}} < H(X) + 1$$

Example

Understanding the "+1" Gap

Consider source with probabilities $\{0.6, 0.3, 0.1\}$:

$$H \approx 1.295 \text{ bits}$$

Ideal (non-integer) lengths: $-\log_2 p_i = \{0.737, 1.737, 3.322\}$
Huffman code: 0.6→0, 0.3→10, 0.1→11

$$L = 0.6 \times 1 + 0.3 \times 2 + 0.1 \times 2 = 1.4 \text{ bits}$$

Comparison:

- Entropy: 1.295 bits
- Huffman: 1.400 bits
- Gap: 0.105 bits (much less than 1!)

2.7.3 Why Huffman is Optimal but Not Perfect

Important

Huffman is Optimal Within a Restricted Class
Huffman is optimal among:

- **Symbol-by-symbol** codes
- **Prefix** codes
- **Static** codes

But real optimality might require:

- **Block coding**
- **Fractional bits** (arithmetic coding)
- **Adaptive probabilities**

2.8 Limitations of Symbol-by-Symbol Coding

2.8.1 Three Fundamental Limitations

1. **Cannot Exploit Dependencies**
2. **Integer Length Constraint**: $\ell_i \in \mathbb{Z}^+$ but $-\log_2 p_i \in \mathbb{R}$
3. **Memoryless Assumption**

Example

English Text: The Cost of Symbol-by-Symbol

- **First-order entropy** (ignoring dependencies): 4.0 bits/letter
- **Actual entropy rate** (with dependencies): 1.5 bits/letter
- **Huffman on letters**: 4.0 bits/letter
- **Gap**: 2.5 bits/letter wasted due to ignoring dependencies

2.9 Block Coding and Improved Efficiency

2.9.1 The Block Coding Idea

Instead of coding symbols individually, group them into blocks of length n:

$$\mathbf{X} = (X_1, X_2, \ldots, X_n)$$

Definition

nth Extension of a Source

For a source with alphabet $\mathcal{X}$, the nth extension has alphabet:

$$\mathcal{X}^n = \{(x_1, \ldots, x_n) : x_i \in \mathcal{X}\}$$

with size $|\mathcal{X}|^n$.

2.9.2 Key Mathematical Results

Theorem 2.5 (Entropy of Block Source)**.** *For a discrete memoryless source:*

$$H(X^n) = nH(X)$$

Theorem 2.6 (Block Coding Performance)**.** *There exists a prefix code C_n for X^n such that:*

$$nH(X) \leq L_n < nH(X) + 1$$

Dividing by n:

$$H(X) \leq \frac{L_n}{n} < H(X) + \frac{1}{n}$$

Important

The Magic of Block Coding

As $n \to \infty$:

$$\frac{L_n}{n} \to H(X)$$

We can approach entropy **arbitrarily closely** by making blocks larger!

2.9.3 Step-by-Step Example

Example

Binary Source: $p(0) = 0.9$, $p(1) = 0.1$, $H \approx 0.469$

Step 1: $n = 1$ **(symbol-by-symbol)**

- Huffman: 0→0, 1→1
- $L_1 = 1$ bit/symbol
- Efficiency: $\eta = 0.469/1 = 46.9\%$

Step 2: $n = 2$ **(code pairs)**

- Block probabilities: $P(00) = 0.81$, $P(01) = 0.09$, $P(10) = 0.09$, $P(11) = 0.01$
- Codes: 00→0, 01→10, 10→110, 11→111
- Per symbol: $L_2/2 = 0.645$ bits/symbol
- Efficiency: $\eta = 0.469/0.645 = 72.7\%$

2.10 Trade-Offs in Block Coding

2.10.1 The Engineering Challenges

1. **Exponential Alphabet Growth**: $|\mathcal{X}^n| = |\mathcal{X}|^n$
2. **Memory Requirements**: Huffman tree has $2m^n - 1$ nodes
3. **Computational Complexity**: $O(m^n \log m^n)$
4. **Delay and Latency**: Must wait for n symbols

2.11 From Block Coding to Modern Compression

2.11.1 Arithmetic Coding: Fractional-Bit Block Coding

Important

Arithmetic Coding as "Infinite Block Coding"
Arithmetic coding cleverly avoids the exponential growth problem:

- **Idea**: Encode entire message as a single real number in [0,1)
- **No explicit blocks**: Processes symbols sequentially
- **Fractional bits**: Achieves $L \approx H(X)$ without large n
- **Removes integer constraint**: No "+1" overhead!

2.11.2 Context Modeling: Approximating Large Blocks

Instead of explicit block coding, modern compressors use:

1. **Context Models**: Predict next symbol based on previous k symbols
2. **Prediction + Residual Coding**: Encode only prediction error
3. **Dictionary Methods (LZ family)**: Build dictionary of previously seen phrases

2.12 What Theory Guarantees vs What Practice Achieves

Aspect	Theory Guarantees	Practice Achieves
Optimality	Can approach entropy arbitrarily closely	Gets close, but with practical limits
Block Size	$n \to \infty$ gives optimality	n limited by memory, latency, complexity
Complexity	Ignored, infinite resources allowed	Critical constraint; often dominates design

2.13 Summary and Key Takeaways

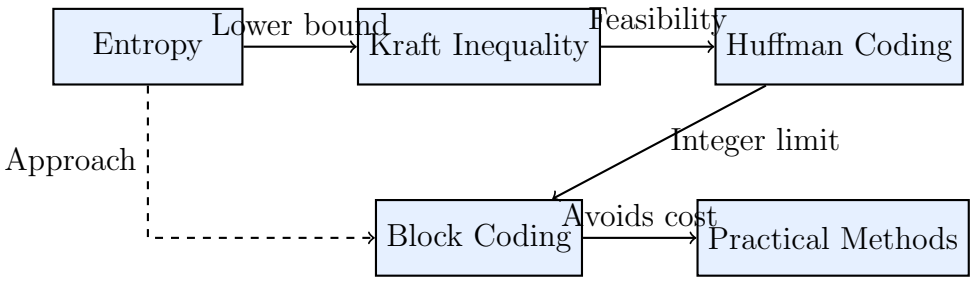
Important

Five Fundamental Lessons

1. **Entropy is the Absolute Limit**: $L \geq H(X)$ for any lossless code

2. **Kraft-McMillan Constrains All Codes**: $\sum 2^{-\ell_i} \leq 1$
3. **Huffman is Optimal Among Prefix Codes**: But limited by integer lengths
4. **Block Coding Allows Approaching Entropy**: $\lim_{n\to\infty} \frac{L_n}{n} = H(X)$
5. **Practical Compression Balances Efficiency and Complexity**

2.13.1 The Big Picture



2.13.2 Looking Forward

- **Next lecture: Arithmetic Coding**: Removes integer constraint
- **Then: Dictionary Methods (LZ family)**: Adaptive to data statistics
- **Finally: Modern Compressors**: Combining multiple techniques

Final Thought

Shannon's 1948 paper told us *exactly how good compression could possibly be.* Every compressor since has been trying to approach that limit while staying within practical constraints.

The gap between theory and practice is where engineering creativity lives!

3: Lecture 3: Advanced Entropy Coding & Extensions

Lecture 3: Beyond Huffman – Advanced Entropy Coding Methods

3.1 Introduction & Motivation

Important

Recall Huffman Coding Limitations:

- **Integer code lengths**: Cannot reach entropy bound for highly skewed distributions
- **Static vs. Adaptive**: Standard Huffman requires prior knowledge of probabilities
- **Codebook overhead**: Need to transmit/store the coding tree
- **Symbol-by-symbol constraint**: Processes one symbol at a time

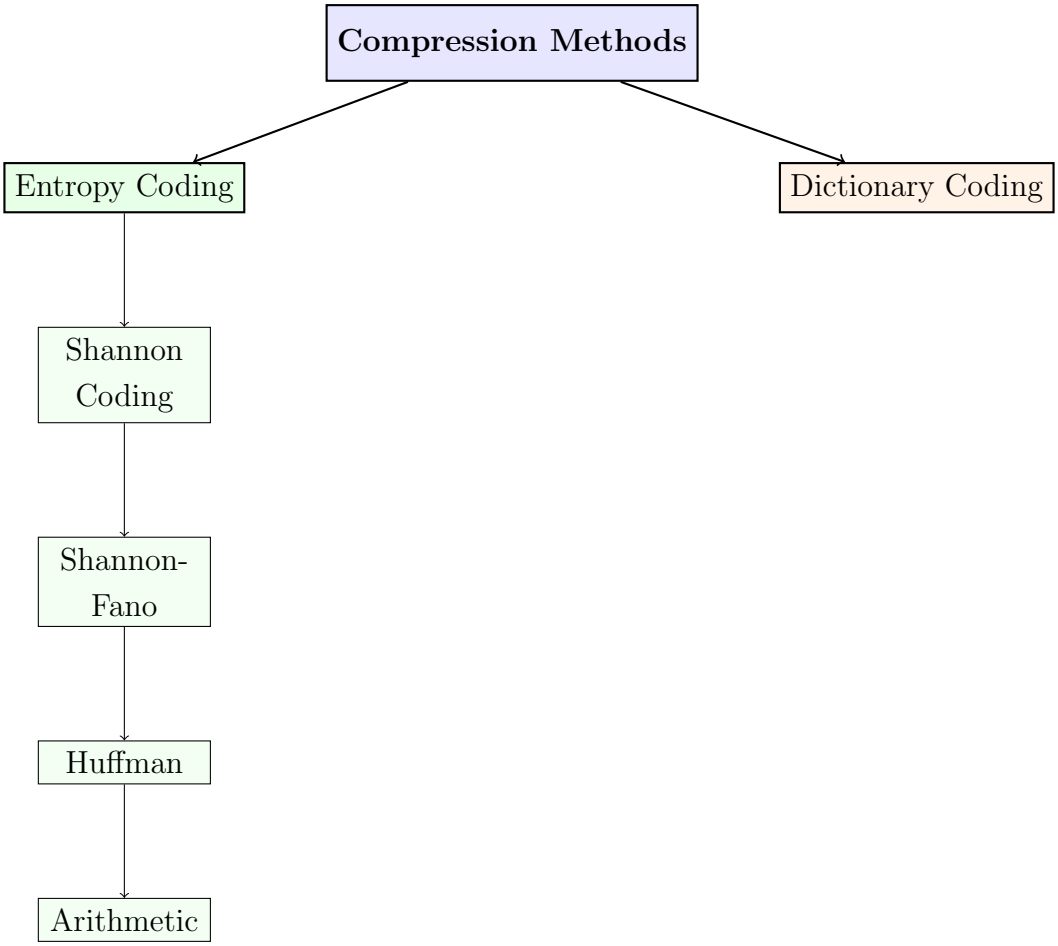
Lecture Roadmap:

1. **Framework**: Coding taxonomy and conceptual organization
2. **Historical methods**: Shannon & Shannon-Fano coding
3. **Practical improvements**: Canonical and Adaptive Huffman
4. **Next generation**: Arithmetic coding paradigm
5. **Synthesis**: Comparison and forward look

3.2 Coding Taxonomy & Framework

Definition

Coding Taxonomy: Classification of compression methods based on key characteristics



Key Dimensions in Compression Algorithm Design

1. **Modeling vs. Coding (Two-Stage View)**:
   - **Modeling Phase**: Analyzes data to estimate symbol probabilities or discover patterns.
     - *Examples*: Frequency counting (Huffman), context modeling (PPM), dictionary construction (LZ77)
   - **Coding Phase**: Converts modeled information into actual bits.
     - *Examples*: Huffman codes, arithmetic codes, LZ77 pointers
   - Some algorithms intertwine both (e.g., LZW builds dictionary while coding).
2. **Knowledge of Source Distribution**:
   - **Static/Fixed**: Uses a predefined model that doesn't change.
     - *Example*: JPEG Huffman tables, known language frequencies
     - Requires prior knowledge of data; fails if distribution differs.
   - **Adaptive**: Learns and updates the model during compression.
     - *Example*: Adaptive Huffman, LZ78 dictionary building

    - No prior knowledge needed; overhead for model transmission.
  - **Universal**: Can compress any source asymptotically optimally.
    - *Theoretical property*: LZ family, arithmetic with adaptive model
    - Note: Most adaptive methods are universal in practice.

3. **Processing Granularity**:
   - **Symbol-by-Symbol**: Each input symbol maps to one codeword.
     - *Example*: Huffman coding
     - Simple but limited to integer bits per symbol.
   - **Block Coding**: Fixed-size groups of symbols coded together.
     - *Example*: Block-sorting (BWT) processes blocks
     - Can capture inter-symbol dependencies within block.
   - **Stream/Incremental**: Continuous processing with immediate output.
     - *Example*: Arithmetic coding, LZ77 sliding window
     - No blocking delay; good for real-time applications.
   - *Note*: "Message-wide" (whole file as one symbol) is theoretical ideal; arithmetic coding approximates it by treating the entire stream as one long fractional code.

4. **Algorithmic Approach (Primary Taxonomy)**:
   - **Statistical Coding**: Uses probability estimates (Huffman, Arithmetic)
   - **Dictionary Coding**: Replaces repeated patterns with references (LZ family)
   - **Transform Coding**: Changes data domain then codes (DCT, wavelet)
   - **Predictive Coding**: Predicts next value, codes difference (DPCM, LPC)

Important Relationships:

- Adaptive methods are usually universal for practical purposes.
- Stream coding is possible with both symbol-by-symbol (Huffman) and message-wide approaches (arithmetic).
- Block processing (like BWT) is often followed by stream coding (like MTF+RLE+arithmetic in bzip2).
- Modeling and coding can be separated (PPM + arithmetic) or combined (LZW).

3.3 Shannon Coding (1948)

Definition

Shannon Coding: A constructive method derived from Shannon's source coding theorem that assigns codewords by taking the binary expansion of cumulative probabilities:

$$l_i = \lceil -\log_2 p_i \rceil \quad \text{and} \quad \text{code}_i = \text{First } l_i \text{ bits of } F_i$$

where p_i is the probability of symbol i, and $F_i = \sum_{j=1}^{i-1} p_j$ is the cumulative probability of symbols ordered by decreasing probability.

Shannon Coding Algorithm

Input: Symbols $S = \{s_1, s_2, \ldots, s_n\}$ with corresponding probabilities $P = \{p_1, p_2, \ldots, p_n\}$
Output: Prefix-free binary code for each symbol

1. **Sort** symbols in non-increasing order of probability: $p_1 \geq p_2 \geq \cdots \geq p_n$
2. **Calculate codeword lengths:** For each symbol i, compute $l_i = \lceil -\log_2 p_i \rceil$
3. **Compute cumulative probabilities:**

$$F_1 = 0, \quad F_i = \sum_{j=1}^{i-1} p_j \quad \text{for } i = 2, \ldots, n$$

4. **Generate codewords:** For each symbol i:
   a) Convert F_i to binary fractional representation (0.xxxx...)
   b) Take the first l_i bits after the binary point as the codeword

Example

Example: Given symbols with probabilities:

Symbol	Probability
A	0.5
B	0.25
C	0.125
D	0.125

Step-by-step construction:

1. **Sort by probability:** Already in non-increasing order ($0.5 \geq 0.25 \geq 0.125 \geq 0.125$)
2. **Calculate lengths:**

- $l_A = \lceil -\log_2 0.5 \rceil = \lceil 1.0 \rceil = 1$
- $l_B = \lceil -\log_2 0.25 \rceil = \lceil 2.0 \rceil = 2$
- $l_C = \lceil -\log_2 0.125 \rceil = \lceil 3.0 \rceil = 3$
- $l_D = \lceil -\log_2 0.125 \rceil = \lceil 3.0 \rceil = 3$

3. **Compute cumulative probabilities:**
   - $F_A = 0$ (first symbol)
   - $F_B = 0.5$ (just A's probability)
   - $F_C = 0.5 + 0.25 = 0.75$ (A + B)
   - $F_D = 0.5 + 0.25 + 0.125 = 0.875$ (A + B + C)
4. **Convert F_i to binary and take first l_i bits:**
   - **A:** $F_A = 0.0_{10}$ in binary is $0.0000\ldots_2$
     - Take first $l_A = 1$ bit: **0**
   - **B:** $F_B = 0.5_{10}$ in binary is $0.1000\ldots_2$
     - Take first $l_B = 2$ bits: **10**
   - **C:** $F_C = 0.75_{10}$ in binary is $0.1100\ldots_2$
     - Take first $l_C = 3$ bits: **110**
   - **D:** $F_D = 0.875_{10}$ in binary is $0.1110\ldots_2$
     - Take first $l_D = 3$ bits: **111**

Resulting code:

Symbol	Probability	l_i	F_i	Shannon Code
A	0.5	1	0.0	0
B	0.25	2	0.5	10
C	0.125	3	0.75	110
D	0.125	3	0.875	111

Expected length: $L = 0.5 \times 1 + 0.25 \times 2 + 0.125 \times 3 + 0.125 \times 3 = 1.75$ bits/symbol

Important

Critical Note on Sorting:
The sorting step is **essential** in Shannon coding because:

- Cumulative probabilities F_i depend on the order of symbols

- Sorting ensures intervals in [0,1) are assigned in decreasing size order
- Without sorting, the resulting code may not be prefix-free
- Sorting guarantees the length condition $\frac{1}{2^{l_i}} \leq p_i < \frac{1}{2^{l_i-1}}$ leads to non-overlapping intervals

Why it works: When probabilities are sorted, each symbol's interval of size p_i starts at F_i and ends at $F_i + p_i$. The codeword length ensures the binary expansion of F_i to l_i bits uniquely identifies the starting point without overlapping with neighboring intervals.

Understanding the Binary Expansion Process

When we write F_i in binary (e.g., $0.5 = 0.1_2$, $0.75 = 0.11_2$), we're essentially:

- Dividing the interval [0,1) into subintervals based on probabilities
- Each symbol gets an interval of size p_i
- The codeword is the **binary fraction** representing the **start** of that interval
- We use exactly $l_i = \lceil -\log_2 p_i \rceil$ bits, which ensures:

$$\frac{1}{2^{l_i}} \leq p_i < \frac{1}{2^{l_i-1}}$$

- This guarantees unique prefixes because intervals don't overlap!

Important

Properties of Shannon Coding:

- **Constructive proof**: Demonstrates that prefix codes exist for any lengths satisfying Kraft inequality
- **Simple to compute**: Direct from probabilities, no tree needed
- **Requires sorting**: Symbols must be ordered by decreasing probability first
- **Not optimal**: Unlike Huffman, doesn't minimize expected length (compare: Huffman would give A=0, B=10, C=110, D=111 **same in this case!**)
- **Theoretical importance**: Foundation for Shannon's source coding theorem
- **Efficiency bound**: $H(X) \leq L < H(X) + 1$ (like Shannon's theorem says)

Example

Counterexample: What happens if we don't sort?
Consider the same symbols but in different order: C (0.125), A (0.5), D (0.125), B (0.25)
Without sorting:

- $F_C = 0$, $l_C = 3 \rightarrow$ codeword: 000
- $F_A = 0.125$, $l_A = 1 \rightarrow$ codeword: 0 (binary: 0.001... $\rightarrow$ first bit is 0)
- $F_D = 0.625$, $l_D = 3 \rightarrow$ codeword: 101
- $F_B = 0.75$, $l_B = 2 \rightarrow$ codeword: 11

Problem: Codeword "0" (for A) is a prefix of "000" (for C) $\rightarrow$ **not prefix-free!**
This demonstrates why sorting is essential in Shannon coding.

3.4 Shannon–Fano Coding (1949)

Definition

Shannon–Fano Coding: A top-down, recursive source coding technique that assigns binary codewords by repeatedly partitioning a set of symbols into two subsets whose total probabilities are as close as possible. The method was developed independently by *Claude Shannon* and *Robert Fano* in 1949.

Algorithm Description

High-level idea: Symbols with higher probabilities should receive shorter codewords. This is achieved by repeatedly splitting the symbol set into two probability-balanced groups and assigning binary prefixes.

Step-by-step procedure:

1. **Sort** the symbols in decreasing order of probability:

$$p_1 \geq p_2 \geq \cdots \geq p_n.$$

2. **Recursive partitioning:**

   - If the current set contains only one symbol, stop (this is the base case).

- Find an index k that minimizes

$$\left|\sum_{i=1}^{k} p_i - \sum_{i=k+1}^{n} p_i\right|.$$

- This divides the symbols into two subsets:

$$S_1 = \{1, \ldots, k\}, \qquad S_2 = \{k+1, \ldots, n\}.$$

- Append bit **0** to the codewords of all symbols in S_1.
- Append bit **1** to the codewords of all symbols in S_2.
- Apply the same procedure recursively to S_1 and S_2.

Example with Six Symbols

Example

Example: Consider six symbols with the following probabilities.

Symbol	Probability	$-\log_2 p_i$
A	0.30	1.74
B	0.25	2.00
C	0.20	2.32
D	0.10	3.32
E	0.10	3.32
F	0.05	4.32

Construction process

Step 1: First split (balance 0.55 vs. 0.45)

- Sorted symbols: A(0.30), B(0.25), C(0.20), D(0.10), E(0.10), F(0.05)
- Best split: $\{A, B\}(0.55)$ and $\{C, D, E, F\}(0.45)$
- Prefix assignment: $\{A, B\} \to 0$, $\{C, D, E, F\} \to 1$

Step 2: Split $\{A, B\}$

- A: **00**, B: **01**

Step 3: Split $\{C, D, E, F\}$

- Best split: $\{C\}(0.20)$ and $\{D, E, F\}(0.25)$
- C: **10**, $\{D, E, F\} \to 11$

Step 4: Split $\{D, E, F\}$

- Best split: $\{D\}(0.10)$ and $\{E, F\}(0.15)$
- D: **110**, $\{E, F\} \to 111$

Step 5: Split $\{E, F\}$

- E: **1110**, F: **1111**

Final codes:

Symbol	Probability	Code	Length
A	0.30	00	2
B	0.25	01	2
C	0.20	10	2
D	0.10	110	3
E	0.10	1110	4
F	0.05	1111	4

Expected code length:

$$L = 2.40 \text{ bits/symbol.}$$

Entropy:

$$H(X) \approx 2.25 \text{ bits/symbol.}$$

Efficiency:

$$\frac{H(X)}{L} \approx 93.8\%.$$

Visual Tree Representation

```
                    ROOT
                   /    \
                 0/      \1
                 /        \
        {A,B} 0.55    {C,D,E,F} 0.45
           / \            / \
         0/   \1        0/   \1
         /     \        /     \
{A} 0.30  {B} 0.25  {C} 0.20  {D,E,F} 0.25
  00        01        10         / \
                                0/   \1
                                /     \
                            {D} 0.10  {E,F} 0.15
                              110        / \
                                       0/   \1
                                       /     \
                                   {E} 0.10  {F} 0.05
                                     1110      1111
```

Key Observations

- **Probability-balanced splits:** The algorithm focuses on balancing probabilities rather than the number of symbols.
- **Variable code lengths:** More probable symbols receive shorter codewords.
- **Prefix-free property:** No codeword is a prefix of another.
- **Near-optimal performance:** The efficiency is high but not guaranteed to be optimal.

Important

Limitations and Historical Context

- Shannon–Fano coding does *not* always produce the optimal code.
- Huffman coding (1952) guarantees the minimum average code length.
- Historically important as a precursor to Huffman coding.

3.5 Canonical Huffman Codes

Definition

Canonical Huffman Code: A standardized representation of a Huffman code in which:

- Codes are assigned in lexicographic (binary) order
- All codewords of the same length are consecutive binary numbers
- The first codeword of each length is the smallest possible binary value
- Only the code lengths are required to reconstruct the entire code

This enables compact storage and fast table-based decoding.

Why Canonical Huffman Codes?

A standard Huffman algorithm produces *optimal code lengths*, but the exact bit patterns depend on implementation details such as tie-breaking and tree construction:

- Different trees can yield the same optimal set of code lengths
- Different bit assignments, but identical compression performance

Canonical Huffman coding fixes **one unique and deterministic assignment** for a given set of code lengths so that:

1. Only code lengths need to be transmitted
2. The decoder can reconstruct the codes without ambiguity
3. Decoding can be implemented efficiently using lookup tables

Two-Step Process

Complete workflow:

1. **Run the standard Huffman algorithm** to obtain optimal code lengths l_i
2. **Apply the canonical transformation** to convert lengths into standardized codes

$$\boxed{\text{Symbol probabilities} \xrightarrow{\text{Huffman}} \text{Code lengths} \xrightarrow{\text{Canonical}} \text{Canonical codes}}$$

Canonical Huffman Construction Algorithm

Input: Code lengths l_i obtained from a standard Huffman algorithm **Output:** Canonical Huffman codes

1. **Sort symbols** in ascending order of code length l_i (primary key), and by symbol order or value (secondary key for tie-breaking)
2. **Count symbols per length:**
   - Let l_{min} and l_{max} be the minimum and maximum code lengths
   - For each length l, compute $\text{count}[l]$ = number of symbols with length l
3. **Compute starting codes:**
   - $\text{start_code}[l_{\text{min}}] = 0$
   - For $l = l_{\text{min}} + 1$ to l_{max}:

$$\text{start_code}[l] = (\text{start_code}[l-1] + \text{count}[l-1]) \times 2$$

4. **Initialize assignment pointers:**

$$\text{next_code}[l] = \text{start_code}[l] \quad \text{for all } l$$

5. **Assign codes to symbols** (in sorted order):
   - For each symbol i with length l_i:

- Assign next_code[l_i] as its code (converted to l_i-bit binary)
- Increment next_code[l_i] by 1

Understanding the Shift Operation: Why Multiply by 2?

The key step start_code[l] = (start_code[$l-1$] + count[$l-1$]) $\times$ 2 ensures:

Important

Visual Intuition: Imagine all codes of length $l-1$ are written as binary numbers. When we multiply by 2 (left shift by 1 bit), we:

- Append a 0 to each existing code
- Create room for new codes that are one bit longer
- Ensure the new codes maintain prefix-free property

Example: If the last code of length 2 is "11" (binary 3), then:

$$(\text{last code} + 1) \times 2 = (3 + 1) \times 2 = 8 = 1000_{\text{binary}}$$

The first code of length 3 is "100" (3 bits of 1000), which follows "11" properly.

Mathematical Explanation:

- start_code[$l-1$] + count[$l-1$] gives the **first unused code value** after all codes of length $l-1$
- Multiplying by 2 (left shift by 1 bit) makes room for an extra bit
- This ensures all codes of length l start with a 0 in their new bit position
- The result is the smallest valid starting point for the next length group

Bit Pattern Visualization:

```
Length 2 codes: 00, 01, 10, 11
Length 3 codes start at: (11 + 1) × 2 = 1000_binary
So length 3 codes are: 100, 101, 110, 111
Notice: 11 is NOT a prefix of 100!
```

Complete Worked Example

Example

Example: Suppose the Huffman algorithm produces the following code lengths:

Symbol	Length (l_i)
A	2
B	3
C	3
D	3
E	4
F	4

Key Point: The code lengths (l_i) are already determined by the Huffman tree construction. These lengths tell us *exactly* how many bits each symbol's code will use.

Step 1: Sort symbols by length (primary), then symbol order (secondary)

- **Critical:** Sorting ensures consistent code assignment
- Order: A (2), B (3), C (3), D (3), E (4), F (4)
- Within same length group, maintain original symbol order

Step 2: Count symbols per length

$$\text{count}[2] = 1, \quad \text{count}[3] = 3, \quad \text{count}[4] = 2$$

Step 3: Compute starting codes (with shift explanation)

$\text{start_code}[2] = 0$ (represented as 00 in binary, using **2 bits** because length=2)

$\text{start_code}[3] = (\text{start_code}[2] + \text{count}[2]) \times 2$

$= (0 + 1) \times 2 = 2$ (represented as 010 in binary, using **3 bits** because length=3)

Interpretation: After 1 code of length 2, shift left (add 0 bit) for length 3

$\text{start_code}[4] = (\text{start_code}[3] + \text{count}[3]) \times 2$

$= (2 + 3) \times 2 = 10$ (represented as 1010 in binary, using **4 bits** because length=4)

Interpretation: After 3 codes of length 3, shift left (add 0 bit) for length 4

Important: The multiplication by 2 is a **left shift** operation. It adds one more bit to the code while maintaining the prefix-free property.

Step 4: Initialize assignment pointers

$$\text{next_code}[2] = 0, \quad \text{next_code}[3] = 2, \quad \text{next_code}[4] = 10$$

Step 5: Assign codes (in sorted order)

Symbol	Length	Decimal	Binary Code	next_code after
A	2	0	`00`	1
B	3	2	`010`	3
C	3	3	`011`	4
D	3	4	`100`	5
E	4	10	`1010`	11
F	4	11	`1011`	12

How binary conversion works:

- For symbol A (length=2, decimal=0): Binary: 00 (padded to 2 bits)
- For symbol B (length=3, decimal=2): Binary: 010 (2 in binary is 10, padded to 3 bits: 010)
- For symbol E (length=4, decimal=10): Binary: 1010 (10 in binary is 1010, already 4 bits)

Algorithm Summary:

1. **Input**: Code lengths l_i for each symbol (from Huffman tree)
2. **For each length** l:
   - First code of length l: start_code[l]
   - Subsequent codes: increment by 1
   - All codes use exactly l bits
3. **Key formula**: start_code[$l + 1$] = (start_code[l] + count[l]) $\times$ 2

Final canonical codes (verify properties):

Symbol	Length	Canonical Code	Check
A	2	00	✓ First length 2 code
B	3	010	✓ First length 3 code
C	3	011	✓ Consecutive with B
D	3	100	✓ Consecutive with C
E	4	1010	✓ First length 4 code
F	4	1011	✓ Consecutive with E

Verification:

- All codes of same length are consecutive binary numbers ✓
- Codes are lexicographically ordered ✓
- No code is a prefix of another ✓

What If We Don't Sort? A Counterexample

Example

Problem without sorting: Suppose we have the same code lengths but assign codes without proper sorting:

Symbol	Length
D	3
A	2
E	4
B	3
F	4
C	3

If we assign codes in this random order using the same algorithm:

- D (length 3) gets code 010
- A (length 2) gets code 00
- E (length 4) gets code 1000
- **Problem:** Code "00" (A) is a prefix of "1000" (E)!

Conclusion: Sorting ensures that shorter codes are assigned first, preventing prefix conflicts between codes of different lengths.

Transmission and Decoding

Transmission format (very compact):

- Transmit only the sequence of code lengths in symbol order:

$$\langle l_1, l_2, \ldots, l_n \rangle$$

- Example for our symbols: $\langle 2, 3, 3, 3, 4, 4 \rangle$
- Each length can be represented using $\lceil \log_2 l_{\max} \rceil$ bits

Decoder reconstruction:

1. Read code lengths for all symbols (in original symbol order)
2. Sort symbols by (l_i, index) to match encoder's order
3. Reconstruct canonical codes using the same algorithm
4. Build decoding tables for fast lookup

Comparison with Standard Huffman

Standard Huffman (tree-based)	**Canonical Huffman (length-based)**
Output: Huffman tree structure	Output: Code lengths only
Must transmit tree (complex)	Transmit only lengths (simple)
Decoding by tree traversal (slower)	Decoding by table lookup (faster)
Multiple equivalent trees possible	Single deterministic assignment
Same optimal compression	Same optimal compression
More complex implementation	Simple, robust implementation
No sorting required	**Requires sorting by length**

Key Properties and Applications

- **Optimality:** Identical compression ratio to standard Huffman
- **Compactness:** Only code lengths stored/transmitted
- **Speed:** Table-based decoding is significantly faster
- **Standardization:** Used in DEFLATE (gzip/ZIP), JPEG, PNG, MPEG
- **Deterministic:** Same lengths always produce same codes

Important

Critical Insights:

1. Canonical Huffman is **not** a different compression algorithm from Huffman
2. The Huffman algorithm determines *how long* each code should be
3. The canonical transformation determines the *exact bit patterns* consistently
4. **Sorting by length** is essential to ensure prefix-free property
5. The $\times 2$ operation (left shift) ensures proper separation between different length groups

3.6 Adaptive Huffman Coding

Overview

Definition

Adaptive Huffman Coding is a single-pass, lossless data compression technique in which the Huffman model is updated dynamically as symbols are encoded and decoded. The Huffman tree evolves on-the-fly, allowing both the encoder and decoder to adapt to changing symbol statistics without any prior knowledge of the source distribution.

Unlike static Huffman coding, adaptive Huffman coding does not require a preprocessing phase to compute symbol frequencies. Instead, symbol statistics are learned incrementally as the data stream is processed.

Limitations of Static Huffman Coding

Static Huffman coding assumes that symbol statistics are known in advance:

- Requires two passes over the data:
  1. Collect symbol frequencies
  2. Encode using the constructed Huffman tree
- The Huffman tree (or code lengths) must be transmitted as side information
- Cannot adapt to non-stationary or evolving symbol distributions

These limitations motivate adaptive schemes when symbol statistics are unknown or change over time.

Key Principle of Adaptive Huffman Coding

Adaptive Huffman coding maintains a Huffman tree that is updated after encoding or decoding each symbol.

Key invariant: After processing each symbol, the encoder and decoder maintain *identical Huffman trees* by applying the same updates in the same order. This ensures correct decoding without transmitting the tree explicitly.

Initialization

The algorithm begins with a special symbol:

- A single **NYT** (Not Yet Transmitted) node

When a symbol appears for the first time:

- The code for the NYT node is output
- The raw symbol value is transmitted
- The NYT node is expanded into:
  - A new NYT node
  - A leaf node representing the new symbol

Note: Some variants preinitialize the tree if the alphabet is fixed, but the standard adaptive Huffman algorithm begins with only the NYT node.

Tree Update Mechanism

After each symbol is processed:

- The frequency (weight) of the corresponding leaf node is incremented
- The tree is updated to preserve the **sibling property**

Definition

Sibling Property: Nodes are numbered such that nodes with higher weights have higher numbers. For any given weight, all nodes with that weight form a contiguous block. Each internal node has exactly two children.

To restore this property, nodes may be swapped with others of equal weight, followed by incrementing parent node weights. This update process proceeds bottom-up toward the root.

Encoding and Decoding Process

- If the symbol has appeared before:
  - Output its current Huffman code
- If the symbol is new:
  - Output the code for the NYT node
  - Output the symbol in raw (fixed-length) form
- Update the tree identically at both encoder and decoder

Note: Exact bit patterns depend on the update order and implementation. Adaptive Huffman coding guarantees prefix-free optimality but not unique codes.

Algorithms

Two classical adaptive Huffman algorithms are widely used:

- **FGK Algorithm** (Faller–Gallager–Knuth)
- **Vitter's Algorithm (Algorithm V)**, which improves worst-case performance and is commonly preferred

Both algorithms maintain the sibling property while ensuring efficient updates.

Performance Characteristics

- Update cost is **amortized** $O(1)$ per symbol
- Worst-case update time is proportional to the tree height
- Compression efficiency:
  - Initially suboptimal
  - Converges toward entropy-optimal coding as statistics stabilize

Comparison with Static Huffman Coding

Feature	**Static Huffman**	**Adaptive Huffman**
Number of passes	Two	One
Prior statistics required	Yes	No
Model transmission	Required	Not required
Adaptation to changes	No	Yes
Initial efficiency	High	Low
Asymptotic efficiency	Optimal	Near-optimal

Applications

Adaptive Huffman coding is well suited for:

- Streaming data sources
- Real-time communication
- Situations where full statistics cannot be collected in advance

However, it may be less suitable when symbol distributions are known and stable, or when computational simplicity is critical.

Summary

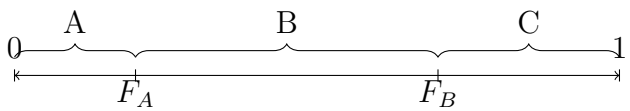
Adaptive Huffman coding extends classical Huffman coding to dynamic environments. By updating the coding model incrementally and maintaining strict structural invariants, it enables efficient, single-pass compression without prior knowledge of source statistics.

3.7 Arithmetic Coding: The Paradigm Shift

Definition

Arithmetic Coding: Encodes an entire message into a single fractional number in the interval $[0, 1)$, approaching the entropy bound very closely.

Core Idea: Represent messages as subintervals of $[0, 1)$:



Algorithm 1 Arithmetic Encoding Algorithm

Require: Message $m = s_1 s_2 \ldots s_k$, symbol probabilities p_i
Ensure: Final interval $[low, high)$

```
low ← 0.0, high ← 1.0
for each symbol s in m do
    range ← high − low
    high ← low + range × F_s          ▷ F_s: cumulative prob up to s
    low ← low + range × F_{s−1}       ▷ F_{s−1}: cumulative prob before s
end forreturn any number in [low, high)
```

Example

Example: Encode message "CAB" with probabilities: A(0.5), B(0.25), C(0.25)

Symbol	Probability	Cumulative
A	0.5	0.5
B	0.25	0.75
C	0.25	1.0

Encoding:

1. Start: $[0, 1)$
2. Process 'C': $[0.75, 1.0)$ (C occupies $[0.75, 1.0)$)
3. Process 'A': $range = 0.25$
   - $low = 0.75 + 0.25 \times 0.0 = 0.75$

- $high = 0.75 + 0.25 \times 0.5 = 0.875$
- New interval: $[0.75, 0.875)$

4. Process 'B': $range = 0.125$
   - $low = 0.75 + 0.125 \times 0.5 = 0.8125$
   - $high = 0.75 + 0.125 \times 0.75 = 0.84375$
   - Final interval: $[0.8125, 0.84375)$

Output: Any number in $[0.8125, 0.84375)$, e.g., 0.8125 in binary

Important

Practical Implementation Issues:

- **Finite precision**: Use integer arithmetic with scaling
- **Renormalization**: Output bits when interval confined to one half
- **Carry-over**: Handle when interval spans midpoint
- **Termination**: Need special end-of-message symbol

Adaptive Arithmetic Coding: Easier than adaptive Huffman - just update probabilities as you go!

3.8 Comparison & Synthesis

Method	**Optimal?**	**Adaptive?**	**Complexity**	**Near Entropy?**	**Key Applications**
Shannon Coding	No	No	Low	No	Theoretical proofs
Shannon-Fano	No	No	Low	Sometimes	Historical
Huffman	Yes*	No	Low	Moderate	General purpose
Canonical Huffman	Yes*	No	Low	Moderate	DEFLATE, JPEG, PNG
Adaptive Huffman	Yes*	Yes	Medium	Moderate	Early compressors
Arithmetic Coding	**Near-opt**	**Yes**	**High**	**Yes (close)**	**JPEG2000, H.264, HEVC**

*Optimal for symbol-by-symbol coding given probabilities

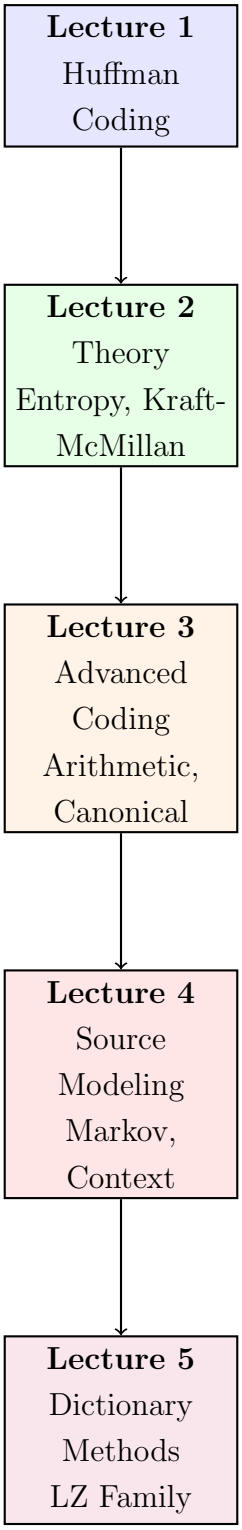
Important

Key Insights:

- **Huffman vs. Arithmetic**: Huffman is simpler but has an "integer penalty"; Arithmetic approaches entropy bound
- **Modern standards**: Arithmetic coding (CABAC) used in video compression for 10-20% better compression
- **Practical choice**: For general compression, Canonical Huffman (DEFLATE); for media, Arithmetic coding
- **The missing piece**: All these methods assume we have good probability estimates. Where do those come from?

3.9 Forward Look

The Complete Picture: What Comes Next?



Next Lecture: Source Modeling and Statistical Dependence

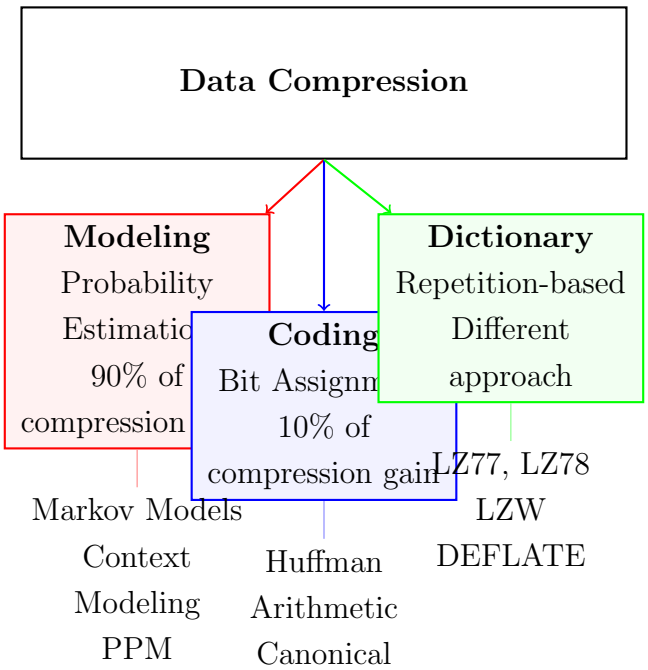
- **The missing half**: We now know how to code efficiently, but where do the probabilities come from?
- **Real data has memory**: 'Q' is usually followed by 'U' in English text
- **Markov models**: Capturing dependencies between symbols

- **Context modeling**: Using past symbols to predict future ones
- **The modeling-coding separation**: Modern compressors separate these two tasks

The Big Question for Next Time:

> **If Arithmetic coding can get within 0.01 bits of entropy,**
> **what's the real limit to compression?**
> **The answer: It's not the coding, it's the *modeling*!**

The Two Pillars of Compression (Revised View):



Exercise 3.0

Exercise 3.1: Given symbols with probabilities: A(0.4), B(0.3), C(0.2), D(0.1)

(a) Construct a Shannon code and compute its expected length

(b) Construct a Shannon-Fano code

(c) Compare with Huffman code from Lecture 2

Exercise 3.1

Exercise 3.2: Convert the following Huffman code to canonical form:

Symbol	Huffman Code
A	0
B	100
C	101
D	110
E	1110
F	1111

Exercise 3.2

Exercise 3.3: Encode the message "ABAC" using arithmetic coding with probabilities: A(0.6), B(0.3), C(0.1). Show each step.

Exercise 3.3

Exercise 3.4: Thinking Ahead: Consider the English phrase "THE QUICK BROWN FOX"

(a) If we use Huffman coding with letter frequencies, what's wrong with this approach?

(b) How might knowing that 'Q' is usually followed by 'U' help compression?

(c) Why would arithmetic coding be better than Huffman for this kind of data?

Exercise 3.4

Exercise: 3.5 Given code lengths: A=2, B=2, C=3, D=3, E=3, F=4

1. Sort the symbols properly
2. Compute the canonical codes
3. Verify no code is a prefix of another
4. What happens if you don't sort by length?

End of Lecture 3 – Advanced Entropy Coding Methods

Next: Lecture 4 – Source Modeling and Statistical Dependence

We now have efficient coding methods. Next: Where do the probabilities come from?

4: Lecture 4: Source Modeling and Statistical Dependence

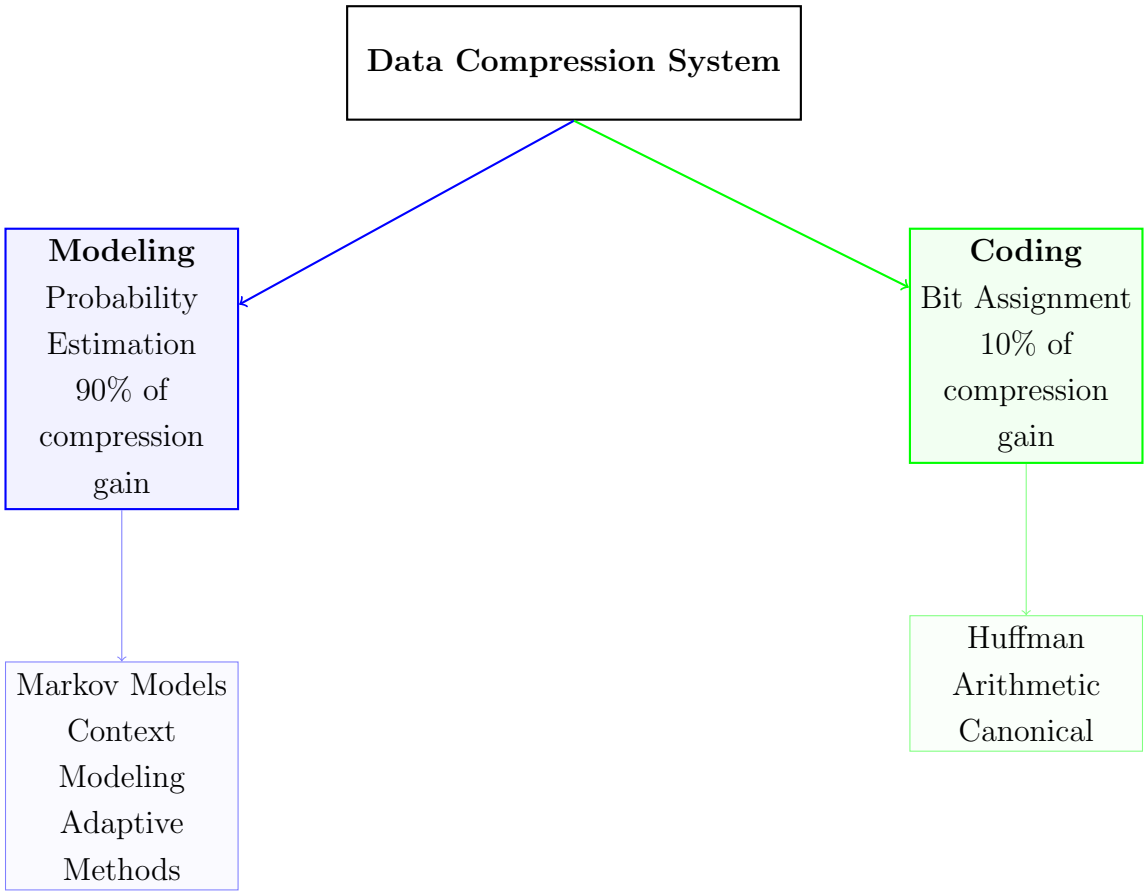
Lecture 4: Beyond Coding – The Power of Source Modeling

4.1 Introduction: Beyond Coding

Important

Key Insight: In the first three lectures, we've focused on **coding** - efficient ways to represent symbols given their probabilities. Now we address the other half: **modeling** - how to get good probability estimates in the first place.

The Big Picture:



Why Real Data Defies IID Assumptions:

- **IID (Independent Identically Distributed)**: Assumption behind simple Huffman
- **Reality**: Data has **memory** and **dependencies**
- Example: In English text, 'Q' is almost always followed by 'U'

- Example: In images, neighboring pixels are highly correlated

Today's Roadmap:

1. Understand statistical dependence in data
2. Learn Markov models for capturing memory
3. Explore context modeling techniques
4. See practical examples with real data
5. Connect modeling to coding (Lecture 3)

4.2 Memoryless vs. Sources with Memory

Definition

Memoryless Source (IID): Each symbol is generated independently of all previous symbols. Probability distribution: $P(X_n = x) = p(x)$ for all n.

Definition

Source with Memory: The probability of a symbol depends on previous symbols. Example: $P(X_n = x | X_{n-1} = y, X_{n-2} = z, \dots)$.

Example

Examples of Dependence in Real Data:

- **Text**: 'TH' is common, 'TQ' is rare
- **Images**: Neighboring pixels have similar colors
- **Audio**: Sound waves have temporal continuity
- **Video**: Consecutive frames are nearly identical
- **Source code**: Keywords, variable names repeat

Important

Measuring Dependence: Autocorrelation

$$\rho(k) = \frac{\mathbb{E}[(X_t - \mu)(X_{t+k} - \mu)]}{\sigma^2}$$

where k is the lag. High $\rho(k)$ means strong dependence at distance k.

4.3 Conditional Entropy and Mutual Information

Definition

Conditional Entropy: The average uncertainty about X given knowledge of Y:

$$H(X|Y) = -\sum_{x\in\mathcal{X}}\sum_{y\in\mathcal{Y}} p(x,y)\log_2 p(x|y)$$

Example

Intuition: "Knowing the Past Helps Predict the Future"
Consider English letters:

- Unconditional: $H(\text{letter}) \approx 4.07$ bits
- Given previous letter: $H(\text{letter}|\text{previous}) \approx 3.36$ bits
- Given previous 2 letters: $H(\text{letter}|\text{previous } 2) \approx 2.77$ bits

Each additional letter of context reduces uncertainty!

Definition

Mutual Information: Measures how much knowing Y reduces uncertainty about X:

$$I(X;Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$$

Example

Worked Example: English Letter Dependence
Let X = current letter, Y = previous letter.

$$\begin{aligned} H(X) &= 4.07 \text{ bits} \\ H(X|Y) &= 3.36 \text{ bits} \\ I(X;Y) &= 4.07 - 3.36 = 0.71 \text{ bits} \end{aligned}$$

This means knowing the previous letter gives us 0.71 bits of information about the current letter.

4.4 Markov Sources

Definition

Markov Property (Memory-m): The future depends only on the last m symbols:

$$P(X_n = x | X_{n-1}, X_{n-2}, \ldots, X_1) = P(X_n = x | X_{n-1}, \ldots, X_{n-m})$$

First-Order Markov Model ($m = 1$):

- Only the immediately previous symbol matters
- Represented by transition probabilities $p_{ij} = P(X_n = j | X_{n-1} = i)$
- Can be shown as a state diagram or transition matrix

Example

Example: Weather Prediction Markov Chain

States: $\{\text{Sunny (S)}, \text{Rainy (R)}\}$

Transition probabilities:

	S	R
S	0.8	0.2
R	0.3	0.7

Interpretation: If today is sunny, 80% chance tomorrow is sunny, 20% chance rainy.

Higher-Order Markov Models:

- Order-k: Depends on last k symbols
- More accurate but exponentially more parameters
- Number of parameters grows as $|\mathcal{A}|^{k+1}$ where $\mathcal{A}$ is alphabet size

Important

Memory-Complexity Trade-off:

- **Order 0**: 26 parameters for English (simple but weak)
- **Order 1**: $26 \times 26 = 676$ parameters
- **Order 2**: $26 \times 26 \times 26 = 17,576$ parameters
- **Order 5**: $26^6 \approx 308$ million parameters!

This is the **context explosion problem**.

4.5 Entropy Rate Revisited

Definition

Entropy Rate of a Stationary Source:
For a stationary stochastic process $\mathcal{X} = (X_1, X_2, \dots)$, the entropy rate is:

$$H(\mathcal{X}) = \lim_{n\to\infty} \frac{1}{n} H(X_1, X_2, \dots, X_n)$$

Equivalently (and equal for stationary processes):

$$H(\mathcal{X}) = \lim_{n\to\infty} H(X_n \mid X_1, \dots, X_{n-1})$$

Special case: Stationary Markov chain of order m
If $\mathcal{X}$ is a stationary m-th order Markov chain, then for all $t > m$:

$$H(X_t \mid X_1, \dots, X_{t-1}) = H(X_t \mid X_{t-m}, \dots, X_{t-1})$$

and this conditional entropy is constant over time. Therefore:

$$H(\mathcal{X}) = H(X_{m+1} \mid X_1, \dots, X_m)$$

No limit needed — the conditional entropy stabilizes immediately.

Example

Example: First-Order Markov Chain ($m = 1$)
Step 1: Define the model Two weather states: S (sunny) and R (rainy).
Transition probabilities (tomorrow's weather depends only on today's):
Let X_t be the weather on day t. Then:

$$P(X_{t+1} = S \mid X_t = S) = 0.8, \qquad P(X_{t+1} = R \mid X_t = S) = 0.2$$
$$P(X_{t+1} = S \mid X_t = R) = 0.3, \qquad P(X_{t+1} = R \mid X_t = R) = 0.7$$

In matrix form (rows = current state, columns = next state):

$$P = \begin{bmatrix} 0.8 & 0.2 \\ 0.3 & 0.7 \end{bmatrix} \qquad \begin{array}{l} \text{Row 1: } X_t = S \\ \text{Row 2: } X_t = R \end{array}$$

Step 2: Find stationary distribution π Stationary distribution $\pi = [\pi_S, \pi_R]$ satisfies $\pi P = \pi$, where $\pi_S = P(X_t = S)$ and $\pi_R = P(X_t = R)$ in the long run.

$$\begin{cases} \pi_S = 0.8\pi_S + 0.3\pi_R \\ \pi_R = 0.2\pi_S + 0.7\pi_R \\ \pi_S + \pi_R = 1 \end{cases}$$

Solving (from first equation):

$$\pi_S - 0.8\pi_S = 0.3\pi_R \Rightarrow 0.2\pi_S = 0.3\pi_R \Rightarrow \pi_S = 1.5\pi_R$$

$$1.5\pi_R + \pi_R = 1 \Rightarrow 2.5\pi_R = 1 \Rightarrow \pi_R = 0.4$$

$$\pi_S = 1.5 \times 0.4 = 0.6$$

$$\pi = [\pi_S, \pi_R] = [0.6, 0.4]$$

Step 3: Compute conditional entropies *Uncertainty about tomorrow given today's weather:*
Given $X_t = S$: Tomorrow's distribution is $[0.8, 0.2]$

$$H(X_{t+1} \mid X_t = S) = -0.8 \log_2 0.8 - 0.2 \log_2 0.2 \approx 0.7219 \text{ bits}$$

Given $X_t = R$: Tomorrow's distribution is $[0.3, 0.7]$

$$H(X_{t+1} \mid X_t = R) = -0.3 \log_2 0.3 - 0.7 \log_2 0.7 \approx 0.8813 \text{ bits}$$

Step 4: Compute entropy rate For a stationary first-order Markov chain:

$$H(\mathcal{X}) = H(X_{t+1} \mid X_t) = \sum_{s \in \{S,R\}} P(X_t = s) \cdot H(X_{t+1} \mid X_t = s)$$

With $P(X_t = s) = \pi_s$ (stationarity):

$$H(\mathcal{X}) = \pi_S \cdot H(X_{t+1} \mid X_t = S) + \pi_R \cdot H(X_{t+1} \mid X_t = R)$$

$$= 0.6 \times 0.7219 + 0.4 \times 0.8813 = 0.43314 + 0.35252 = 0.78566 \text{ bits}$$

$$\approx 0.786 \text{ bits per day}$$

Important

What This Means for Data Compression:
Two approaches to compression:

1. **IID (Independent Identically Distributed) assumption:**

- Treat each symbol as independent of previous ones
- Best achievable rate: $H(X)$ (marginal entropy)
- Example: English letters ≈ 4.07 bits/letter

2. **With modeling (using context):**
   - Account for dependencies between symbols (Markov structure)
   - Use conditional probabilities based on previous symbols
   - Best achievable rate: $H(\mathcal{X})$ (entropy rate)
   - Example: English text ≈ 2.3 bits/letter

Compression gain:

$$\frac{H(X) - H(\mathcal{X})}{H(X)} \approx \frac{4.07 - 2.3}{4.07} \approx 43\%$$

Up to $\sim 45\%$ better compression by exploiting dependencies!

4.6 Context Modeling in Practice

Definition

Context Modeling: Maintain separate probability distributions for each possible context (history).

Fixed-Length vs. Variable-Length Contexts:

- **Fixed-length**: Always use last k symbols as context
- **Variable-length**: Use longest matching context in database
- Example: PPM (Prediction by Partial Matching) uses variable-length

Example

The Context Explosion Problem:
For English text (26 letters + space):

Order	Contexts	Parameters
0	1	27
1	27	729
2	729	19,683
3	19,683	531,441
4	531,441	14,348,907
5	14,348,907	387,420,489

By order 5: 387 million parameters need estimation!

Solutions to Context Explosion:

1. **Escaping**: Fall back to lower-order model when context unseen
2. **Blending**: Combine predictions from different order models
3. **Pruning**: Remove low-frequency contexts
4. **Adaptive methods**: Update probabilities as data arrives

4.7 Case Study: Text Compression Modeling

Example

1. Entropy Estimates for English Text (Theoretical Limits)

Model/Estimation Method	Bits/Letter	Year/Source
Letter frequencies only (Order-0)	4.07	Shannon 1948
+ 1st order Markov (bigrams)	3.36	
+ 2nd order (trigrams)	2.77	
+ 3rd order	2.43	
Human prediction experiment	1.3	Shannon 1951
Modern best estimates	1.0–1.5	Various studies

2. Practical Compression Performance

Compressor/Method	Bits/Letter	Notes
gzip (LZ77 + Huffman)	2.5–3.0	Fast, widely used
bzip2 (BWT + MTF + Huffman)	2.3–2.8	Better but slower
PPMd	2.1–2.4	Context modeling
PAQ variants	1.5–1.8	State of the art, very slow

Key insight: Better models (using more context) $\rightarrow$ lower entropy $\rightarrow$ better compression.
But practical compressors face tradeoffs: memory, speed, and model complexity.

PPM (Prediction by Partial Matching):

- Uses **variable-length** contexts
- Tries highest-order model first
- Escapes to lower order if context unseen
- Blends probabilities from different orders
- State-of-the-art for text compression in 1990s

Important

Practical Entropy Reduction:

$$\begin{aligned} \text{IID model (Huffman)} &: 4.07 \text{ bits/letter} \\ \text{With context modeling} &: 2.23 \text{ bits/letter} \\ \text{Improvement} &: 45\% \text{ better compression!} \end{aligned}$$

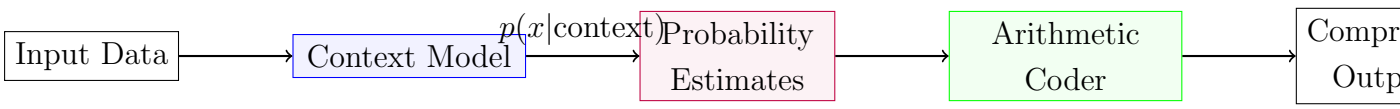
4.8 The Modeling–Coding Separation Principle

Definition

Modeling–Coding Separation: Modern compressors separate probability estimation (modeling) from bit assignment (coding).

Historical Evolution:

- **Early**: Integrated (Huffman builds tree from frequencies)
- **Modern**: Separated (Model → Probabilities → Arithmetic Coder)



Example

How PPM + Arithmetic Beats Huffman:

1. **PPM**: Sees context "TH" → predicts E with 80% probability
2. **Arithmetic**: Encodes E using $p = 0.8 \rightarrow$ 0.32 bits
3. **Huffman**: Would need at least 1 bit for any symbol

4. **Gain**: 0.32 bits vs 1+ bits = 3× better for this symbol!

Important

Why Arithmetic Coding is the Perfect Backend:

- Can handle **fractional bits** per symbol
- Accepts **changing probabilities** symbol by symbol
- Works with **adaptive models** naturally
- Achieves entropy bound for good models

4.9 Adaptive vs. Static Modeling

Definition

Static Models: Train once on representative data, use fixed model for all files.

- **Pros**: Fast encoding/decoding
- **Cons**: Model may not match specific file
- **Example**: Early text compressors using English statistics

Definition

Semi-Adaptive (Two-Pass): First pass: collect statistics; Second pass: encode.

- **Pros**: Tailored to specific file
- **Cons**: Need to transmit model (overhead)
- **Example**: Standard Huffman with tree transmission

Definition

Fully Adaptive (One-Pass): Update model while encoding/decoding.

- **Pros**: No model transmission, adapts to local changes
- **Cons**: Slower, initial poor compression
- **Example**: Adaptive Huffman, PPM with update

Example

Comparison in Practice:

Type	Compression	Speed	Memory
Static	Medium	Fast	Low
Semi-Adaptive	Good	Medium	Medium
Fully Adaptive	Best	Slow	High

Choice depends on application constraints!

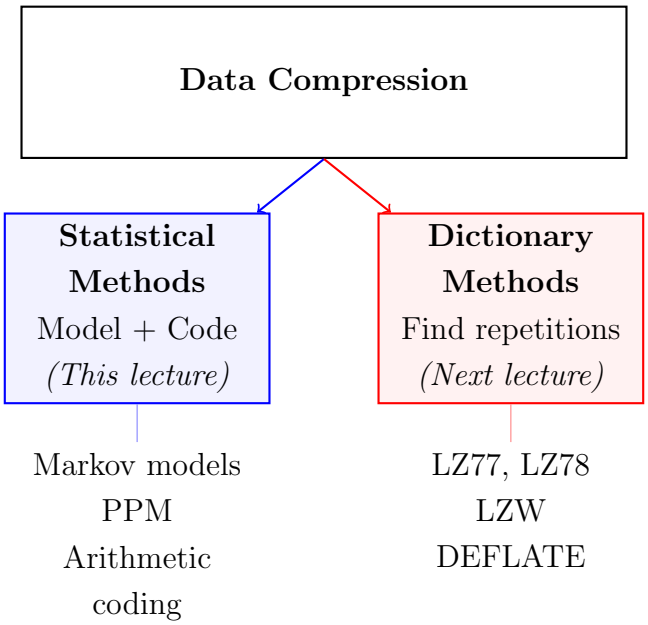
4.10 Summary and Forward Look

Important

Key Takeaways:

1. **The real compression is in modeling**, not just coding
2. **Context matters**: Using past symbols reduces uncertainty
3. **Markov models** capture memory in data
4. **Context explosion** limits practical model order
5. **Arithmetic coding** enables efficient use of good models
6. **Adaptive methods** avoid model transmission overhead

The Two Pillars Revisited:



Preview: Next Lecture on LZ Family (Dictionary Methods):

- A completely different approach: find repeating patterns
- No probability estimation needed
- Works well for files with exact repetitions
- Used in ZIP, GIF, PDF, and many modern formats
- Often combined with statistical methods in practice

Exercise 4.0

Exercise 4.1: Given the first-order Markov chain for weather:

	S	R
S	0.7	0.3
R	0.4	0.6

a) Find the stationary distribution $\pi = [\pi_S, \pi_R]$ b) Calculate the entropy rate $H(\mathcal{X})$ c) How does this compare to an IID source with $P(S) = 0.55, P(R) = 0.45$?

Exercise 4.1

Exercise 4.2: Consider the text fragment: "THE CAT SAT ON THE MAT" a) Build an order-1 (bigram) model for this text b) Calculate $H(X)$ (order-0 entropy) c) Calculate $H(X|Y)$ where Y is previous letter (order-1 conditional entropy) d) How much mutual information exists between consecutive letters?

Exercise 4.2

Exercise 4.3: Explain why arithmetic coding is better than Huffman coding when used with: a) A high-order Markov model b) An adaptive context model c) A model that gives very skewed probabilities (e.g., $p = 0.99$ for one symbol)

End of Lecture 4 – Source Modeling and Statistical Dependence

Next: Dictionary-based Compression (LZ Family)

5: Lecture 5: Dictionary-Based Compression: The Lempel–Ziv Revolution

5.1 Motivation: Hitting the Limits of Statistical Coding

Statistical coding methods such as Huffman and arithmetic coding achieve near-optimal compression *when the source statistics are known or can be accurately estimated.* However, these methods fundamentally rely on estimating probabilities over symbols or blocks of symbols.

As discussed earlier, block-based statistical coding faces a fundamental dilemma.

5.1.1 Recap: The Block Coding Dilemma

Increasing block length allows a coder to better capture dependencies between symbols and approach the entropy rate of the source. However, this comes at a steep cost:

- The number of possible blocks grows exponentially with block length.
- Estimating probabilities reliably requires exponentially more data.
- Memory and computational complexity quickly become impractical.

As a result, practical statistical coders are constrained to small contexts and local dependencies.

5.1.2 The Promise of Exploiting Long-Range Repetition

Real-world data—text, executable files, logs, DNA sequences—often contains *long-range repetition.* Patterns may repeat far apart, well beyond the reach of fixed-size statistical contexts.

Important

Statistical coding models *how often* symbols occur, but does not directly model *where long repeated patterns occur.*

This observation motivates a radically different approach to compression.

5.2 Paradigm Shift: From Statistics to Dictionaries

Instead of estimating probabilities, dictionary-based compression learns the source *by example.*

5.2.1 Core Philosophy of Dictionary Coding

Dictionary-based coders operate on a simple idea:

> *Replace repeated substrings by references to earlier occurrences.*

As the input is processed sequentially, a dictionary of previously seen substrings is constructed. Future occurrences are encoded by references into this dictionary.

Important

You can think of Lempel–Ziv methods as *learning the source structure rather than estimating probabilities.*

Crucially, the encoder and decoder process the input in exactly the same order and therefore build identical dictionaries *without transmitting the dictionary explicitly.*

5.2.2 Explicit vs. Implicit (Adaptive) Dictionaries

Dictionary-based methods fall into two categories:

- **Implicit dictionaries**: The dictionary is defined by a sliding window of recent output (e.g., LZ77, LZSS).
- **Explicit dictionaries**: The dictionary is stored and indexed explicitly (e.g., LZ78).

5.3 The Lempel-Ziv Algorithms

5.3.1 LZ77: The Sliding Window Algorithm

Published in 1977 by Abraham Lempel and Jacob Ziv, LZ77 introduced the concept of using recent history as a dictionary. It forms the foundation for many practical compression algorithms including gzip, PNG, and ZIP.

The Sliding Window Model: LZ77 maintains a sliding window divided into two parts:

- **Search buffer**: Contains the N most recently processed symbols (the "dictionary").
- **Look-ahead buffer**: Contains the next L symbols to be encoded.

The Encoding Tuple: (Offset, Length, Next Symbol): At each step, the encoder searches for the longest string in the search buffer that matches the beginning of the look-ahead buffer. It outputs a triple:

$$(\text{offset}, \text{length}, \text{next symbol})$$

5.3.2 LZSS: Improving Practical Efficiency

Published in 1982 by James Storer and Thomas Szymanski, LZSS addresses a key inefficiency in LZ77.

Flag Bits: Literal vs. Match: LZSS introduces a **flag bit** preceding each encoded element:

- **Flag = 0**: Next item is a literal symbol (8 bits for ASCII).
- **Flag = 1**: Next item is a match pair (offset, length).

Match Threshold: LZSS introduces a **match threshold**: only encode matches that are *long enough* to actually save bits.

5.3.3 LZ78: The Dictionary Growth Algorithm

Published in 1978, LZ78 takes a different approach: it builds an explicit dictionary that grows as encoding proceeds.

Encoding Pairs: (Dictionary Index, New Symbol): LZ78 outputs pairs:

$$(\text{index}, \text{symbol})$$

where the new phrase (dictionary[index] + symbol) is added to the dictionary.

5.3.4 LZW (Lempel-Ziv-Welch)

LZW (1984) is a popular LZ78 variant used in GIF, TIFF, and UNIX compress.

LZW Key Features:

- **No explicit next symbol**: Dictionary entries are added *before* they're used.
- **Fixed-width codes**: Typically 12-bit codes (4096 dictionary entries).
- **Adaptive dictionary**: Starts with all single symbols (256 entries for bytes).

5.4 Theoretical Properties: Why Lempel–Ziv Works

5.4.1 The Universality Principle

Lempel–Ziv algorithms have a remarkable theoretical property:

Definition

Universal Compression: An algorithm is universal if it can compress any stationary ergodic source to its entropy rate *without prior knowledge* of the source statistics.

Both LZ77 and LZ78 are universal compressors.

5.4.2 Asymptotic Optimality

Theorem 5.1 (Ziv & Lempel, 1977-1978)**.** *For any stationary ergodic source with entropy rate H, let L_n be the length of the LZ77 or LZ78 encoding of n source symbols. Then:*

$$\lim_{n\to\infty} \frac{\mathbb{E}[L_n]}{n} = H$$

5.5 Bridging Paradigms: Dictionary vs. Entropy Coding

5.5.1 Complementary Approaches

Dictionary coding and entropy coding address different aspects of compression:

- **Dictionary coding**: Exploits *structural redundancy* (repetitions).
- **Entropy coding**: Exploits *statistical redundancy* (skewed symbol frequencies).

5.5.2 The Hybrid Approach

Modern compressors combine both approaches. The DEFLATE algorithm (used in ZIP, PNG, gzip) works this way:

1. LZSS finds repeated strings, outputs literals and matches.
2. Huffman coding compresses:
    - Literal symbols (0-255)
    - Match lengths (257-285)
    - Match distance codes (0-29)

5.6 Summary and Forward Look

5.6.1 Key Takeaways

- **Paradigm shift**: Dictionary coding exploits repetition rather than probabilities.
- **LZ77**: Sliding window approach, encodes matches as triples.
- **LZSS**: Practical improvement with flag bits and match thresholds.
- **LZ78**: Explicit dictionary growth, encodes pairs.
- **Universality**: LZ methods work on any stationary source without prior knowledge.
- **Modern practice**: Hybrid systems (LZSS + Huffman) dominate.

End of Chapter Exercises

Exercise 5.0

Exercise 5.1 (LZ77 Encoding)
Encode the string `TOBEORNOTTOBEORTOBEORNOT` using LZ77 with a search buffer of size 12. Show your work step by step with the search buffer, look-ahead buffer, and output tuples.

Exercise 5.1

Exercise 5.2 (LZSS Efficiency Analysis)
Compare LZ77 and LZSS for encoding a sequence with parameters: 8-bit symbols, 12-bit offsets, 4-bit lengths.

(a) How many bits does LZ77 use for a non-match?

(b) How many bits does LZSS use for a literal?

(c) What is the minimum match length that saves bits in LZSS?

(d) When would LZ77 be more efficient than LZSS?

Exercise 5.2

Exercise 5.3 (LZ78 Dictionary Growth)
Encode the string `ABABABABA` using LZ78. Show the dictionary after each step and the complete output sequence. How many dictionary entries are created?

Exercise 5.3

Exercise 5.4 (Algorithm Comparison)
For each string below, predict which would perform better: LZ77/LZSS or LZ78? Explain why.

(a) `AAAAAAAAAAAAAAAAAAAA` (20 A's)

(b) `ABCDEFGHIJKLMNOPQRSTUVWXYZ`

(c) `ABABABABABABABABABAB`

(d) A file containing the same 1KB block repeated 100 times

Exercise 5.4

Exercise 5.5 (Decoding Challenge)
Decode the following LZSS encoded sequence (match threshold = 3):
`0 T`

```
0 H
0 E
1 (3,5)
0 _
0 _
1 (5,4)
0 .
```

Where: - Each line represents one encoded element - Flag 0 indicates a literal character - Flag 1 indicates a match (offset, length) - _ represents space character - . represents period

What is the original string? Show your decoding step by step.

Exercise 5.5

Exercise 5.6 (Window Size Analysis)

Consider LZ77 with search buffer size N.

(a) How many bits are needed to encode an offset value?

(b) If we increase N from 4096 to 65536, how many more bits are needed for offsets?

(c) What is the trade-off in choosing N?

(d) In practice, why might we choose $N = 32768$ over $N = 65536$ even if memory is available?

Exercise 5.6

Exercise 5.7 (Universal Compression Intuition)

Explain in your own words why Lempel–Ziv algorithms are universal. Why don't they need to know source statistics in advance? How do they "learn" the source structure?

Exercise 5.7

Exercise 5.8 (Hybrid Coding Design)

Design a simple hybrid compressor that:

(a) Uses LZSS with match threshold 3

(b) Collects statistics on literals, lengths, and offsets

(c) Applies Huffman coding to each alphabet separately

How would the decoder know which Huffman trees to use?

Exercise 5.8

Exercise 5.9 (Search Efficiency)
A naive LZ77 implementation searches for matches by comparing the look-ahead buffer against every position in the search buffer.

(a) What is the time complexity of this approach?

(b) Describe how a hash table can improve search efficiency.

(c) What information would you store in the hash table?

(d) How would you handle hash collisions?

Exercise 5.9

Exercise 5.10 (Theoretical Limits)
Prove or provide intuition for:

(a) LZ77 cannot achieve compression ratio better than the entropy rate for i.i.d. sources.

(b) LZ78 may perform poorly on very short inputs.

(c) For sources with long-range dependencies, LZ methods can outperform block Huffman coding even with large blocks.

6: Lecture 6: DEFLATE Algorithm: The Standard in Practice

6.1 Introduction: The Universal Compression Standard

The DEFLATE compression algorithm, specified in RFC 1951 (1996), is arguably the most widely deployed compression algorithm in history. Its success lies not in being the theoretically optimal algorithm, but in striking an exceptional balance between compression ratio, speed, and implementation simplicity.

Definition

DEFLATE: A lossless data compression algorithm combining LZ77-style string matching with Huffman coding. It is the compression engine behind gzip, ZIP, PNG, and HTTP compression.

6.1.1 Where DEFLATE Is Used: gzip, ZIP, PNG, HTTP

- **gzip**: The Unix compression utility (RFC 1952)
- **ZIP file format**: Phil Katz's PKZIP format (1989), later standardized
- **PNG image format**: Replaced GIF, uses DEFLATE on filtered scanlines
- **HTTP compression**: Content-Encoding: gzip/deflate in web traffic
- **PDF documents**: FlateDecode filter for stream compression
- **Java JAR files**: Based on ZIP format

6.1.2 Historical Context: Phil Katz, ZIP Format, and Open Standards

The story of DEFLATE is intertwined with the "compression wars" of the late 1980s-1990s:

- **1985**: Terry Welch's LZW algorithm patented (used in GIF)
- **1986**: Phil Katz creates PKARC (based on ARC format)
- **1989**: Katz releases PKZIP with proprietary compression
- **1991**: gzip created as free alternative to UNIX compress (which used LZW)
- **1993**: Unisys enforces LZW patents, creating demand for patent-free alternatives
- **1996**: DEFLATE specification published as RFC 1951

The key innovation was making DEFLATE specification **open** and **royalty-free**, leading to widespread adoption.

6.2 High-Level Architecture: A Two-Stage Pipeline

DEFLATE employs a classic transform-coding approach:

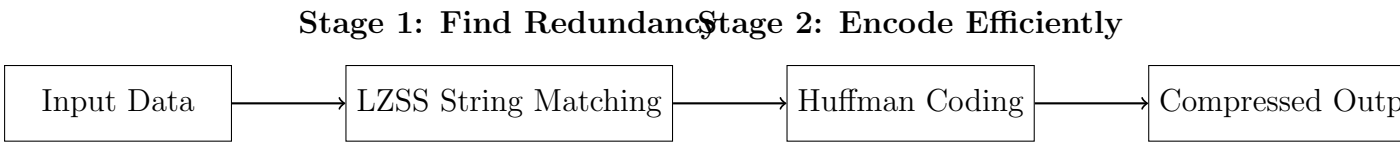


Figure 1: DEFLATE's two-stage compression pipeline

6.2.1 Stage 1: LZSS String Matching (Finding Redundancy)

The LZSS (Lempel-Ziv-Storer-Szymanski) stage identifies repeated sequences in the data:

- Scans input with a **sliding window** (32KB history buffer)
- For each position, looks for longest match in recent history
- Outputs either:
  - **Literal byte**: When no good match found
  - **(Length, Distance) pair**: When match found

Example: For the string "the cat sat on the mat":

- First occurrence of "the": output as literals 't','h','e'
- Second occurrence of "the": output as (length=3, distance=15)

6.2.2 Stage 2: Huffman Coding (Encoding the Reduced Data)

The Huffman stage compresses the output from Stage 1:

- Takes stream of literals and (length,distance) pairs
- Builds **two Huffman trees**:
  - One for literals and lengths (combined alphabet)
  - One for distances
- Uses **canonical Huffman codes** for compact representation

6.2.3 Why the Hybrid Approach Works: Transforming Redundancy

The genius of DEFLATE is in how it **transforms redundancy**:

1. **LZSS finds structural redundancy**: Repeated sequences become (length,distance) pairs
2. **Huffman finds statistical redundancy**: Frequent symbols get shorter codes
3. Together they exploit **both types** of redundancy in data

Important

Key Insight: LZSS is excellent at finding *long-range* repetition (phrases, patterns). Huffman is excellent at compressing *skewed distributions*. Together they handle both English text (skewed letter frequencies + repeated words) effectively.

6.2.4 Design Philosophy: Fast Decoding Over Maximum Compression

DEFLATE was designed with clear priorities:

Priority	Implementation Choice
Fast decoding	Canonical Huffman (O(1) decode)
Memory efficiency	32KB sliding window (tiny by today's standards)
Implementation simplicity	Two independent stages
Streaming capability	Independent compression blocks
Acceptable compression	"Good enough" ratio for most data

This "fast decode" philosophy made DEFLATE ideal for distribution formats (ZIP, PNG) where files are compressed once but decompressed many times.

6.3 LZSS in DEFLATE: Detailed Implementation

6.3.1 Sliding Window: 32KB History and Lookahead Buffer

DEFLATE's LZSS uses specific, fixed parameters:

- **History buffer**: 32,768 bytes (2^{15}) of recently processed data
- **Lookahead buffer**: 258 bytes maximum match length
- **Minimum match length**: 3 bytes (shorter matches not worth encoding)

Example

Why 32KB?

- 32KB was a practical limit in 1990s memory-constrained systems
- Larger windows provide diminishing returns for most data
- Cache-friendly size for modern processors
- Still adequate for most short-to-medium range repetitions

6.3.2 Hash Chains for Fast Match Finding

Instead of brute-force searching 32KB for matches, DEFLATE uses a hash table:

Algorithm 2 DEFLATE Match Finding with Hash Chains

```
procedure FindBestMatch(current_position, data)
    hash ← hash(data[current_position : current_position + 3])
    best_length ← 0
    best_distance ← 0
    for each prev_pos in hash_table[hash] do
        if current_pos − prev_pos > 32768 then          ▷ Outside window
            continue
        end if
        length ← MatchLength(prev_pos, current_pos)
        if length > best_length then
            best_length ← length
            best_distance ← current_pos − prev_pos
        end if
    end for
    return (best_length, best_distance)
end procedure
```

The hash is typically computed from the first 3 bytes of the potential match. Hash chains link all positions with the same 3-byte prefix.

6.3.3 Lazy Matching and Greedy Heuristics

DEFLATE uses sophisticated heuristics to find better matches within the sliding window constraints:

- **Greedy parsing**: Immediately take the longest match found at the current position

- **Lazy matching**: Consider outputting a literal instead, hoping for a better match starting at the next position

Important

Window Constraints in Lazy Matching:

- All matches must reference data within the 32KB search buffer
- The lookahead buffer contains data to be encoded (maximum 258 bytes)
- Lazy matching decides: "match now" vs. "literal now + potentially better match next"

Example

Lazy Matching Example Where Lazy Parsing Wins

Consider the input data:

`aaaaabaaaaa`

Indexing the data:

0	1	2	3	4	5	6	7	8	9	10
a	a	a	a	a	b	a	a	a	a	a

Assume that the first six bytes have already been processed. The sliding window state is:

a	a	a	a	a	b	a	a	a	a	a
Search Buffer						Lookahead Buffer				

Search buffer: ``aaaaab'' (positions 0–5)
Lookahead buffer: ``aaaaa'' (positions 6–10)
Decision point: position 6 (first 'a' in lookahead)

- **Greedy parsing**:
  - Find longest match ≥ 3 starting at position 6
  - ``aaaaa'' matches ``aaaaa'' starting at position 1 in search buffer
  - Match length = 5, distance = 5
  - Greedy immediately emits: (distance=5, length=5)
  - Covers 5 bytes
- **Lazy parsing**:
  - At position 6: Finds match length = 5, distance = 5

- Checks next position (position 7)
- Starting at position 7: ``aaaa'' (4 bytes in lookahead)
- Cannot find match longer than 5 bytes (only 4 bytes remain)
- Greedy wins - lazy doesn't help here

Example where lazy actually wins:
Consider: ``abcabcdabcde''
After processing first 3 bytes:

a	b	c	a	b	c	d	a	b	c	d	e	
Search Buffer			Lookahead Buffer									

Search buffer: ``abc'' (positions 0-2)
Lookahead: ``abcdabcde'' (positions 3-11)
At position 4 ('a'):

- **Greedy**:
  - ``abc'' matches at position 0 (distance=3, length=3)
  - Encodes: (3,3)
  - Advances to position 6
  - Position 6: ``dabcde'' - no match ≥ 3
  - Output literals: 'd','a','b','c','d','e'
- **Lazy**:
  - At position 3: Finds ``abc'' (length=3, distance=3)
  - Checks position 4: ``bcdabcde''
  - ``bcd'' doesn't match (only ``bc'' in search buffer)
  - Length=2 < 3
  - Greedy wins

The reality: Simple examples showing dramatic lazy advantage are mathematically rare. Lazy matching typically provides small gains (1-2 bytes) in specific edge cases.

Algorithm 3 Lazy Matching with Sliding Window Constraints

```
procedure LazyMatch(search_buffer, lookahead)
    pos ← 0                                          ▷ Position within lookahead buffer
    best_match_here ← FindBestMatch(search_buffer, lookahead, pos)
    if best_match_here.length ≥ 3 then
        best_match_next ← FindBestMatch(search_buffer, lookahead, pos + 1)
        if best_match_next.length > best_match_here.length then
            Output lookahead[pos] as literal
            return pos + 1                           ▷ Move to next position
        else
            Output match (best_match_here.distance, best_match_here.length)
            return pos + best_match_here.length
        end if
    else
        Output lookahead[pos] as literal
        return pos + 1
    end if
end procedure
```

Important

Why Lazy Matching Works with Sliding Windows:

- By outputting a literal, we shift the window by 1 byte
- This adds that literal to the search buffer for future matches
- Sometimes the literal + new search buffer position enables a longer match
- The decision is: "Is sacrificing 1 byte now worth gaining several bytes later?"

Typical lazy matching optimization:

- Only consider lazy if current match length < 8 (configurable threshold)
- Check only next 1-2 positions (not all possible positions)
- Stop lazy checking if a very long match (e.g., > 128 bytes) is found

6.3.4 Output Format: (Length, Distance) Pairs vs. Literal Bytes

The LZSS stage produces a stream of symbols from two alphabets:

Type	Range	Encoding Details
Literal bytes	0–255	Direct byte values
Length codes	3–258	Base code + 0–5 extra bits for longer lengths
Distance codes	1–32768	Base code + 0–13 extra bits in logarithmic ranges
End-of-block	256	Special marker (end of DEFLATE block)

Important encoding details:

- **Lengths 3–258**: Encoded using symbols 257–285
  - For example: Length 3 = symbol 257 (no extra bits)
  - Length 10 = symbol 264 (no extra bits)
  - Length 258 = symbol 285 (no extra bits)
  - Lengths 11–258 use 1–5 extra bits for precise encoding
- **Distances 1–32768**: Encoded using symbols 0–29 with logarithmic ranges
  - Example ranges:
    * Symbol 0: Distance 1 (no extra bits)
    * Symbol 4: Distances 5–6 (1 extra bit)
    * Symbol 29: Distances 24577–32768 (13 extra bits)
  - Distance 100 would use symbol for range 97–128 with extra bits

Example

Encoding Distance 100:

1. Distance 100 falls in range 97–128
2. According to DEFLATE specification:
   - Range 97–128 corresponds to distance symbol 18
   - Base distance for this symbol: 97
   - Extra bits needed: 2 bits (since 128-97+1 = 32 possibilities)
3. Compute extra bits: 100 - 97 = 3
4. Encode as: symbol 18 followed by binary '11' (3 in 2 bits)

This logarithmic encoding allows representing 32768 possible distances with only 30 symbols plus extra bits, significantly reducing the Huffman alphabet size.

6.4 DEFLATE's Unique Huffman Coding Scheme

This is where DEFLATE shows its brilliance. Instead of storing full Huffman trees, it stores only **code lengths**.

6.4.1 Two Alphabets: Literals (0-285) and Distances (0-29)

DEFLATE uses separate Huffman trees for different symbol types:

- **Literals/Lengths alphabet**: 286 symbols
  - 0-255: Literal bytes
  - 256: End-of-block marker
  - 257-285: Length codes (with extra bits)
- **Distances alphabet**: 30 symbols
  - 0-3: Direct distances 1-4
  - 4-29: Distance codes with extra bits

6.4.2 Canonical Huffman via Code Lengths: The Brilliant Compression

Instead of storing:

- Full tree structure (large), or
- Symbol-to-code mapping (variable size)

DEFLATE stores only:

- **Code length for each symbol** (4 bits each, 19 maximum length)
- **Canonical reconstruction rules**:
  1. Sort symbols by code length
  2. Assign first code of length k as 0
  3. Each subsequent code = previous code + 1
  4. When increasing length, left-shift and add zeros

Example

Canonical Huffman Example:
Given code lengths: A=2, B=2, C=3, D=3, E=3

1. Sort by length: A(2), B(2), C(3), D(3), E(3)
2. Length 2: Start with 00

3. Assign: A=00, B=01 (00+1)
4. Length 3: Start with 100 (01¡¡1)
5. Assign: C=100, D=101, E=110

Decoder only needs code lengths to reconstruct this!

6.4.3 Run-Length Encoding of Code Lengths (19-Symbol Alphabet)

Code lengths themselves are compressed using RLE on a 19-symbol alphabet:

- 0-15: Actual code length 0-15
- 16: Repeat previous length 3-6 times (2 extra bits)
- 17: Repeat zero 3-10 times (3 extra bits)
- 18: Repeat zero 11-138 times (7 extra bits)

This compression means the Huffman tree description is itself heavily compressed!

6.4.4 Why Canonical Codes Enable O(1) Decoding

With canonical codes, decoding becomes a table lookup:

```
// Precomputed tables from code lengths
min_code[length] = smallest code of this length
max_code[length] = largest code of this length
first_symbol[length] = first symbol with this length

// Decoding loop
while (need more symbols) {
    code = peek_bits(MAX_BITS);  // Look at next bits
    for (len = 1 to MAX_BITS) {
        if (code <= max_code[len]) {
            symbol = first_symbol[len] + (code - min_code[len]);
            consume_bits(len);
            break;
        }
    }
}
```

Listing 1: Canonical Huffman Decoding (Pseudocode)

This is **O(1)** per symbol (constant-time lookup), crucial for fast decompression.

6.5 Block Structure and Streaming Format

DEFLATE supports streaming through independent compression blocks.

6.5.1 Three Block Types: Stored, Fixed Huffman, Dynamic Huffman

Type	Bits	Use Case
Stored	00	Uncompressible data
Fixed Huffman	01	Default trees (predefined)
Dynamic Huffman	10	Custom trees per block
Reserved	11	(Error)

- **Stored blocks**: Raw data with length header (for already-compressed data)
- **Fixed blocks**: Use predefined Huffman trees (no tree description overhead)
- **Dynamic blocks**: Custom optimized trees (best compression)

6.5.2 Bit-Level Layout: BFINAL, BTYPE, and Data

Each block has precise bit-level layout:

```
// Block header
BFINAL: 1 bit  (1 = last block in stream)
BTYPE:  2 bits (00, 01, or 10)

// If BTYPE == 00 (Stored block)
//    Skip to next byte boundary
//    LEN: 16 bits (block length)
//    NLEN: 16 bits (~LEN for error checking)
//    Data: LEN bytes of raw data

// If BTYPE == 01 (Fixed Huffman)
//    Compressed data using fixed trees

// If BTYPE == 10 (Dynamic Huffman)
//    HLIT: 5 bits (257-286 literal/length codes)
//    HDIST: 5 bits (1-30 distance codes)
//    HCLEN: 4 bits (4-19 code length codes)
//    Code lengths for code length alphabet (HCLEN+4 times)
//    Compressed literal/length and distance code lengths
//    Compressed data
```

Listing 2: DEFLATE Block Structure

6.5.3 Streaming Support: Independent Blocks and Reset Points

- Each block compressed independently
- History buffer **does not reset** between blocks (maintains 32KB window)
- Allows streaming compression of unlimited data
- Enables random access in some formats (ZIP seeks to file boundaries)

6.6 Step-by-Step Compression Walkthrough

Let's trace through compressing a simple English phrase.

6.6.1 Input: "The cat sat on the mat"

Assume ASCII encoding (simplified for example):

```
Position: 0123456789012345678901234 56
Text:     The cat sat on the mat
Indices:  0         1         2
```

6.6.2 LZSS Processing: Finding Matches

1. Position 0-2: "The" - No match in empty history → Output literals 'T','h','e'
2. Position 3: ' ' (space) - No match → Output literal ' '
3. Position 4-6: "cat" - No match → Output 'c','a','t'
4. Continue... Position 18-20: "the" matches position 0-2!
5. Match length = 3, distance = 18
6. Encode as (length=3, distance=18)

Final LZSS output (simplified):

```
Literals: T h e   c a t   s a t   o n   (length,distance=3,18) m a t
Symbols: 84 104 101 32 99 97 116 32 115 97 116 32 111 110 (257,18) 109 97 116
```

(Note: 257 is the symbol for length=3 in DEFLATE's encoding)

6.6.3 Building Canonical Huffman Tables

From the symbol stream, compute frequencies:

Symbol	Frequency
32 (space)	4
97 ('a')	3
116 ('t')	3
Others	1 each
257 (length=3)	1
18 (distance)	1

Build Huffman trees, then extract code lengths. For example, space (32) gets shortest code.

6.6.4 Bitstream Assembly: Final Compressed Output

1. Write block header: BFINAL=1, BTYPE=10 (dynamic)
2. Write HLIT, HDIST, HCLEN values
3. Write compressed code lengths for the 19 code-length symbols
4. Write compressed literal/length and distance code lengths
5. Write compressed data using the Huffman codes
6. Pad final byte with zeros if needed

6.7 Compression Levels and Performance Trade-offs

zlib (the reference implementation) offers 9 compression levels.

6.7.1 zlib's 9 Levels: From "fast" to "best"

Level	Strategy	Use Case
0	No compression (store only)	Testing/disabled
1	Fastest LZ77, lazy matching off	Real-time compression
6	Default: moderate search, lazy matching	General purpose
9	Full search, optimal parsing	Archival storage

Levels mainly differ in:

- Hash table size and chain length
- Lazy matching on/off
- Match selection heuristics

6.7.2 Speed vs. Ratio: Why Level 6 is the Default

The default level 6 represents the "knee" in the speed/ratio curve:

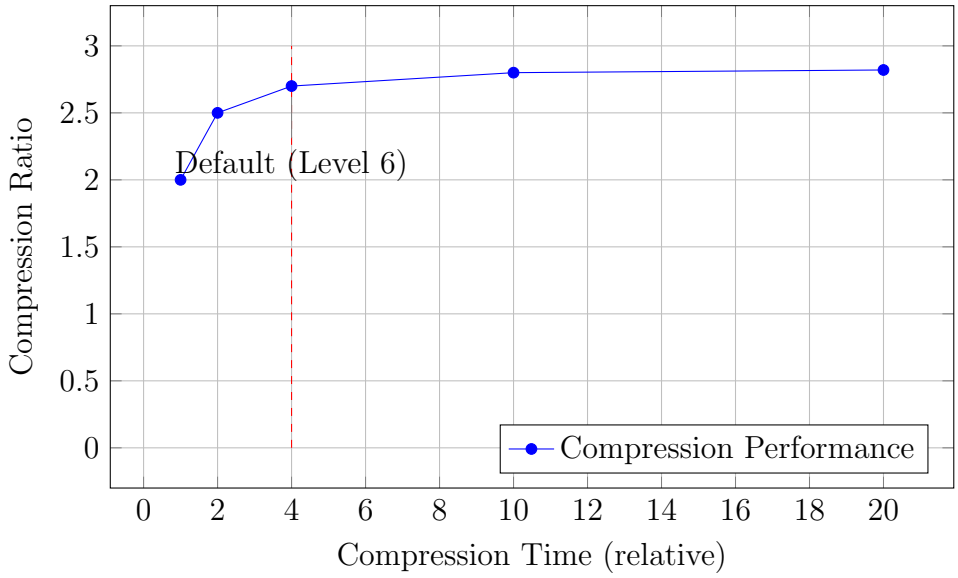


Figure 2: Trade-off between compression time and ratio in zlib

Level 9 is only ∼5% better than Level 6 but takes 2-5x longer.

6.7.3 Memory Footprint: 32KB to 256KB Windows

- **Default**: 32KB history buffer + 16KB hash table ≈ 48KB
- **Maximum**: Some implementations allow 256KB windows
- **Modern systems**: Typically use 32KB-64KB for cache efficiency

6.8 Practical Applications and File Formats

6.8.1 gzip Format: Headers, Footers, and CRC32

A gzip file contains:

```
+--------+------+------+-----+------+------+-------+------+
| Header | Extra| Name |Comment|HCRC | DEFLATE | CRC32 | ISIZE |
+--------+------+------+-----+------+------+-------+------+
  10 bytes optional optional 2    compressed  4      4
```

- **Header**: Magic bytes (0x1F, 0x8B), compression method (8=DEFLATE)
- **CRC32**: 32-bit checksum of uncompressed data
- **ISIZE**: Uncompressed size modulo 2^{32}

6.8.2 ZIP Format: Multiple Files and DEFLATE

ZIP is a container format supporting multiple compression methods:

- Each file compressed independently with DEFLATE
- Central directory at end enables random access
- Supports encryption, comments, extra fields

6.8.3 PNG: DEFLATE on Filtered Scanlines

PNG applies a **filter** to each scanline before DEFLATE:

- Subtract pixel from left pixel (Sub filter)
- Subtract pixel from above pixel (Up filter)
- Various predictors to create more compressible data
- Then DEFLATE compresses the filtered bytes

6.8.4 HTTP Compression: Accept-Encoding Header

Web servers compress responses on-the-fly:

```
Request:  GET /page.html HTTP/1.1
          Accept-Encoding: gzip, deflate

Response: HTTP/1.1 200 OK
          Content-Encoding: gzip
          [gzip-compressed body]
```

Saves 60-80% bandwidth for text content (HTML, CSS, JS).

6.9 Limitations and When DEFLATE Fails

6.9.1 Poor Performance on Already-Compressed or Random Data

- **Already compressed**: JPEG, MP3, ZIP files → DEFLATE can't compress further
- **Encrypted data**: Random-looking → No patterns to find
- **Result**: May actually *increase* size due to overhead

Good compressors detect this and use "stored" (uncompressed) blocks.

6.9.2 The "Shakespeare Problem": Global vs. Local Repetition

DEFLATE's 32KB window can't capture **global repetition**:

Example

Shakespeare's Complete Works:

- Word "the" appears thousands of times
- But only instances within 32KB of each other are linked
- Distant repetitions (across chapters) aren't captured
- Result: Compresses to ∼2 bits/byte instead of potential ∼1.5

Modern compressors (LZMA, zstd) use larger dictionaries (MBs) for this case.

6.9.3 Why Not Arithmetic Coding? Patents, Speed, and Complexity

In the 1990s, arithmetic coding was:

- **Patented**: IBM held key patents
- **Slower**: 2-10x slower than Huffman
- **Complex**: Harder to implement correctly
- **Bit-level operations**: Unfriendly to byte-oriented systems

Today, arithmetic coding is patent-free but DEFLATE's ecosystem is entrenched.

6.9.4 Security Issues: CRIME, BREACH, and Compression Side-Channels

Compression can leak information:

- **CRIME (2012)**: HTTPS compression reveals session cookies
- **BREACH (2013)**: HTTP compression reveals CSRF tokens
- **Principle**: If secret affects compressed size, attacker can deduce it
- **Solution**: Disable compression on sensitive data or use random padding

6.10 Modern Alternatives and Successors

6.10.1 LZMA and 7-Zip: Better Compression, Slower Speed

LZMA (Lempel-Ziv-Markov chain Algorithm):

- **Larger window**: Up to 4GB vs DEFLATE's 32KB
- **Range coding**: Arithmetic coding variant

- **Better compression**: 30-50% smaller than DEFLATE
- **Slower**: 5-10x slower compression
- **Used in**: 7z format, xz utilities, software installers

6.10.2 Zstandard (zstd): Facebook's Balanced Alternative

zstd (2016) aims to replace DEFLATE:

- **Faster**: 5-10x faster decompression than zlib
- **Better compression**: 10-20% better than zlib at same speed
- **Modern features**: Dictionary training, reproducible builds
- **Adoption**: Linux kernel, SquashFS, Facebook, Netflix

6.10.3 Brotli: Google's Web-Optimized Successor

Brotli (2013) targets web compression:

- **Static dictionary**: 120KB of common web strings
- **Better text compression**: 20-26% better than gzip
- **Slower compression**: But fast decompression
- **Adoption**: CDNs, web servers, browsers

6.10.4 Why DEFLATE Still Dominates: The Legacy Effect

Despite better alternatives, DEFLATE persists because:

- **Ubiquitous support**: Every system has zlib
- **Standardized**: RFC 1951, no licensing issues
- **Good enough**: For most use cases
- **Network effects**: Hard to change entrenched standards
- **Hardware support**: Some processors have DEFLATE acceleration

6.11 Hands-On Analysis and Debugging

6.11.1 Using gzip -v -l and infgen for Inspection

```
# Basic compression
gzip -9 file.txt              # Maximum compression
gzip -1 file.txt              # Fastest compression

# Analyze compressed file
gzip -l file.txt.gz           # List compression ratio
gzip -v -l file.txt.gz        # Verbose listing

# Decompress with verbose output
gzip -v -d file.txt.gz        # Show compression ratio

# Use infgen for detailed DEFLATE analysis
infgen -d file.gz             # Disassemble DEFLATE stream
```

6.11.2 Visualizing Matches and Huffman Trees

Tools for deeper analysis:

- **puff**: Educational DEFLATE implementation (readable C)
- **infgen**: DEFLATE stream disassembler
- **Custom scripts**: Parse zlib output with ZLIB_DEBUG

6.11.3 Benchmarking: Compression Ratio vs. Speed

```
# Time compression
time gzip -9 large_file.txt

# Compare compression levels
for level in {1..9}; do
    echo -n "Level $level: "
    gzip -c -$level file.txt | wc -c
done

# Benchmark suite
pigz                  # Parallel gzip implementation
zstd -b               # zstd benchmark mode
```

6.12 Summary and Key Insights

6.12.1 Why DEFLATE Dominated for 30+ Years

1. **Right trade-offs**: Fast decode over maximum compression

2. **Clever engineering**: Canonical Huffman, compact tree representation
3. **Open standard**: RFC 1951, no patents, reference implementation
4. **Good enough**: Adequate compression for most real-world data
5. **Ecosystem**: Tools, libraries, hardware support

6.12.2 Lessons for Compression Engineers: Practical Beats Perfect

- **Decode speed matters more than encode speed** for distribution formats
- **Implementation simplicity** leads to wider adoption
- **Open standards** defeat technically superior proprietary formats
- **Backward compatibility** is a powerful force
- Sometimes **"good enough" wins over "best"**

6.12.3 Looking Forward: The Future of Compression Standards

- **DEFLATE will persist** in legacy systems for decades
- **zstd and Brotli** gaining ground in new applications
- **Hardware acceleration** becoming common (Intel QAT, AWS Nitro)
- **AI/ML approaches** showing promise but not yet practical
- The next "DEFLATE" will need to be **10x better** to displace it

Important

Final Thought: DEFLATE represents a peak in practical algorithm design. It shows how understanding both theory (information theory) and practice (hardware constraints, use cases) leads to enduring solutions. Study DEFLATE not just to use it, but to learn how to design algorithms that stand the test of time.

End of Chapter Exercises

Exercise 6.0

Exercise 6.1: DEFLATE Architecture Understanding

Explain in your own words why DEFLATE uses a two-stage approach (LZSS + Huffman) instead of just one of these techniques. Consider:

(a) What types of redundancy does LZSS exploit that Huffman cannot?

(b) What types of redundancy does Huffman exploit that LZSS cannot?

(c) Why can't Huffman coding alone achieve good compression on typical text data?

(d) Why can't LZSS alone achieve optimal compression?

Exercise 6.1

Exercise 6.2: LZSS Match Analysis

Given the input string: "abracadabraabracadabra"

(a) Using a sliding window of size 12, trace through LZSS processing and show the output as a sequence of literals and (length, distance) pairs.

(b) Calculate the compression ratio if literals are 8 bits and each (length, distance) pair requires 24 bits (8 for length, 16 for distance).

(c) What would be the output if the sliding window was only size 6 instead of 12?

Exercise 6.2

Exercise 6.3: Distance Encoding Practice

Using DEFLATE's distance encoding scheme (logarithmic ranges):

(a) Encode the following distances: 1, 5, 10, 100, 1000, 20000

(b) For each, specify: symbol number, base distance, extra bits required, and final binary encoding.

(c) What is the maximum distance that can be encoded without extra bits?

(d) Why does DEFLATE use logarithmic encoding instead of direct binary representation?

Hint: Refer to RFC 1951 Section 3.2.5 for the exact distance code table.

Exercise 6.3

Exercise 6.4: Canonical Huffman Construction

Given the following symbol frequencies for a small alphabet:

Symbol	Frequency
A	40
B	30
C	15
D	10
E	5

(a) Build a Huffman tree and determine code lengths for each symbol.

(b) Generate the canonical Huffman codes from these lengths.

(c) Show the exact bit sequence DEFLATE would store to represent these code lengths.

(d) Calculate the average code length and compare it to the entropy.

Exercise 6.4

Exercise 6.5: DEFLATE Block Analysis

Consider the following DEFLATE block header (in binary):

1 10 01101 00101 0100

Where the bits represent: BFINAL (1), BTYPE (10), HLIT (5 bits), HDIST (5 bits), HCLEN (4 bits).

(a) Decode each field and explain what it means.

(b) What type of block is this?

(c) How many literal/length codes will follow?

(d) How many distance codes will follow?

(e) How many code-length codes will follow?

Exercise 6.5

Exercise 6.6: Compression Level Trade-offs

You are designing a compression system for:

(a) A web server compressing HTML pages in real-time

(b) An archival system storing historical documents

(c) A mobile app with limited CPU and battery

For each scenario:

- Recommend a DEFLATE compression level (0-9) with justification
- Explain whether you would use fixed or dynamic Huffman blocks
- Discuss any modifications you might make to the standard DEFLATE parameters

Exercise 6.6

Exercise 6.7: DEFLATE vs. Modern Alternatives

Compare DEFLATE with two modern alternatives (choose from LZMA, zstd, or Brotli):

(a) Create a comparison table covering: compression ratio, speed (encode/decode), memory usage, and typical use cases.

(b) For each of the following, recommend which compressor to use and why:

- Compressing software updates for download
- Real-time compression in a database
- Web assets on a content delivery network

(c) Explain why DEFLATE is still widely used despite newer, better algorithms.

Exercise 6.7

Exercise 6.8: Implementation Challenge

Write pseudocode for a DEFLATE decoder that:

(a) Reads the block header and identifies block type

(b) For dynamic blocks, reconstructs the Huffman trees from code lengths

(c) Decodes the compressed data using canonical Huffman decoding

(d) Handles both literals and (length, distance) pairs

Focus on the key algorithms, not edge cases or optimizations. Your pseudocode should show the overall structure and critical steps.

Exercise 6.8

Exercise 6.9: Security Analysis

The CRIME and BREACH attacks exploit compression side-channels:

(a) Explain the fundamental vulnerability that enables these attacks.

(b) Why does DEFLATE's LZSS stage make this vulnerability particularly dangerous?

(c) Propose three different strategies to mitigate this risk while still using compression.

(d) Would switching to a different compression algorithm (like zstd or Brotli) solve the problem? Why or why not?

Exercise 6.9

Exercise 6.10: Research and Reflection

(a) DEFLATE was standardized in 1996. Identify three technological constraints from that era that influenced DEFLATE's design decisions.

(b) If you were designing DEFLATE today, what would you change given modern hardware (multi-core CPUs, gigabytes of RAM, SSD storage)?

(c) Research the current state of hardware-accelerated compression (Intel QAT, AWS Nitro, etc.). How might these technologies change the optimal compression algorithm design?

(d) Predict: Will DEFLATE still be widely used in 2030? Justify your answer.

Important

Exercise Guidelines:

- **Exercises 6.1-6.5**: Core concepts – recommended for all students
- **Exercises 6.6-6.8**: Intermediate application – for deeper understanding
- **Exercises 6.9-6.10**: Advanced topics – for interested students or projects
- **Time estimates**: Basic exercises (30 min each), advanced (60-90 min each)
- **Resources**: Refer to RFC 1951, zlib source code, and infgen tool for hands-on exploration

Optional Project Challenge

Build a Simple DEFLATE Analyzer: Write a program in your preferred language that:

- Takes a gzip file as input
- Parses the gzip header and trailer
- Extracts the DEFLATE stream
- Identifies block boundaries and types
- Reports basic statistics (compression ratio, block types used, etc.)
- **Bonus**: Visualize the Huffman trees or match distances

Use existing libraries for decompression, but implement the analysis logic yourself. This project will deepen your understanding of the actual byte-level format.

7: Lecture 7: Run-Length Encoding: Exploiting Equality Redundancy

Important

Learning Objectives:

- Understand the concept of equality redundancy and why it's the simplest form of compressible pattern
- Master the basic RLE algorithm and its variants for different data types
- Analyze compression ratios, best/worst cases, and theoretical limits of RLE
- Learn four major RLE variants: Count-Based, Escape-Based, Block-Based, and Zero RLE
- Understand why RLE fails on some data types and how to combine it with other techniques
- Build the conceptual bridge to transform-based compression, especially BWT

7.1 Introduction and Motivation

7.1.1 The Simplest Form of Redundancy: Repetition

When we look at data through the lens of compression, we're constantly asking: "What patterns repeat?" Among all possible patterns, the absolute simplest is when the *same symbol repeats consecutively*. We call this **equality redundancy**.

Definition

Equality Redundancy: A form of data redundancy where the same symbol appears multiple times in consecutive positions within a sequence. Each such group of identical consecutive symbols is called a **run**.

Consider these examples:

- Text: `AAAABBBCCCC` contains three runs: `AAAA`, `BBB`, and `CCCC`
- Images: A blue sky photo has long runs of similar blue pixel values
- Signals: Silence in audio contains runs of zero amplitude
- Documents: White space in scanned text creates runs of white pixels

The key insight is elegant in its simplicity: instead of storing each symbol individually, we can store each symbol once along with a count of how many times it repeats. For `AAAA`, instead of storing 4 bytes, we store (`A`,4).

Example

Intuition Building: Raw data: `WWWWWWWWWWWWWBBWWWWWWWWWWWWWBBBWWWWWWWWW`
With runs identified:

- 13 W's
- 2 B's
- 13 W's
- 3 B's
- 9 W's

Storing as pairs: (W,13)(B,2)(W,13)(B,3)(W,9)
Original: 40 characters Encoded: 10 values (5 symbols + 5 counts) $\rightarrow$ significant savings!

7.2 Fundamental Concepts and Theory

7.2.1 Formal Definition of a Run

Before we can encode runs, we need a precise mathematical definition.

Definition

Run: Given a sequence $S = s_1, s_2, s_3, \ldots, s_n$ over an alphabet Σ, a run is a maximal consecutive subsequence $s_i, s_{i+1}, \ldots, s_j$ such that $s_i = s_{i+1} = \ldots = s_j$, and either $i = 1$ or $s_{i-1} \neq s_i$, and either $j = n$ or $s_{j+1} \neq s_j$.
In simpler terms: a run is the longest possible sequence of identical symbols that appears consecutively.

Properties of Runs:

1. **Maximality:** A run cannot be extended—the symbol before it (if any) is different, and the symbol after it (if any) is different.
2. **Disjointness:** Runs partition the sequence; every symbol belongs to exactly one run.
3. **Alternation:** Adjacent runs always contain different symbols.

7.2.2 Run-Length Distribution Theorem

Understanding the statistical properties of runs is fundamental to compression analysis.

Theorem 7.1. *For an i.i.d. (independent and identically distributed) random sequence over an alphabet of size k, the expected run length is:*

$$E[\textit{run length}] = \frac{1}{1-p} = \frac{k}{k-1}$$

where $p = 1/k$ is the probability of a symbol repeating.

Proof. For a random sequence with equiprobable symbols:

- Probability that next symbol equals current $= 1/k$
- Run length follows geometric distribution: $P(\text{run length} = L) = (1/k)^{L-1} \cdot (1-1/k)$
- Expected value: $\sum_{L=1}^{\infty} L \cdot (1/k)^{L-1} \cdot (1-1/k) = \frac{1}{1-1/k} = \frac{k}{k-1}$

□

Example

Expected Run Lengths for Different Alphabets:

Alphabet	Size (k)	Expected run length
Binary	2	$2/(2-1) = 2$
DNA bases	4	$4/3 \approx 1.333$
Lowercase letters	26	$26/25 = 1.04$
ASCII	256	$256/255 \approx 1.0039$

Implications:

- Binary data has the longest expected runs (2)
- ASCII text has runs barely longer than 1
- For large alphabets, runs of length > 1 are rare events

7.2.3 Compression Ratio Analysis

Definition

Compression Ratio:

$$\text{Ratio} = \frac{\text{Original Size}}{\text{Compressed Size}}$$

Higher ratio means better compression.

Theorem 7.2. *For a sequence of length n consisting of a single symbol, the best-case compression ratio achievable with basic RLE is approximately $\frac{n}{c}$, where c is the size (in bytes) of one (symbol, count) pair.*

Theorem 7.3. *For a sequence of length n with no repeated consecutive symbols (all runs of length 1), basic RLE achieves a compression ratio of $\frac{n}{2n} = 0.5$ when using fixed-size counts equal to symbol size, resulting in 100% expansion.*

Proof. If every symbol is different from its predecessor:

- Number of runs $= n$ (each symbol is its own run)
- Encoded output $= n$ (symbol, count) pairs
- If count field size equals symbol size (e.g., 1 byte each):
  - Original size $= n$ bytes
  - Compressed size $= n$ symbols $+ n$ counts $= 2n$ bytes
  - Ratio $= n/(2n) = 0.5$
  - Expansion $= (2n - n)/n = 1 = 100\%$

□

7.3 Taxonomy of RLE Methods

RLE variants differ in their answers to three fundamental questions:

1. How do we distinguish between runs and non-repeated data?
2. How do we represent the count?
3. How do we handle maximum run lengths?

Based on these choices, we can identify four major RLE families:

Important

The Four Major RLE Variants:

1. **Count-Based RLE (Classic RLE):** Stores each run as (symbol, count). Simple and effective for data with many runs.
2. **Escape-Based RLE:** Uses special markers to distinguish runs from literals. Handles mixed data efficiently.
3. **Block/Header-Based RLE (PackBits Style):** Divides data into chunks with headers that describe literal or run data. Balances overhead and efficiency.

4. **Zero Run-Length Encoding (Zero RLE):** Specialized for compressing runs of zeros in sparse data, commonly used in transform coding.

7.4 Count-Based RLE (Classic RLE)

Count-based RLE is the original and simplest form—each run is represented by its symbol and a count of how many times it repeats.

7.4.1 Basic Encoding Algorithm

Algorithm 4 Count-Based RLE Encoding

```
procedure RLE-Encode(S[1..n])
    result ← empty list
    if n = 0 then
        return result
    end if
    current_symbol ← S[1]
    run_length ← 1
    for i ← 2 to n do
        if S[i] = current_symbol then
            run_length ← run_length + 1
        else
            append (current_symbol, run_length) to result
            current_symbol ← S[i]
            run_length ← 1
        end if
    end for
    append (current_symbol, run_length) to result        ▷ Don't forget the last run!
    return result
end procedure
```

7.4.2 Basic Decoding Algorithm

Algorithm 5 Count-Based RLE Decoding

```
procedure RLE-Decode(pairs[1..m])
    result ← empty list
    for each (symbol, count) in pairs do
        for i ← 1 to count do
            append symbol to result
        end for
    end for
    return result
end procedure
```

7.4.3 Count-After-Symbol vs. Count-Before-Symbol

The ordering of symbol and count in the output stream affects decoding and has subtle implications.

Important

Two Common Conventions:

1. Count-After-Symbol (Symbol then Count):

- Format: `[symbol][count][symbol][count]...`
- Example: `A5B3C2` means "5 A's, 3 B's, 2 C's"
- **Advantages:**
  - Natural reading order: "Here's what to output, then how many"
  - Easier for streaming: see symbol first, can prepare output
  - Common in many implementations

2. Count-Before-Symbol (Count then Symbol):

- Format: `[count][symbol][count][symbol]...`
- Example: `5A3B2C` means "5 A's, 3 B's, 2 C's"
- **Advantages:**
  - Decoder knows length before seeing symbol
  - Can allocate buffer space if needed
  - Used in some formats (e.g., PCX)

7.4.4 Fixed-Width Count RLE

In fixed-width count RLE, each run length is stored in a fixed number of bits.

Important

Fixed-Width Count Fields Design:

Design: Each run length stored in a fixed number of bits (e.g., 8 bits, 16 bits, 32 bits)

Advantages:

- Simple to implement
- Fast encoding/decoding (no bit-level parsing)

- Random access possible (know where each run ends)
- Good for hardware implementation

Disadvantages:

- Wasted space for short runs
- Limited maximum run length (e.g., 255 for 8 bits)
- Need to split longer runs into multiple chunks
- Not adaptive to run length distribution

Typical trade-offs:

Count size	Max run	Overhead/run	Best for
4 bits (nibble)	15	0.5 bytes	Very short runs
8 bits (byte)	255	1 byte	General purpose
16 bits	65,535	2 bytes	Long runs
24 bits	16.7M	3 bytes	Images
32 bits	4.3B	4 bytes	Very large files

Example

Fixed-Width RLE Example:
Input sequence: `WWWWWWWWWWWWWBBWWWWWWWWWWWWWBBBWWWWWWWWW`
With 8-bit counts:

- Runs: (W,13), (B,2), (W,13), (B,3), (W,9)
- Each run: 1 byte symbol + 1 byte count = 2 bytes
- Total: 5 runs $\times$ 2 bytes = 10 bytes
- Original: 40 bytes
- Compression ratio: 4:1 (75% savings)

With 4-bit counts (nibbles):

- Need to pack two counts per byte
- Runs: (W,13), (B,2), (W,13), (B,3), (W,9)
- Symbols: 5 bytes
- Counts: 5 $\times$ 4 bits = 20 bits = 2.5 bytes

- Total: 7.5 bytes (assuming we can pack)
- But 13 exceeds 4-bit max (15) — OK here
- Compression ratio: 5.33:1 (81% savings)

7.4.5 Handling Large Runs with Fixed-Width Counts

When runs exceed the maximum representable count, we need splitting strategies.

Important

Splitting Runs Strategy:
When count exceeds maximum, output multiple run chunks:

- Example: max count = 255, actual run = 1000
- Output: (sym,255), (sym,255), (sym,255), (sym,235)
- Overhead increases but preserves maximum limit

Alternative: Use larger count fields

- Switch to 16-bit or 32-bit counts
- But overhead increases for all runs
- May still need splitting for very large runs

Example

Run Splitting Example:
Assume 8-bit counts (max = 255)
Input run: 1000 'A's
Split into max-sized chunks:

- (A,255), (A,255), (A,255), (A,235)
- Overhead: 4 pairs instead of 1
- Extra overhead: 3 extra count bytes + 3 extra symbol bytes = 6 extra bytes
- Still much better than storing 1000 raw bytes!

7.4.6 Variable-Width Count RLE (Elias Gamma)

Variable-width count RLE uses codes that use fewer bits for small numbers and more bits for large numbers, matching the information content of the count.

Important

Variable-Length Count Fields Design:
Design: Use a variable-length code (like Elias gamma coding, Fibonacci coding) to represent run lengths
Common approaches:

1. **Gamma/Elias codes:** Good for small numbers, inefficient for large
2. **Fibonacci codes:** Better balance, no need for separator
3. **Exponential-Golomb:** Used in H.264/AVC video coding
4. **Adaptive Huffman:** Best compression, but complex

Advantages:

- Optimal for the run length distribution
- No arbitrary maximum run length
- No wasted bits for short runs
- Can achieve near-entropy compression of run lengths

Disadvantages:

- Complex implementation (bit-level I/O)
- Slower encoding/decoding
- No random access without decoding entire stream
- Error propagation if bit errors occur

Elias Gamma Coding Deep Dive

Elias gamma coding provides the clearest illustration of variable-length coding principles.

Definition

Elias Gamma Code: A variable-length code for positive integers where each number $N \geq 1$ is encoded as:

1. Write N in binary.

2. Let the binary representation of N use L bits.
3. Write $L - 1$ zeros.
4. Append the full binary representation of N.

Equivalently, if $k = \lfloor \log_2 N \rfloor$, then:

$$\text{Gamma}(N) = \underbrace{00\ldots0}_{k \text{ zeros}} \text{ binary}(N)$$

Total code length: $2k + 1$ bits.

Building Elias Gamma Codes: Step by Step

1. **For N = 1:**
   - Binary: 1 (L=1)
   - Zeros: L-1 = 0 zeros
   - Code: "1"
2. **For N = 2:**
   - Binary: 10 (L=2)
   - Zeros: L-1 = 1 zero
   - Code: "010"
3. **For N = 3:**
   - Binary: 11 (L=2)
   - Zeros: 1 zero
   - Code: "011"
4. **For N = 4:**
   - Binary: 100 (L=3)
   - Zeros: 2 zeros
   - Code: "00100"
5. **For N = 5:**
   - Binary: 101 (L=3)
   - Zeros: 2 zeros
   - Code: "00101"

Complete Reference Table

N	Binary of N	k = floor(log2 N)	Gamma Code
1	1	0	1
2	10	1	010
3	11	1	011
4	100	2	00100
5	101	2	00101
6	110	2	00110
7	111	2	00111
8	1000	3	0001000
9	1001	3	0001001
10	1010	3	0001010

Decoding Elias Gamma

Algorithm 6 Elias Gamma Decoding

```
procedure DecodeGamma(bitstream)
    k ← 0
    while next bit is 0 do                          ▷ Count leading zeros
        k ← k + 1
        advance bitstream
    end while
    advance bitstream                               ▷ Skip the 1
    value ← 1                                       ▷ The leading 1 is the most significant bit
    for i ← 1 to k do
        value ← (value ≪ 1)| next bit
    end for
    return value
end procedure
```

Example

Decoding Example:

Given bitstream: `001011...`

1. Read zeros: "00" then a 1 → found "001", so $k = 2$
2. The 1 we read is part of the code
3. Read next $k = 2$ bits: "01"
4. Construct value: leading 1 + "01" = "101" binary = 5

5. First number decoded is 5, remaining bits continue from "..."

Fixed-Width vs. Variable-Width: Detailed Comparison

Example

Count Field Comparison:

Consider a file with runs of lengths: 3, 5, 2, 100, 4, 200

Fixed 8-bit counts:

- Each run: 1 byte for count
- Total count overhead: 6 bytes = 48 bits
- Can represent all runs (max 200 < 255)
- Simple, fast, randomly accessible

Fixed 4-bit counts:

- Cannot represent runs >15
- Would need to split run 100 into: 15,15,15,15,15,15,10 (7 runs!)
- Run 200: even worse, would need 14 runs (13×15 + 5)
- Total overhead: 6 runs become 6+7+14 = 27 runs!
- 27 runs × 0.5 bytes = 13.5 bytes (worse than 8-bit!)

Variable-length (Elias gamma):

- 3 → "011" (3 bits)
- 5 → "00101" (5 bits)
- 2 → "010" (3 bits)
- 100 → Let's construct: binary 100 = 1100100 (7 bits), k=6, so "0000001" + "100100" = "0000001100100" (13 bits)
- 4 → "00100" (5 bits)
- 200 → binary 200 = 11001000 (8 bits), k=7, so "00000001" + "1001000" = "000000011001000" (15 bits)
- Total: 44 bits ≈ 5.5 bytes

Comparison Table:

Method	Total Bits	Characteristics
Fixed 4-bit	Not possible (runs too long)	Would need run splitting
Fixed 8-bit	48 bits	Simple, fast, random access
Elias gamma	44 bits	8.3% savings, complex, no random access

Bit-by-bit breakdown:

Method	3	5	2	100	4	200	Total
Fixed 8-bit	8	8	8	8	8	8	48
Elias gamma	3	5	3	13	5	15	44
Savings	+5	+3	+5	-5	+3	-7	+4

Notice the economic trade-off: small numbers (2,3,4,5) use fewer bits than fixed-width, which subsidizes the extra bits needed for large numbers (100,200). In typical data where short runs are much more common than long runs, this trade-off yields net compression.

Decision Matrix: Fixed vs. Variable-Width Counts

Design Question	Fixed-Width Counts	Variable-Width Counts
Implementation complexity	Very simple. Each count always uses the same number of bits.	More complex. Must read bits carefully to determine where each count ends.
Encoding/decoding speed	Very fast. No need to examine individual bits.	Slower. Must count prefix bits and parse variable-length codes.
Random access	Yes. If each count uses (say) 8 bits, boundaries are known immediately.	No. Must decode sequentially to know where the next count begins.
Space efficiency	When run lengths are roughly similar or bounded (e.g., always between 1 and 200).	When most run lengths are small but occasionally very large. Small values use very few bits.
Maximum run length	Limited by chosen width. For 8 bits, maximum is 255.	No fixed upper bound. Large values simply use more bits.
Error resilience	Damage usually affects only that run.	Damage may affect all following runs because boundaries shift.

7.5 Escape-Based RLE

Escape-based RLE addresses the fundamental weakness of count-based RLE: expansion on non-repetitive data. By using special markers, it treats runs and literals differently.

7.5.1 Core Concept

Definition

Escape Symbol: A special marker that indicates whether the following data represents a run or a sequence of literal (uncompressed) symbols.

The key insight: instead of encoding every symbol as a (symbol, count) pair (which doubles the size for non-repeated symbols), we only encode runs when they're beneficial, and leave non-repeated data as-is.

7.5.2 Basic Escape Scheme Design

Important

Standard Escape Scheme:

1. Choose an escape symbol (often a value not likely to appear in data, or one that can be escaped itself)
2. When encountering a run of length ≥ 3:
   - Output escape symbol
   - Output the repeated symbol
   - Output the run length
3. For non-repetitive data:
   - Output symbols literally
   - If the literal data contains the escape symbol, need a way to represent it (e.g., double escape)

7.5.3 Encoding Algorithm

Algorithm 7 Escape-Based RLE Encoding (Threshold T=3)

```
procedure EscapeRLE-Encode(S[1..n], escape)
    result ← empty list
    i ← 1
    while i ≤ n do
        run_length ← 1
        while i + run_length ≤ n and S[i + run_length] = S[i] do
            run_length ← run_length + 1
        end while
        if run_length ≥ 3 then                    ▷ Long run - encode with escape
            append escape to result
            append S[i] to result
            append run_length to result
        else                                      ▷ Short run - output as literals
            for j ← 0 to run_length − 1 do
                if S[i + j] = escape then
                    append escape to result       ▷ Escape the escape!
                    append escape to result
                else
                    append S[i + j] to result
                end if
            end for
        end if
        i ← i + run_length
    end while
    return result
end procedure
```

7.5.4 Decoding Algorithm

Algorithm 8 Escape-Based RLE Decoding

```
procedure EscapeRLE-Decode(data[1..m], escape)
    result ← empty list
    i ← 1
    while i ≤ m do
        if data[i] = escape then
            if i + 2 > m then
                error
            end if
            symbol ← data[i + 1]
            count ← data[i + 2]
            for j ← 1 to count do
                append symbol to result
            end for
            i ← i + 3
        else
            if i + 1 ≤ m and data[i + 1] = escape then ▷ Double escape means literal escape
                append escape to result
                i ← i + 2
            else
                append data[i] to result
                i ← i + 1
            end if
        end if
    end while
    return result
end procedure
```

7.5.5 Worked Example

Example

Escape Scheme Example with Escape = 0xFF:

Input: `AAAAABCDEFFFFGG` (15 bytes)

Step-by-step encoding:

- Position 1-5: `AAAAA` (5 A's) → run length $5 \geq 3$
  - Output: `FF A 05`

- Position 6: `B` (literal) → output: `B`
- Position 7: `C` (literal) → output: `C`
- Position 8: `D` (literal) → output: `D`
- Position 9: `E` (literal) → output: `E`
- Position 10-13: `FFFF` (4 F's) → run length $4 \geq 3$
  - Output: `FF F 04`
- Position 14-15: `GG` (2 G's) → run length $2 < 3$, output as literals
  - Output: `G G`

Encoded: `FF A 05 B C D E FF F 04 G G`
Analysis:

- Original: 15 bytes
- Encoded: 11 bytes
- Savings: 4 bytes (27% compression)
- Without escape scheme (basic RLE): would be 16 bytes (expansion!)

Example

Handling Escape Symbols in Literal Data:
Input: `ABC\xFF\xFFDEF` (contains two 0xFF bytes)
Encoding:

- `A` → literal A
- `B` → literal B
- `C` → literal C
- `\xFF` (first) → this is the escape symbol! Need to escape it
  - Output: `FF FF` (double escape means literal FF)
- `\xFF` (second) → another escape symbol
  - Output: `FF FF`
- `D` → literal D

- `E` → literal E
- `F` → literal F

Encoded: `A B C FF FF FF FF D E F`
Notice how each literal 0xFF becomes two bytes in the output (the escape marker twice).

7.5.6 Choosing the Threshold

The threshold determines the minimum run length that triggers escape encoding.

Important

Threshold Trade-offs:

- **Low threshold (T=2):**
  - Catches more runs (even pairs)
  - Overhead: each run costs 3 bytes instead of 2 bytes for literals
  - Break-even: 3 bytes for run vs. 2 bytes for literals → not beneficial!
  - T=2 actually causes expansion on pairs
- **Medium threshold (T=3):**
  - Run of 3: 3 bytes encoded vs. 3 bytes literals → break-even
  - Run of 4+: 3 bytes encoded vs. 4+ bytes literals → savings
  - Most common choice
- **High threshold (T=4):**
  - Only encodes longer runs
  - Misses runs of 3 (which are break-even anyway)
  - Safer but less aggressive

Optimal threshold analysis: With 1-byte escape, 1-byte symbol, 1-byte count:

- Cost to encode run of length L: 3 bytes
- Cost to store as literals: L bytes
- Break-even when L = 3
- Savings when L > 3: L - 3 bytes saved
- Loss when L < 3: 3 - L bytes wasted

7.5.7 Advantages and Disadvantages

Important

Escape-Based RLE: Pros and Cons

Advantages:

- No expansion on non-repetitive data (except for escaping the escape symbol)
- Handles mixed data types efficiently
- Simple to implement
- Used in many real-world formats (e.g., TIFF, some fax formats)

Disadvantages:

- Overhead of escape symbol reduces efficiency on highly repetitive data
- Need to handle escape symbol in literal data (double-escape)
- Fixed threshold may not be optimal for all data
- 3-byte overhead per run (vs. 2 bytes in basic RLE)

7.6 Block/Header-Based RLE (PackBits Style)

Block-based RLE divides the data into chunks, each preceded by a header that describes how to interpret the following bytes. PackBits is the classic example of this approach.

7.6.1 PackBits Overview

Definition

PackBits Encoding: Each chunk begins with a 1-byte header h interpreted as a signed 8-bit integer:

- If $0 \leq h \leq 127$: copy the next $h + 1$ bytes literally
- If $-127 \leq h \leq -1$: repeat the next byte $1 - h$ times
- $h = -128$: reserved (no operation, sometimes used for special purposes)

Important

PackBits Header Interpretation:
Positive (0 to 127): Literal run

- Value n means "copy next $n + 1$ bytes as-is"
- Range: 1 to 128 bytes per literal chunk

Negative (-1 to -127): Repeated run

- Value n means "repeat next byte $1 - n$ times"
- Since n is negative, $1 - n$ is positive
- Range: 2 to 128 repetitions
- Example: $h = -3$ means repeat next byte $1 - (-3) = 4$ times

-128: Reserved (some implementations ignore, others use as EOF marker)

7.6.2 Encoding Algorithm

Algorithm 9 PackBits Encoding

```
procedure PackBits-Encode(S[1..n])
    result ← empty list
    i ← 1
    while i ≤ n do
        run_length ← 1
        while i + run_length ≤ n and S[i + run_length] = S[i] and run_length < 128 do
            run_length ← run_length + 1
        end while
        if run_length ≥ 2 then                                  ▷ We have a run
            header ← −(run_length − 1)                          ▷ Negative header
            append header to result
            append S[i] to result
            i ← i + run_length
        else                                  ▷ Literal run - collect up to 128 literals
            literal_start ← i
            literal_count ← 0
            while i ≤ n and literal_count < 128 do
                                              ▷ Check if next 3+ bytes form a run
                if i + 2 ≤ n and S[i] = S[i + 1] and S[i] = S[i + 2] then
                    break                     ▷ Found a run, stop collecting literals
                end if
                i ← i + 1
                literal_count ← literal_count + 1
            end while
            header ← literal_count − 1                          ▷ Positive header
            append header to result
            for j ← literal_start to literal_start + literal_count − 1 do
                append S[j] to result
            end for
        end if
    end while
    return result
end procedure
```

7.6.3 Decoding Algorithm

Algorithm 10 PackBits Decoding

```
procedure PackBits-Decode(data[1..m])
    i ← 1
    result ← empty list
    while i ≤ m do
        h ← data[i]                                  ▷ header byte as signed integer
        i ← i + 1
        if h ≥ 0 then                                ▷ Literal run
            len ← h + 1
            for j ← 1 to len do
                append data[i] to result
                i ← i + 1
            end for
        else if h = −128 then                        ▷ Reserved, usually skip
                                                     ▷ No operation, sometimes used as marker
        else                                         ▷ Repeated run (h is negative)
            len ← 1 − h                              ▷ Since h is negative
            value ← data[i]
            i ← i + 1
            for j ← 1 to len do
                append value to result
            end for
        end if
    end while
    return result
end procedure
```

7.6.4 Worked Examples

Example

PackBits Example 1: Mixed Data

Input: `AAAAABCDEFFFFGG`

Encoding process:

- Run of 5 A's → header = -4 (since 1-(-4)=5), then byte 'A'
- Literals B,C,D,E (4 bytes) → header = 3 (copy 3+1=4 bytes), then B C D E
- Run of 4 F's → header = -3 (repeat 4 times), then byte 'F'

- Literals G,G (2 bytes) → header = 1 (copy 2 bytes), then G G

Encoded stream (as bytes with decimal headers):

```
[-4, 'A', 3, 'B','C','D','E', -3, 'F', 1, 'G','G']
```

Decoding:

- Read -4 → repeat next byte 5 times → output "AAAAA"
- Read 3 → copy next 4 bytes → output "BCDE"
- Read -3 → repeat next byte 4 times → output "FFFF"
- Read 1 → copy next 2 bytes → output "GG"

Result matches original!

Example

PackBits Example 2: Long Literal Run
Input: `ABCDEFGHIJKLMNOP` (16 distinct letters)
Since PackBits literal chunks max at 128 bytes, we can encode as one chunk:

- Header = 15 (copy 16 bytes)
- Then all 16 letters
- Total: 1 header + 16 literals = 17 bytes
- Original: 16 bytes → slight expansion (1 byte)

Example

PackBits Example 3: Long Repeated Run
Input: 150 bytes of 'X'
We need multiple chunks:

- First chunk: header = -127 (repeat 128 times), byte 'X'
- Second chunk: header = -21 (repeat 22 times), byte 'X'
- Total: 2 headers + 2 bytes = 4 bytes
- Original: 150 bytes
- Compression ratio: 37.5:1 (97.3% savings!)

Example

PackBits Example 4: Boundary Cases

Single byte: Input: `X`

- Must encode as literal
- Header = 0 (copy 1 byte), then 'X'
- 2 bytes for 1 byte → 100% expansion

Two identical bytes: Input: `XX`

- Option 1: Encode as run of 2 → header = -1 (repeat 2 times), then 'X'
- Option 2: Encode as literals → header = 1 (copy 2 bytes), then 'X','X'
- Both use 3 bytes vs original 2 → expansion
- PackBits has no efficient encoding for pairs!

Two different bytes: Input: `XY`

- Literals: header = 1, then 'X','Y' → 3 bytes vs 2 → expansion

7.6.5 Optimal Use of PackBits

Important

When PackBits Excels:

- **Long runs (≥3):** Excellent compression
- **Mixed data with some long runs:** Good balance
- **Data with runs ≥128:** Still efficient (splits automatically)

When PackBits Struggles:

- **Short runs (pairs):** No efficient encoding (3 bytes for 2)
- **Alternating data:** All literals, slight expansion
- **Very short chunks:** Header overhead dominates

Break-even analysis:

- Literal chunk: cost = 1 + L bytes for L bytes (overhead = 1)

- Run chunk: cost = 2 bytes for L bytes (overhead = 2/L)
- For L=3, both cost 3 bytes
- For L=4, run saves 1 byte vs literals
- For L=2, run costs 3 bytes vs 2 bytes literals → worse

7.6.6 PackBits in Real-World Formats

Important

Applications of PackBits:

- **TIFF:** Optional compression method (tag 32773)
- **Macintosh resource forks:** Used extensively
- **PDF:** Can use PackBits for stream compression
- **PostScript:** Supported as a filter
- **Many embedded systems:** Simple, efficient implementation

Why PackBits remains popular:

- Simple to implement correctly
- No patents
- Works well on mixed data
- Byte-aligned (fast)
- Reasonable worst-case behavior (0.5-1 byte overhead per chunk)

7.7 Zero Run-Length Encoding (Zero RLE)

Zero RLE is a specialized variant that focuses exclusively on compressing runs of zero values. It's particularly important in transform-based compression systems.

7.7.1 Motivation: Sparse Data in Transform Coding

Important

Why Zero Runs Occur in Transform Coding:

1. **Energy compaction:** Transforms (DCT, wavelet) concentrate signal energy into few coefficients
2. **Quantization:** Small coefficients become zero after quantization
3. **High-frequency coefficients:** Often near-zero in natural images
4. **Zigzag scanning:** Orders coefficients from low to high frequency, creating long zero runs at the end

Definition

Zero Run-Length Encoding: A specialized form of RLE that focuses exclusively on runs of zero values, often combined with encoding the non-zero values that interrupt them.

7.7.2 Basic Zero RLE Algorithm

The simplest form of Zero RLE treats the data as a sequence of (zero run length, non-zero value) pairs.

Algorithm 11 Zero RLE Encoding

```
procedure ZeroRLE-Encode(S[1..n])
    result ← empty list
    i ← 1
    while i ≤ n do
        if S[i] = 0 then
            zero_run ← 0
            while i ≤ n and S[i] = 0 do
                zero_run ← zero_run + 1
                i ← i + 1
            end while
            if i > n then                          ▷ End-of-block special case
                append (zero_run, EOB) to result
            else
                append (zero_run, S[i]) to result
                i ← i + 1
            end if
        else
            append (0, S[i]) to result             ▷ Zero run length 0
            i ← i + 1
        end if
    end while
    return result
end procedure
```

7.7.3 Zero RLE in JPEG: A Detailed Case Study

JPEG's encoding of AC coefficients is the most famous example of Zero RLE.

Important

JPEG's Zero-Run Encoding:
After DCT and quantization, JPEG processes AC coefficients as follows:

1. **Zigzag scan:** Reorder 8×8 block in zigzag pattern to maximize zero runs
2. **Run-length encoding:** Encode as (RUNLENGTH, LEVEL) pairs
   - **RUNLENGTH:** Number of consecutive zeros preceding this coefficient (0-15)
   - **LEVEL:** Non-zero coefficient value
   - Special case: (0,0) = End-of-Block (EOB)

- Special case: (15,0) = ZRL (16 zeros, handled specially)

3. **Huffman coding:** Each (RUNLENGTH, LEVEL) pair is Huffman coded

Example

JPEG Zigzag Scan Example:
Original 8×8 block (quantized coefficients):

```
[150, -25, 0,   0,   0,   0,   12,  0
 0,   0,   0,   0,   0,   0,   0,   0
 0,   0,   0,   0,   0,   0,   0,   0
 0,   0,   0,   0,   0,   0,   0,   0
 0,   0,   0,   0,   0,   0,   0,   0
 0,   0,   0,   0,   0,   0,   0,   0
 0,   0,   0,   0,   0,   0,   0,   0
 0,   0,   0,   0,   0,   0,   0,   0]
```

Zigzag scan order (simplified):

- Position 1: 150 (DC coefficient - encoded separately)
- Positions 2-10: -25, 0, 0, 0, 0, 12, 0, 0, 0
- Remaining 54 positions: all zeros

Zero-run encoding:

- After DC, first AC: (0, -25) meaning 0 zeros then -25
- Next: (4, 12) meaning 4 zeros then 12
- Then: (0,0) EOB meaning rest are zeros

JPEG representation: (0,-25), (4,12), EOB

Example

JPEG Encoding Example:
Consider this AC coefficient sequence: 0, 0, 0, 5, 0, 0, -3, 0, 0, 0, 0, 2, [rest zeros]
Step 1: Identify zero runs and non-zero values:

- 3 zeros then 5
- 2 zeros then -3
- 4 zeros then 2

- All remaining zeros

Step 2: Encode as (RUNLENGTH, LEVEL):

- (3,5)
- (2,-3)
- (4,2)
- (0,0) for EOB

Step 3: Huffman coding: Each pair is Huffman coded using tables optimized for typical distributions. The Huffman tables assign shorter codes to common (RUNLENGTH, LEVEL) combinations.

7.7.4 Handling Long Zero Runs

JPEG's RUNLENGTH field is limited to 15, so special handling is needed for longer zero runs.

Important

JPEG's ZRL (Zero Run Length) Code:
When a zero run exceeds 15 zeros:

- Output (15,0) for each complete block of 16 zeros
- Then encode the remainder normally

Example: 37 zeros followed by a value 5

- 37 zeros = 16 + 16 + 5
- Output: (15,0), (15,0), (5,5)

This allows arbitrary-length zero runs while keeping the RUNLENGTH field small.

7.7.5 Zero RLE in Other Transform Codecs

Important

Zero RLE in Modern Codecs:
MPEG video codecs:

- Similar to JPEG for intra-coded blocks

- (run, level) coding for DCT coefficients
- Variable-length codes optimized for video statistics

JPEG 2000 (wavelet-based):

- Embedded block coding with optimized truncation (EBCOT)
- Fractional bit-plane coding
- Zero-run encoding implicit in bit-plane representation
- Arithmetic coding with context modeling

H.264/AVC:

- CAVLC (Context-Adaptive Variable-Length Coding)
- Encodes runs of zeros before non-zero coefficients
- Adapts to local statistics

7.7.6 Zero RLE Pipeline in Transform Coding

Important

Complete Zero-RLE Pipeline:

```
Raw Image → Transform → Quantization → Zigzag Scan →
Zero-RLE → Entropy Coding → Compressed Bitstream
```

Why this pipeline works:

1. **Transform:** Converts spatial redundancy to frequency coefficients
2. **Quantization:** Reduces precision, creates zeros
3. **Zigzag scan:** Reorders coefficients to maximize zero runs
4. **Zero-RLE:** Compresses zero runs efficiently
5. **Entropy coding:** Further compresses (run, level) pairs

7.8 Comparison of RLE Variants

7.8.1 Decision Matrix

Criteria	**Count-Based Fixed**	**Count-Based Variable**	**Escape-Based**	**PackBits**	**Zero RLE**
Implementation complexity	Very low	High	Medium	Medium	Medium
Speed	Very fast	Slow	Fast	Fast	Fast
Random access	Yes	No	No	No	No
Best for	Uniform runs	Skewed runs	Mixed data	Mixed data	Sparse data
Worst-case expansion	100%	100% (plus bit overhead)	Small (escape handling)	Small (header overhead)	Small
Max run length	Fixed limit	Unlimited	Unlimited	128 per chunk	Unlimited
Common uses	Simple formats, hardware	Advanced codecs, BWT pipeline	TIFF, some fax	TIFF, Mac formats	JPEG, MPEG, DCT-based codecs

7.8.2 When to Use Each Variant

Important

Selection Guide:

- **Choose Fixed-Width Count RLE when:**
  - You need maximum speed and simplicity
  - Run lengths are bounded and relatively uniform
  - Hardware implementation is required
  - Random access to runs is needed
- **Choose Variable-Width Count RLE when:**
  - Run length distribution is highly skewed (many short runs)
  - Maximum compression ratio is critical
  - You can tolerate bit-level I/O complexity
  - Part of a larger pipeline (e.g., BWT+MTF)

- **Choose Escape-Based RLE when:**
  - Data has mixed runs and literals
  - You need to guarantee no worst-case expansion
  - Simple byte-aligned implementation is desired
- **Choose PackBits when:**
  - You need a standard, widely-supported format
  - Data has moderate runs and literals
  - Implementation simplicity is important
  - Maximum run length of 128 is acceptable
- **Choose Zero RLE when:**
  - Working with transform-coded data (images, video)
  - Data is sparse with many zeros
  - Part of a larger compression pipeline
  - Non-zero values need special handling

7.9 Limitations of Run-Length Encoding

7.9.1 Why RLE Fails on Natural Language Text

Natural language text represents RLE's worst nightmare.

Important

Characteristics of Natural Language Text:

1. **Large alphabet:** 26 letters + punctuation + case + spaces
2. **Rare repeats:** In English, repeated letters are uncommon
   - Double letters: "book", "little", "success" — relatively rare
   - Triple letters: Almost nonexistent (except "beeeeep" in comics)
3. **Statistical distribution:** Letter frequencies follow Zipf's law
4. **Context structure:** Redundancy is in patterns, not runs

Example

RLE on Shakespeare's Sonnet:
Consider the opening of Sonnet 18: `Shall I compare thee to a summer's day?`
Run analysis:

- Total characters (including spaces): 500
- Runs of length 1: 490
- Runs of length 2: 5 (double letters: "ll" in "shall", "mm" in "summer")
- Runs of length 3: 0
- Average run length: 1.01

RLE encoding:

- Original: 500 bytes
- Encoded: 500 runs $\times$ 2 = 1000 values
- At 1 byte per value: 1000 bytes (100% expansion)

Why the expansion?

- Each run of length 1 requires a count of 1
- The count field adds 100% overhead to each character
- The few runs of length 2 save only 1 byte each (2 chars $\rightarrow$ 2 bytes vs 4 bytes for two singles)
- Savings from runs: 5 runs $\times$ (2 bytes saved per run) = 10 bytes saved
- But overhead from counts: 500 extra bytes
- Net loss: 490 bytes

7.9.2 Sensitivity to Symbol Ordering

RLE's effectiveness depends critically on how symbols are arranged.

Definition

Order Sensitivity: The property that RLE compression ratio can change dramatically if the same data is reordered, even though the multiset of symbols remains identical.

Example

Dramatic Effect of Ordering:
Consider the multiset: 100 A's, 100 B's, 100 C's
Ordering 1: Interleaved (ABC ABC ABC...) `ABCABCABC...` (300 bytes)

- Each run length = 1
- Number of runs = 300
- Encoded size: 600 values (assuming 1-byte counts)
- Result: 100% expansion!

Ordering 2: Grouped (AAA...BBB...CCC...) `AAA... (100) BBB... (100) CCC... (100)`

- 3 runs, each length 100
- Encoded size: 3×2 = 6 values
- Result: 98% compression!

7.10 RLE as a Preprocessing Technique

7.10.1 Combining RLE with Huffman Coding

The RLE+Huffman combination is a classic and effective approach.

Important

Why Combine RLE and Huffman:

- **RLE handles runs:** Converts repeated symbols into (symbol, count) pairs
- **Huffman handles frequencies:** Assigns shorter codes to frequent symbol-/count combinations
- **Synergy:** Each addresses a different type of redundancy

Example

CCITT Group 3 Fax: The Classic Example
Group 3 fax does exactly this:

- Step 1: Identify runs of white and black
- Step 2: For each run length, use a Huffman code

- Step 3: Special make-up codes for long runs
- Step 4: Separate Huffman tables for white and black runs

Result: Achieves 10-20:1 compression on typical documents.

7.10.2 The Crucial Role of RLE in BWT Pipeline

This is the key connection to the next chapter.

Important

The BWT Pipeline:

`Original Text → BWT → MTF → RLE → Huffman/Arithmetic`

Why RLE is essential after BWT+MTF:

1. **BWT groups similar contexts:** Characters with similar contexts end up near each other
2. **MTF converts to small numbers:** Recently seen characters get small indices (often 0)
3. **Result:** Long runs of zeros and other small numbers appear
4. **RLE compresses these runs:** Exactly what RLE is good at!

7.11 Summary

Important

Key Takeaways:

1. **Equality redundancy** is the simplest form of data redundancy—consecutive identical symbols.
2. **Four major RLE variants** address different data characteristics:
   - *Count-Based RLE:* Classic (symbol, count) approach. Fixed-width for simplicity, variable-width for optimal compression.
   - *Escape-Based RLE:* Uses markers to distinguish runs from literals. Prevents worst-case expansion.
   - *Block-Based RLE (PackBits):* Header-based chunks balance overhead and efficiency.

- *Zero RLE:* Specialized for sparse data, essential in transform coding.

3. **Performance bounds:**
   - Best case: near-infinite compression (single run)
   - Worst case: 100% expansion (no runs, fixed-width)
   - Break-even: depends on count field size and run length distribution
4. **RLE is passive:** It can only exploit existing runs. This limitation motivates transform-based compression (BWT) which *creates* runs.

Preview: The Burrows-Wheeler Transform

From RLE to BWT: The Evolution of Compression Thinking

Level 1: "I see runs, I compress runs." (Basic RLE)

Level 2: "I adapt RLE to handle different situations." (RLE variants)

Level 3: "I transform data to create the runs RLE needs." (BWT)

The next chapter will show you how to achieve Level 3 thinking.

8: Lecture 8: Transform-Based Compression: The Burrows–Wheeler Transform

8.1 The Three Paradigms of Lossless Compression

Before diving into the Burrows-Wheeler Transform (BWT), let us step back and consider the intellectual journey we have taken so far. Lossless compression algorithms generally fall into three broad paradigms:

1. **Statistical/Entropy Coding (Chapters 1-4):** These methods build a probability model of the source and assign shorter codes to more probable symbols. Huffman coding and arithmetic coding are the quintessential examples. The fundamental limit is the source entropy $H(S)$.

2. **Dictionary Methods (Chapters 5-6):** Instead of modeling probabilities, these methods identify repeated phrases and replace subsequent occurrences with references to earlier ones. LZ77, LZ78, and their descendant DEFLATE dominate practical compression due to their speed and simplicity.

3. **Transform-Based Methods (Chapters 7-8):** These methods do not compress directly. Instead, they *reorganize* the data to make it more amenable to compression by subsequent stages. In Chapter 7, we studied Run-Length Encoding (RLE), which exploits equality redundancy. In this chapter, we explore the Burrows-Wheeler Transform, which *creates* the very runs that RLE excels at compressing.

This chapter introduces the third paradigm through the lens of the Burrows-Wheeler Transform, showing how a reversible sorting operation can dramatically improve compressibility when combined with the RLE variants from Chapter 7.

8.2 Motivation: Beyond Local Matching

8.2.1 The Limits of Sliding Windows

Recall from Chapter 5 that LZ77-based compressors (like DEFLATE) use a sliding window to find repetitions. While effective, this approach has fundamental limitations:

- **Local vs. Global Redundancy:** LZ77 can only detect repetitions that occur within the window (typically 32KB in DEFLATE). Repetitions separated by larger distances are missed entirely.

- **The "Shakespeare Problem":** Consider the complete works of Shakespeare. The phrase "to be or not to be" appears multiple times throughout the text, but these occurrences might be hundreds of kilobytes apart. An LZ77 compressor with a

32KB window will treat each occurrence as novel text, missing this global repetition entirely.

- **Computational Cost of Large Windows:** While we could increase the window size (LZMA uses up to 4GB), the cost of searching grows significantly. More importantly, the *pointer* to reference distant matches requires more bits to encode, eating into compression gains.

Important

The fundamental insight of the Burrows-Wheeler Transform is this: instead of searching for repetitions, why not *rearrange* the data so that similar contexts—and hence similar symbols—are brought together? This rearrangement is reversible, so the original data can be recovered exactly.

8.2.2 The Global Reordering Idea

Imagine you have a long string of text. If you could somehow gather all characters that appear in similar contexts (e.g., all characters that follow "th"), they would cluster together. Within these clusters, the data would exhibit strong local correlations—exactly the kind of runs that the RLE variants from Chapter 7 can exploit.

This is precisely what the Burrows-Wheeler Transform achieves. It does not compress data; it *transforms* data into a form where:

- Identical characters tend to appear in runs
- The local structure is significantly enhanced
- The RLE variants from Chapter 7 become extraordinarily effective

Definition

[The Transform-Then-Compress Principle] Many modern compression algorithms follow a two-stage process:

1. **Transform:** Apply a reversible transformation to the data that reveals its structure (redundancy becomes explicit).
2. **Compress:** Apply statistical or dictionary coding to the transformed data.

The Burrows-Wheeler Transform is the canonical example of this principle in lossless compression.

8.3 The Burrows–Wheeler Transform

8.3.1 Historical Background

The Burrows-Wheeler Transform was developed by Michael Burrows and David Wheeler at DEC Systems Research Center in 1994. Their technical report, "A Block-sorting Lossless Data Compression Algorithm," introduced a completely new approach to compression that combined:

- The global structure exploitation of statistical methods
- The speed and simplicity of dictionary methods
- A clever use of sorting—a fundamental computer science operation

The algorithm was immediately recognized as revolutionary. Within years, it became the basis for `bzip2`, which offered significantly better compression than `gzip` on many file types, particularly text.

8.3.2 Intuition: Grouping Similar Contexts

The core idea of BWT is deceptively simple:

1. Generate all *cyclic rotations* of the input string (with a special end-of-string marker).
2. Sort these rotations lexicographically (dictionary order).
3. Extract the last column of the sorted rotation matrix.

The remarkable property is that the last column tends to contain long runs of identical characters. Why? Because rows that start with similar prefixes (contexts) are adjacent in the sorted order, and their last characters—which are the characters *preceding* those contexts in the original string—tend to be similar.

Example

[Intuition Through a Simple Example] Consider the word "BANANA". In English text, the letter 'N' often follows 'A'. In the BWT output, all the characters that precede 'N' in the original string will be grouped together because all rotations starting with 'N' are adjacent in the sorted list. This clustering effect is what makes the transform powerful.

8.3.3 Step-by-Step Example: `banana$`

Let us work through a complete example to understand the mechanics. We will use the string "banana" with a special end-of-file marker '$' that is lexicographically smaller than any other character.

Example

[Computing the BWT of "banana$"]

Step 1: Generate all cyclic rotations

Original string (with marker): `banana$`

All rotations (starting at each character and wrapping around):

1. banana$
2. anana$b
3. nana$ba
4. ana$ban
5. na$bana
6. a$banan
7. $banana

Step 2: Sort rotations lexicographically

We sort these strings as you would in a dictionary:

Index	Sorted Rotation	Original Index
1	$banana	7
2	a$banan	6
3	ana$ban	4
4	anana$b	2
5	banana$	1
6	na$bana	5
7	nana$ba	3

Step 3: Extract the last column

The BWT output is the last character of each sorted rotation:

Sorted Rotation	Last Character
$banana	a
a$banan	n
ana$ban	n
anana$b	b
banana$	$
na$bana	a
nana$ba	a

Thus, **BWT("banana$") = "annb$aa"**

Notice the runs: "nn" appears, and "aa" appears at the end. The original "banana" had no consecutive identical characters!

8.3.4 Visualizing the Effect: Before and After

The transformation becomes clearer when we visualize the effect:

Original: banana$** → **BWT Output: annb$aa

b	a	n	a	n	a	$

a	n	n	b	$	a	a
	run				run	

The BWT output has:

- A run of two 'n's (positions 2-3)
- A run of two 'a's at the end (positions 6-7)
- The original had *no* runs at all!

8.4 Formal Definition of the BWT

8.4.1 Cyclic Rotations and Lexicographic Sorting

Let us formalize what we have done. Given an input string S of length n with a unique end-of-string marker $ that is lexicographically smaller than all other characters:

Definition

[Burrows-Wheeler Transform]
The Burrows-Wheeler Transform of S, denoted BWT(S), is computed as:

1. Form a matrix M of size $n \times n$ where row i (0-indexed) is the cyclic rotation of S starting at position i:

$$M[i] = S[i:] + S[:i]$$

2. Sort the rows of M in lexicographic order (dictionary order, with '$' < all other characters).
3. Let I be the index of the row containing the original string S in the sorted matrix.
4. The BWT output L is the last column of the sorted matrix:

$$L[j] = M_{\text{sorted}}[j][n-1]$$

5. The transform also outputs I, the index of the original row, which is needed for inversion.

Thus, $\text{BWT}(S) = (L, I)$.

In our "banana$" example:

- $L =$ "annb$aa"
- $I = 4$ (the original string "banana$" appears at row 5, which is index 4 in 0-based indexing)

8.4.2 The Last Column Construction

Why do we take the last column? Consider any two rows that are adjacent in the sorted order. They share a common prefix (the longer the prefix, the more similar their contexts). The last characters of these rows are the characters that *precede* these contexts in the original string. When contexts are similar, the preceding characters often come from a limited set, creating runs.

Important

The BWT is a *reversible* transformation. Given only L (the last column) and I (the index of the original row), we can reconstruct the original string S exactly. This reversibility is what makes the transform useful for compression—we can recover the original data after decompression.

8.4.3 Properties of the Transform

The BWT has several remarkable properties:

1. **Reversibility:** The transform is bijective—every input string maps to a unique (L, I) pair, and the mapping is invertible.
2. **Run Formation:** The output L tends to contain long runs of identical characters, especially in structured data like text.
3. **Context Grouping:** Characters that appear in similar contexts in the original string are brought together in the output.
4. **Structure Revealing:** Higher-order dependencies in the original string become lower-order dependencies in the transformed string.
5. **Block Structure:** The transform operates on fixed-size blocks (typically 100KB–900KB), allowing a trade-off between compression ratio and memory usage.

8.5 Inverting the BWT

The inversion algorithm is where the true cleverness of the BWT reveals itself. Given only L (the last column) and I (the index of the original row), we can reconstruct the original string without ever storing the full rotation matrix.

8.5.1 The First–Last (LF) Mapping

The key insight is that the first column F of the sorted rotation matrix can be derived from L alone:

1. F is simply L sorted lexicographically.
2. Why? Because the rows are sorted lexicographically, the first column is just all characters of the original string in sorted order.

In our example:

- $L = [a, n, n, b, \$, a, a]$
- Sorting L gives: $[\$, a, a, a, b, n, n] = F$

Now we have a crucial observation: For any character in the last column, we can find its corresponding character in the first column through a mapping that preserves the relative order of identical characters.

Definition

[LF-Mapping] The LF-mapping (Last-to-First mapping) is a function $\mathrm{LF}(i)$ that returns the row index in the first column that corresponds to the same character occurrence as position i in the last column.
For a character c that appears k times in the string, the k occurrences appear in the same order in both F and L. Therefore, if $L[i]$ is the r-th occurrence of character c in L (counting from 1), then $\mathrm{LF}(i)$ is the position of the r-th occurrence of c in F.

8.5.2 Algorithm for Computing LF-Mapping

Algorithm 12 Compute LF-Mapping

Require: $L[0..n-1]$ (last column)

Ensure: $LF[0..n-1]$ (last-to-first mapping)

```
n ← length(L)
F ← sort(L)                                          ▷ First column
                                                     ▷ Count occurrences of each character
Initialize arrays C[|Σ|] and rank[0..n − 1]
for i ← 0 to n − 1 do
    c ← L[i]
    rank[i] ← count of c seen so far in L            ▷ 1-based rank
    Increment C[c]
end for
                                                     ▷ Build cumulative counts for first column
cum[0] ← 0
for each character c in sorted order do
    cum[c] ← cum[prev] + C[prev]
end for
                                                     ▷ Compute LF mapping
for i ← 0 to n − 1 do
    c ← L[i]
    LF[i] ← cum[c] + (rank[i] − 1)                   ▷ Subtract 1 for 1-based rank
end for
return LF
```

Example

[Step-by-Step LF-Mapping for "annb$aa"]
We will compute the LF-mapping by building it incrementally, explaining each operation in detail.

Step 1: Understanding the Goal

The LF-mapping answers: "If I am at row i in the last column L, which row in the first column F contains the SAME occurrence of the SAME character?"
This works because:

- The first column F is just L sorted
- Identical characters appear in the same order in both columns
- The k-th occurrence of character c in L corresponds to the k-th occurrence of c in F

Step 2: Our Data

The last column L from BWT("banana$") is:

$$L = [a,\ n,\ n,\ b,\ \$,\ a,\ a]$$

Positions (indices): 0 1 2 3 4 5 6

Step 3: Compute the First Column F

F is simply L sorted in ascending order (with '$' ¡ 'a' ¡ 'b' ¡ 'n'):

$$\text{Sort}(L) = [\$,\ a,\ a,\ a,\ b,\ n,\ n]$$

So the first column F is:

$$F = [\$,\ a,\ a,\ a,\ b,\ n,\ n]$$

Positions in F: 0 1 2 3 4 5 6

Step 4: Count Occurrences in L (Build Rank Array)

We go through L from left to right, counting how many times we have seen each character so far. This count is called the "rank" - it tells us "this is the rank-th occurrence of this character."

Index i	L[i]	Rank in L	Explanation
0	a	1	First 'a' encountered in L (none seen before)
1	n	1	First 'n' encountered in L (none seen before)
2	n	2	Second 'n' (we saw one 'n' before at i=1)
3	b	1	First 'b' encountered in L (none seen before)
4	$	1	First '$' encountered in L (none seen before)
5	a	2	Second 'a' (we saw one 'a' before at i=0)
6	a	3	Third 'a' (we saw two 'a's before at i=0 and i=5)

So the rank array R (matching each position in L to its occurrence number) is:

$$R = [1,\ 1,\ 2,\ 1,\ 1,\ 2,\ 3]$$

Step 5: Count Occurrences in F

Now we count how many times each character appears in the first column F:

Character	Count in F	Positions in F
$	1	Position 0
a	3	Positions 1, 2, 3
b	1	Position 4
n	2	Positions 5, 6

Step 6: Build Cumulative Starting Positions for F

For each character, we need to know: "Where does the first occurrence of this character appear in F?" This is the cumulative count.
We calculate cum[c] = the starting index of character c in F:

Character	Count	Cumulative Calculation	cum[c]
$	1	cum[$] = 0 (first character)	0
a	3	cum[a] = cum[$] + count[$] = 0 + 1	1
b	1	cum[b] = cum[a] + count[a] = 1 + 3	4
n	2	cum[n] = cum[b] + count[b] = 4 + 1	5

This means:

- The first '$' in F is at position cum[$] = 0
- The first 'a' in F is at position cum[a] = 1
- The second 'a' in F is at position cum[a] + 1 = 2

- The third 'a' in F is at position cum[a] + 2 = 3
- The first 'b' in F is at position cum[b] = 4
- The first 'n' in F is at position cum[n] = 5
- The second 'n' in F is at position cum[n] + 1 = 6

Step 7: The Key Formula

For any position i in L:

- Let c = L[i] (the character at that position)
- Let r = rank[i] (this is the r-th occurrence of c in L)
- In F, the r-th occurrence of c appears at: cum[c] + (r - 1)

Why (r - 1)? Because:

- The first occurrence (r=1) goes to cum[c] + 0
- The second occurrence (r=2) goes to cum[c] + 1
- The third occurrence (r=3) goes to cum[c] + 2

Therefore:

$$LF[i] = cum[c] + (rank[i] - 1)$$

Step 8: Compute LF for Each Position

Now we apply this formula to each position i:

i	**L[i]**	**c**	**r = rank[i]**	**Formula Applied**	**Calculation**	**LF[i]**
0	a	a	1	cum[a] + (1-1)	1 + 0	1
1	n	n	1	cum[n] + (1-1)	5 + 0	5
2	n	n	2	cum[n] + (2-1)	5 + 1	6
3	b	b	1	cum[b] + (1-1)	4 + 0	4
4	$	$	1	cum[$] + (1-1)	0 + 0	0
5	a	a	2	cum[a] + (2-1)	1 + 1	2
6	a	a	3	cum[a] + (3-1)	1 + 2	3

Step 9: Understanding Each Result

Let's verify each mapping to build intuition:

LF[i]	Explanation
LF[0] = 1	L[0] is the first 'a'. In F, the first 'a' is at position 1.
LF[1] = 5	L[1] is the first 'n'. In F, the first 'n' is at position 5.
LF[2] = 6	L[2] is the second 'n'. In F, the second 'n' is at position 6.
LF[3] = 4	L[3] is the first 'b'. In F, the first 'b' is at position 4.
LF[4] = 0	L[4] is the first '$'. In F, the first '$' is at position 0.
LF[5] = 2	L[5] is the second 'a'. In F, the second 'a' is at position 2.
LF[6] = 3	L[6] is the third 'a'. In F, the third 'a' is at position 3.

Step 10: Final LF-Mapping Array

$$LF = [1,\ 5,\ 6,\ 4,\ 0,\ 2,\ 3]$$

This array tells us: from row i in L, go to row LF[i] in F to find the same character occurrence.

8.5.3 Reconstruction Algorithm

With the correct LF-mapping, we can now reconstruct the original string by walking backwards from the known original row:

Algorithm 13 BWT Inversion

Require: $L[0..n-1]$ (last column), I (index of original row)
Ensure: Original string S

```
n ← length(L)
Compute LF-mapping array LF[0..n − 1] using Algorithm 8.2
Initialize S as empty string
row ← I
for k ← 0 to n − 1 do
    Prepend L[row] to S                ▷ Add character to front
    row ← LF[row]                      ▷ Move to previous row
end for
return S
```

Example

[Step-by-Step Inversion of "annb$aa" with I=4]
Now we use the LF-mapping we computed to reconstruct the original string.

Step 1: Initial Setup

We have:

- Last column L = [a, n, n, b, $, a, a]
- LF-mapping = [1, 5, 6, 4, 0, 2, 3]
- Primary index I = 4 (this tells us the original string is at row 4 in the sorted rotations matrix)

We start at row = I = 4.

Step 2: Walk Backwards Through the Rows

At each step, we:

1. Take the character at L[current row] and add it to the FRONT of our result
2. Move to the next row using LF-mapping: row = LF[row]

Step	**Current** Row	**Char =** L[row]	**Next Row =** LF[row]	**Explanation**
Start	4	–	–	Begin at primary index
1	4	$	LF[4] = 0	Take '$', move to row 0
2	0	a	LF[0] = 1	Take 'a', move to row 1
3	1	n	LF[1] = 5	Take 'n', move to row 5
4	5	a	LF[5] = 2	Take 'a', move to row 2
5	2	n	LF[2] = 6	Take 'n', move to row 6
6	6	a	LF[6] = 3	Take 'a', move to row 3
7	3	b	LF[3] = 4	Take 'b', move to row 4 (back to start)

Step 3: Build the String by Prepending

We add each character to the FRONT of our result string:

Step	Character Prepended	Current String S	Explanation
1	$	"$"	Start with just the '$'
2	a	"a$"	Add 'a' to front
3	n	"na$"	Add 'n' to front
4	a	"ana$"	Add 'a' to front
5	n	"nana$"	Add 'n' to front
6	a	"anana$"	Add 'a' to front
7	b	"banana$"	Add 'b' to front

Step 4: Remove the Marker

The reconstructed string is "banana$". Removing the end marker '$' gives us the original: "banana".

Why This Works

The LF-mapping creates a cycle that visits every row exactly once:

$$4 \to 0 \to 1 \to 5 \to 2 \to 6 \to 3 \to 4$$

Walking this cycle backwards and collecting L[row] at each step reconstructs the original string in reverse order. Prepending each character (rather than appending) gives us the correct forward order.
This is the magic of the BWT - we can recover the original string using only L and I, without ever storing the full rotation matrix!

8.5.4 Proof of Correctness (Sketch)

The inversion algorithm works because of a fundamental property of the sorted rotation matrix:

Theorem 8.1 (LF-Mapping Property)**.** *For any row i in the sorted rotation matrix, the character $L[i]$ (last column) is the character that precedes $F[i]$ (first column) in the original string. Moreover, the row $LF[i]$ in the sorted matrix corresponds to the rotation that starts with $L[i]$.*

This creates a chain that cycles through all rows exactly once. Starting from the row containing the original string (index I) and following the LF-mapping backwards, we visit each row exactly once in reverse cyclic order, allowing us to reconstruct the original string by collecting the last column characters in reverse order.

8.6 Why BWT Improves Compressibility

8.6.1 Run Formation and Structure Revelation

The BWT's power for compression comes from its tendency to create runs of identical characters. To understand why, consider the following:

1. In the sorted rotation matrix, rows with similar prefixes are adjacent.
2. The last column contains the characters that *precede* these prefixes in the original string.
3. If the original text has statistical regularities (e.g., 'q' is almost always followed by 'u'), then all rows starting with 'u' will be adjacent, and their last characters will all be 'q' (or characters that commonly precede 'u').

Important

[Important Clarification on Entropy] It is crucial to understand that the BWT does **not** reduce the zero-order entropy of the data. In fact, the zero-order entropy of L equals that of S because the transform is a permutation of the original characters. What the BWT achieves is something more subtle: it transforms **higher-order dependencies** in the original string into **lower-order dependencies** in the transformed string. After BWT, patterns that were previously spread across the text become localized, making them exploitable by simple zero-order coding methods like Move-To-Front and the RLE variants we studied in Chapter 7.
In other words, the BWT rearranges the data so that the *conditional probabilities* that characterize higher-order structure become visible as *empirical frequencies* that zero-order coders can exploit.

Example

[Visualizing the Effect] Consider English text. The character 'u' frequently follows 'q'. In the original text, these 'u's are scattered. In the BWT output:

- All occurrences of 'u' (as first column entries) will be grouped together because rotations starting with 'u' are adjacent in sorted order.
- Their corresponding last column entries will be the characters that preceded these 'u's—often 'q'.
- Result: A run of 'q's appears in the output, which can be efficiently compressed by the Count-Based RLE (fixed or variable-width) we studied in Chapter 7.

8.6.2 Connection to Context Modeling

Recall from Chapter 4 that higher-order Markov models capture dependencies by considering the previous k characters. The BWT achieves something analogous without explicit probability modeling:

- In Chapter 4, we built explicit models: $P(\text{next symbol} \mid \text{previous } k \text{ symbols})$
- In BWT, sorting rotations groups symbols that share the same context (the following characters)
- The result is that conditional probabilities become visible as local patterns in the output

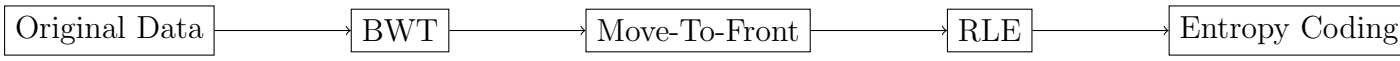
This is why BWT can be seen as an *implicit context modeling* transform—it reorganizes data so that context-dependent structure becomes spatially local.

Important

The BWT doesn't compress; it reveals structure. The actual compression is done by subsequent stages that exploit the local correlations the BWT creates. This is where the RLE variants from Chapter 7 become essential.

8.7 The Full BWT Compression Pipeline

A complete BWT-based compressor typically consists of several stages:



8.7.1 Stage 1: BWT Transform

The BWT rearranges the data to create local correlations and runs. As we saw with "banana$", the output contains runs that didn't exist in the original.

8.7.2 Stage 2: Move-To-Front (MTF) Encoding

After the BWT, we have a string with local correlations but not yet in a form optimal for entropy coding. Move-To-Front encoding transforms this string into a sequence of small integers that are highly compressible.

Definition

[Move-To-Front Encoding] The MTF encoder maintains a list of symbols (typically all 256 byte values). For each input symbol:

1. Output its current index in the list (0-based)

2. Move that symbol to the front of the list

Algorithm 14 Move-To-Front Encoding

Require: Input sequence $X[0..n-1]$ over alphabet Σ

Ensure: Output sequence $Y[0..n-1]$ of indices

```
Initialize list L ← sorted(Σ)                    ▷ Typically 0..255 in order
for i ← 0 to n − 1 do
    Find position p such that L[p] = X[i]
    Y[i] ← p
    Remove L[p] from list and insert at front
end for
return Y
```

Example

[MTF of BWT Output] For pedagogical clarity, we use a simplified alphabet containing only the characters that appear in our example: $[a, b, n, \$]$. In practice, MTF operates on the full byte alphabet (0–255), but the principle remains identical.

Apply MTF to "annb$aa" with initial list [a, b, n, $] (sorted order for predictability):

Input	Current List	Index	Output
a	[a, b, n, $]	0	0
	Move a to front → [a, b, n, $] (already at front)		
n	[a, b, n, $]	2	2
	Move n to front → [n, a, b, $]		
n	[n, a, b, $]	0	0
	Move n to front → [n, a, b, $] (already at front)		
b	[n, a, b, $]	2	2
	Move b to front → [b, n, a, $]		
$	[b, n, a, $]	3	3
	Move $ to front → [$, b, n, a]		
a	[$, b, n, a]	3	3
	Move a to front → [a, $, b, n]		
a	[a, $, b, n]	0	0
	Move a to front → [a, $, b, n]		

MTF output: [0, 2, 0, 2, 3, 3, 0]

Notice how runs of identical symbols in the BWT output become runs of zeros in the MTF output! This is the key to BWT's effectiveness—local structure in the

BWT output translates to runs of small numbers after MTF.

8.7.3 Stage 3: Run-Length Encoding (RLE)

After MTF, we have a sequence dominated by small numbers, often with long runs of zeros. This is where the RLE variants from Chapter 7 become essential.

Important

Choosing the Right RLE Variant for BWT Output:
The MTF output after BWT has specific characteristics that make certain RLE variants particularly effective:

- **Long runs of zeros:** Common, especially in highly structured text
- **Occasional small non-zero values:** Typically 1, 2, 3, etc.
- **Few large values:** Rare, but possible

Based on these characteristics:

- **Zero RLE (Chapter 7, Section 7.6):** Highly effective for the long zero runs. Can be combined with encoding the non-zero values.
- **Variable-Width Count RLE (Chapter 7, Section 7.4.2):** Optimal for the skewed distribution of run lengths (many short runs, few long runs).
- **Escape-Based RLE (Chapter 7, Section 7.5):** Useful if the data has mixed runs and isolated values.
- **PackBits (Chapter 7, Section 7.6):** Simple and effective, though the 128-byte chunk limit may require splitting long zero runs.

`bzip2` uses a custom RLE stage optimized for MTF output, but the principles are the same.

Example

[RLE of MTF Output Using Different Variants]
MTF output: [0, 2, 0, 2, 3, 3, 0]
Option 1: Count-Based Fixed-Width RLE (assuming 4-bit counts)

- (0,1), (2,1), (0,1), (2,1), (3,2), (0,1)
- Each pair: 1 byte for value + 0.5 bytes for count = 1.5 bytes
- Total: 6 pairs $\times$ 1.5 = 9 bytes

Option 2: Count-Based Variable-Width RLE (Elias gamma)

- Values: 0,1,2,3 → encode with Elias gamma
- Counts: mostly 1, one 2 → variable-length
- Total: approximately 25-30 bits ≈ 3.5-4 bytes
- Better compression but more complex

Option 3: Zero RLE (focus on zero runs)

- Zero runs: (1), then later (1), then final (1) — not very effective here
- Better for longer zero runs

Option 4: PackBits

- Since most values are isolated, mostly literal chunks
- Header per chunk adds overhead
- Would likely expand this short sequence

This example shows that for short sequences, simpler methods may be sufficient, but for real data with long zero runs, the more sophisticated RLE variants shine.

8.7.4 Stage 4: Final Entropy Coding (Huffman or Arithmetic)

The final stage applies statistical coding (Huffman or arithmetic coding) to the RLE output. Because the data now consists of small integers with skewed distributions, entropy coding achieves excellent compression ratios.

Example

[Complete BWT Pipeline for "banana$"]
Let's trace the complete pipeline:

1. **Original:** `banana$`
2. **BWT:** `annb$aa` (with I=4)
3. **MTF:** [0, 2, 0, 2, 3, 3, 0]
4. **RLE (simplified):** (0,1), (2,1), (0,1), (2,1), (3,2), (0,1)
5. **Huffman coding:** Assign codes based on frequencies

- (0,1) appears 3 times → shortest code
- (2,1) appears 2 times → medium code
- (3,2) appears 1 time → longer code

The final compressed size is much smaller than the original 7 characters!

8.8 Why RLE is Essential After MTF

The connection between BWT and the RLE variants from Chapter 7 is profound:

Important

The BWT-MTF-RLE Synergy:

1. **BWT creates local correlations:** Characters that appear in similar contexts are grouped together.
2. **MTF converts correlations to runs:** When the same character appears multiple times in the BWT output, MTF outputs zeros. When similar characters appear (e.g., 'a' then 'b' that recently moved to front), MTF outputs small numbers.
3. **RLE compresses the runs:** The long runs of zeros and short runs of small numbers are exactly what the RLE variants from Chapter 7 are designed to compress efficiently.

Without RLE, the BWT+MTF combination would be much less effective. Without understanding RLE, you cannot fully understand BWT.

Example

[Real-World: How bzip2 Uses RLE]
`bzip2` actually uses RLE in two places:

1. Preliminary RLE (before BWT):

- Compresses runs of 4 or more identical characters
- Simplifies later stages by reducing extreme cases
- Example: `AAAAAA` → `AAAA` with a special marker

2. Main RLE (after MTF):

- Compresses runs in the MTF output
- Uses a custom encoding optimized for the distribution

- Handles runs of zeros and other small values

This two-stage approach shows how different RLE variants can be combined in a single compressor.

8.9 Efficient Implementation

8.9.1 Naive Rotation Matrix: Why It Is Impractical

The naive BWT implementation described earlier—constructing an $n \times n$ matrix of rotations—is completely impractical for real data:

- For a 1MB file ($n \approx 10^6$), storing all rotations would require 10^{12} characters (1TB) of memory!
- Even storing indices rather than full strings, we need n rotations of length n, which is $O(n^2)$ space.

Clearly, we need a more efficient approach.

8.9.2 Suffix Arrays for Efficient Construction

The key insight is that we don't need to store all rotations explicitly. For a string with a unique end marker, the cyclic rotations correspond exactly to the suffixes of the string when we consider the end marker as a special character.

Definition

[Suffix Array] A suffix array for a string T of length n is an array of integers $SA[0..n-1]$ that gives the starting positions of the suffixes of T in lexicographic order.

For BWT construction, we only need the suffix array of $S\$$ (the original string with the unique end marker). The relationship is:

- The suffix starting at position i corresponds to the rotation that begins at position i in the original string.
- The lexicographic order of suffixes is identical to the lexicographic order of rotations because the end marker ensures no suffix is a prefix of another.

Algorithm 15 Efficient BWT Construction Using Suffix Arrays

Require: Input string S of length n with unique end marker '\$'

Ensure: BWT output $L[0..n-1]$ and primary index I

```
T ← S                                          ▷ T includes the end marker
Build suffix array SA for T                    ▷ O(n log n) or O(n) time
for i ← 0 to n − 1 do
    pos ← SA[i]
    if pos = 0 then
        L[i] ← $                               ▷ Last character when suffix starts at beginning
        I ← i                                  ▷ Record position of original string
    else
        L[i] ← T[pos − 1]                      ▷ Character before this suffix
    end if
end for
return (L, I)
```

8.9.3 Time and Space Complexity

Modern BWT implementations achieve excellent efficiency:

- **Time Complexity:** $O(n \log n)$ for sorting-based suffix array construction, or $O(n)$ for advanced algorithms (SA-IS, DC3)
- **Space Complexity:** $O(n)$ for the suffix array plus input/output buffers
- **Practical Performance:** `bzip2` processes data at several MB/s, with block sizes typically 100KB–900KB

Important

The development of practical linear-time suffix array construction algorithms and efficient in-place implementations was crucial for making BWT feasible for real-world compression.

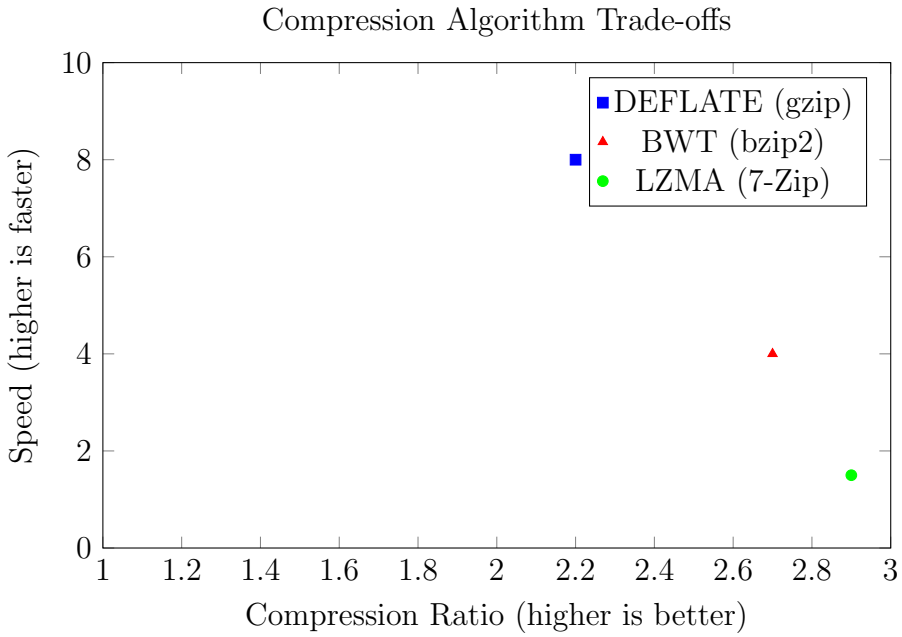
8.10 Comparison with Lempel–Ziv Methods

8.10.1 Local vs. Global Redundancy

The fundamental difference between LZ77 and BWT lies in how they exploit redundancy:

Aspect	LZ77 (DEFLATE)	BWT (bzip2)
Redundancy Type	Local (within sliding window)	Global (across entire block)
Detection Method	String matching in window	Sorting all rotations
Memory Reference	Explicit (distance, length)	Implicit (context grouping)
RLE Role	Optional (some implementations)	Essential (after MTF)
Adaptivity	Online (streaming)	Block-based

8.10.2 Compression Ratio vs. Speed Trade-offs



8.11 Case Study: `bzip2`

8.11.1 Architecture Overview

`bzip2` is the canonical BWT-based compressor. Its pipeline is:

1. **Run-Length Encoding Prelim:** A preliminary RLE stage to compress repeated characters (simplifies later stages)
2. **Burrows-Wheeler Transform:** Applied to blocks (default 900KB)
3. **Move-To-Front Transform:** Converts local correlations into runs of zeros
4. **Run-Length Encoding:** Compresses runs from MTF (custom variant)
5. **Huffman Coding:** Multiple Huffman tables for different parts of the data

Important

`bzip2`'s RLE stage after MTF is specifically designed for the characteristics of MTF output:

- Long runs of zeros are encoded with a special marker
- Small non-zero values are encoded directly
- The encoding is byte-aligned for speed

This is essentially a specialized version of the Zero RLE variant we studied in Chapter 7.

8.11.2 Why BWT Beats DEFLATE on Text

On natural language text, `bzip2` typically achieves 15-30% better compression than `gzip` because:

- Text has global structure (frequent words, common phrases) that spans entire documents
- LZ77's window limits miss these long-range dependencies
- BWT's global reordering captures these dependencies implicitly
- The RLE stages efficiently compress the resulting runs

Example

[Compression Comparison] On the Canterbury Corpus (standard compression benchmark):

- File: `alice29.txt` (Alice in Wonderland, 152KB)
- `gzip -9`: 56KB (compression ratio 2.71)
- `bzip2 -9`: 44KB (compression ratio 3.45)
- Improvement: 21% better compression

8.12 Limitations and Modern Extensions

8.12.1 Block Size Constraints

BWT operates on fixed-size blocks. The choice of block size involves trade-offs:

- **Larger blocks:** Better compression (more context), but more memory, slower processing
- **Smaller blocks:** Faster, less memory, but poorer compression

`bzip2` allows block sizes from 100KB to 900KB. Beyond 900KB, compression gains diminish while memory usage grows linearly.

8.12.2 Memory Usage Considerations

BWT requires:

- $O(n)$ memory for the suffix array (4-8 bytes per character)
- Additional memory for MTF and RLE stages
- For 900KB blocks, `bzip2` uses about 7MB of RAM—modest by modern standards but significant for embedded systems

8.12.3 When BWT May Not Improve Compression

While BWT is reversible and theoretically cannot reduce compressibility, in practice several factors can lead to worse compression than simpler methods:

- **Block overhead:** The transform operates on fixed-size blocks. Very small blocks ($<$ 1KB) may have overhead that outweighs any compression benefit.
- **Primary index storage:** The index I must be stored with each block, adding a small constant overhead.
- **MTF inefficiency on random data:** If the input has no structure, sorting creates no useful patterns, and MTF may actually increase the entropy by spreading out values.
- **RLE expansion on no runs:** As we saw in Chapter 7, RLE can expand data with no runs. If BWT fails to create runs, the RLE stage may hurt compression.
- **Algorithmic overhead:** The multi-stage pipeline (BWT + MTF + RLE + entropy coding) adds computational cost that may not be justified for data that compresses well with simpler methods.

For these reasons, BWT-based compressors like `bzip2` excel on structured data like text but may be outperformed by simpler LZ77 variants on binary data or already-compressed files.

8.12.4 Relation to FM-Index and Text Indexing

The BWT's impact extends far beyond compression. In 2000, Paolo Ferragina and Giovanni Manzini introduced the FM-index (Full-text index in Minute space), which uses the BWT to create a compressed data structure that supports:

- **Counting queries:** Find the number of occurrences of a pattern P in $O(|P|)$ time, regardless of the text size.
- **Locating queries:** Find the positions of matches in $O(|P| + \text{occ} \cdot \log^{\epsilon} n)$ time, where occ is the number of occurrences and the log factor depends on sampling rates.
- **Extract queries:** Retrieve arbitrary substrings of the original text without full decompression.

The FM-index revolutionized bioinformatics:

- **Bowtie and BWA:** DNA sequence aligners used in countless genomics projects, aligning millions of short reads against reference genomes.
- **Pattern matching in compressed data:** Search without full decompression, enabling efficient querying of large genomic databases.
- **Compressed suffix arrays:** Building block for many string algorithms that operate directly on compressed representations.

Important

The BWT demonstrates a profound principle: compression and indexing are two sides of the same coin. A good compressed representation often enables fast search directly on the compressed data, with time complexity that depends only on pattern size, not on text size.

8.13 Summary and Key Insights

8.13.1 The Transform-Then-Compress Principle

The BWT embodies a powerful design philosophy:

1. **Separate concerns:** First transform the data to reveal its structure, then compress the transformed representation
2. **Reversibility is key:** The transform must be exactly invertible
3. **Structure exploitation:** Different transforms reveal different kinds of structure

8.13.2 The BWT-RLE Connection

The relationship between BWT and the RLE variants from Chapter 7 is fundamental:

- **BWT creates the conditions** for effective RLE by grouping similar contexts
- **MTF converts** these groupings into actual runs
- **RLE variants** compress these runs efficiently

Without understanding RLE, you cannot fully appreciate why BWT works so well. The two techniques are complementary: BWT reorganizes, RLE exploits.

8.13.3 When to Use BWT-Based Compression

Scenario	Recommendation	Rationale
Text files, source code	BWT (bzip2)	Global repetition matters
DNA sequences	BWT-based + FM-index	Search+compress combined
Web content (HTTP)	DEFLATE/gzip	Streaming, compatibility
Archival storage	LZMA, Zstandard	Maximum compression
Real-time systems	LZ77 variants	Low latency, streaming
Embedded systems	Custom/simple RLE	Memory constraints

8.14 Hands-On Exercises

Exercise 8.0

Apply the Burrows-Wheeler Transform to the string "mississippi$" by hand. Show:

1. All cyclic rotations
2. The sorted rotations
3. The BWT output L and primary index I
4. Identify any runs in the output
5. Apply MTF and then the different RLE variants from Chapter 7
6. Compare which RLE variant works best

Exercise 8.1

Implement a simple BWT encoder in Python:

```python
def bwt_encode(s):
    """Return BWT(s) and primary index."""
    # Add end marker
```

```
    s = s + '$'
    # Generate rotations (hint: use list comprehension)
    # Sort rotations
    # Extract last column and find original row
    # Return (L, I)
    pass
```

Test your implementation on "banana" and "mississippi". Then implement the full pipeline with MTF and at least two RLE variants.

Exercise 8.2

Compare compression performance on a text file:

1. Find a text file > 1MB (e.g., a book from Project Gutenberg)
2. Compress with `gzip -9` and `bzip2 -9`
3. Compare sizes and compression times
4. Use `time` command to measure speed
5. Explain the results in terms of the algorithms' characteristics

Exercise 8.3

Challenge: Why does BWT perform poorly on random data? Connect your answer to:

1. The sorting process in BWT
2. The subsequent MTF and RLE stages
3. The concept of structure versus randomness
4. The run formation properties we studied in Chapter 7

Exercise 8.4

Research question: The FM-index enables substring search on BWT-compressed data. Explain:

1. How the LF-mapping enables backward search
2. Why counting queries are $O(|P|)$ time
3. Applications in bioinformatics (BWA, Bowtie)

Exercise 8.5

Compare the three compression paradigms we've studied:

Aspect	**Statistical**	**Dictionary**	**Transform**
Core idea	Model probabilities	Find repetitions	Reorganize data
Example	Huffman, Arithmetic	LZ77, LZ78, DEFLATE	BWT, bzip2
RLE role	Optional	Optional	Essential
Strengths	Optimal for known model	Fast, adaptive	Global structure
Weaknesses	Needs good model	Limited window	Block-based, slower
Best for	Well-characterized sources	General purpose	Text, structured data

Fill in this table based on Chapters 1-8.

Important

[Key Takeaway] The Burrows-Wheeler Transform teaches us that sometimes the best way to solve a problem is to transform it into a different problem. By reordering data to reveal its inherent structure, we make the subsequent compression task dramatically easier—and RLE becomes the perfect tool to exploit that revealed structure. This synergy between transformation and exploitation is a lesson that applies far beyond data compression.

9: Lecture 9: Prediction by Partial Matching (PPM): The Statistical Pinnacle

Definition

Prediction by Partial Matching (PPM) is an adaptive statistical data compression technique that uses a set of variable-length contexts to predict the next symbol based on previously seen data. It dynamically builds models for contexts of different lengths, favouring longer contexts when possible while falling back to shorter ones to handle unseen patterns. In this chapter, we focus on the PPMC variant (Method C), which is the most commonly used practical form of PPM.

PPM was introduced by Cleary and Witten in 1984 and represents a breakthrough in context-based modelling for lossless compression. By intelligently combining multiple orders of Markov prediction, PPM achieves compression ratios close to the theoretical entropy limits for many real-world data sources, such as text and binary files.

9.1 Introduction: Beyond Finite-Context Models

9.1.1 Recap of Markov Sources

From Section 4.4, we recall that a Markov source of order k models the probability of the next symbol based solely on the previous k symbols, known as the context c with $|c| = k$. The conditional probability of symbol s given c is estimated as:

$$P(s \mid c) = \frac{\text{Count}(c, s)}{\sum_{s' \in \mathcal{A}} \text{Count}(c, s')},$$

where $\mathcal{A}$ is the alphabet and $\text{Count}(c, s)$ is the number of times symbol s has followed context c in the training data (or adaptively during compression).

Fixed-order Markov models, while simple, suffer from significant limitations:

- **Order-0 models:** Treat each symbol independently, ignoring all context. This yields near-uniform probabilities, leading to poor compression (e.g. 8 bits per symbol for ASCII text).
- **Low-order models** ($k = 1$ or 2): Capture short-range dependencies (e.g. letter bigrams) but fail to exploit long-distance correlations, such as repeated phrases in natural language.
- **High-order models** ($k > 3$): Can model complex patterns but encounter the *context sparsity problem*. With large alphabets (e.g. $|\mathcal{A}| = 256$ for byte data), most

possible contexts never occur in finite data, resulting in zero-frequency estimates and unreliable predictions.

These issues motivate a more flexible approach that adapts the context length dynamically.

9.1.2 The Fundamental Insight

The genius of PPM lies in its simple yet powerful principle: **attempt to use the longest matching context available, but escape to shorter contexts if the current one provides no information about the next symbol**.

Important

PPM operates by maintaining a hierarchy of Markov models, from order-0 (no context) up to a maximum order M (typically $M = 3$ to 7, depending on memory constraints). To predict or encode a symbol, PPM begins with the highest-order context (the last M symbols). If that context has no record of the symbol (zero frequency), it encodes an *escape* event and descends to the next lower order ($M-1$), repeating until a matching context is found or the **order -1** level is reached. This mechanism ensures that non-zero probabilities can be assigned to all symbols.

This multi-order strategy leverages:

- **High-order contexts** for precise predictions in repetitive or structured data (e.g. `"the "` is often followed by common words).
- **Low-order fallbacks** for robustness against novel or sparse contexts.
- **Escape mechanism** to avoid zero-probability issues by assigning probability mass to unseen symbols via the escape event.

The result is a predictor that approximates the true conditional entropy of the source, often significantly outperforming fixed-order models on natural language data.

9.2 The PPM Algorithm

Definition

[Notation] Throughout this chapter we use:

- M — the maximum context order (e.g., $M = 2$ means we maintain models for contexts of length 0, 1, and 2)
- For a specific context string c (e.g., `"ab"`):
  - D_c — number of **distinct** symbols that have followed context c

- N_c — total number of times context c has occurred (sum of all counts)
- $\text{Count}(c, s)$ — how many times symbol s followed context c

- For the order-0 context (empty string), we write $D_\emptyset$ and $N_\emptyset$
- $\mathcal{A}$ — the alphabet (e.g., 256 possible byte values)

When the context is clear from the discussion, we use the notation directly.

9.2.1 Core Components

PPM is typically paired with arithmetic coding (or range coding) for entropy encoding, using the predicted probabilities to assign variable-length codes. The model is *adaptive*: probabilities update after each symbol is encoded or decoded.

For each order $k = 0, 1, \ldots, M$, PPM maintains a context model storing:

- A frequency table for symbols that have followed the context (only observed symbols are stored to save memory).
- Cumulative frequencies for efficient range selection in arithmetic coding.
- An *escape weight* to estimate the probability of an unseen (novel) symbol.

Contexts are represented as strings consisting of the last k symbols from the history buffer.

Definition

The **escape symbol** is a virtual event signalling that the next symbol is unseen in the current context. Encoding or decoding the escape triggers a shift to the lower-order model, where the process repeats.

Definition

[Order -1] The **order -1** model is a conceptual, final fallback mechanism that is invoked when the escape chain has exhausted all valid statistical models from order-M down to order-0. At this level, the next symbol is guaranteed to be unseen in all higher-order contexts under the exclusion mechanism (i.e., it has not appeared in any considered context during this prediction step), though it may have occurred earlier in the input. The probability assigned to any single, currently unseen symbol x is:

$$P(x \mid \text{order -1}) = \frac{1}{|\mathcal{A}| - |seen|},$$

where *seen* is the set of all symbols that have been excluded by higher orders during this encoding step. This ensures that the total probability mass for all currently

unseen symbols sums to 1.

Important

Integration with Arithmetic Coding: The probabilities computed by PPM are fed directly into an arithmetic coder. For each coding step, the interval $[0, 1)$ is partitioned according to the cumulative probabilities of all possible symbols (including escape) at the current context. The exclusion principle must be applied consistently in both the probability calculation and the interval partitioning.

9.2.2 The Escape and Prediction Mechanism

The core challenge is estimating probabilities for unseen symbols without biasing seen ones too heavily. PPM addresses the **zero-frequency problem** through escape, but the escape probability itself must be carefully tuned.

Decoding is symmetric to encoding: the decoder uses the same context hierarchy and probabilities to select the symbol, updating models identically.

9.2.3 Escape Estimation: The PPMC Method

The core challenge in PPM is estimating the probability of an escape — how often should we expect to see a novel symbol in a given context? Among the many proposed methods, **PPMC** (Method C) emerged as the standard because it strikes an excellent balance between simplicity and effectiveness.

Definition

[PPMC Escape Estimation] For any context c that has occurred in the input:

- Let D_c = number of **distinct** symbols that have followed c
- Let N_c = total number of times context c has appeared (sum of all counts for symbols following c)

PPMC assigns probabilities as:

$$P(\text{escape} \mid c) = \frac{D_c}{N_c + D_c}, \qquad P(s \mid c) = \frac{\text{Count}(c, s)}{N_c + D_c} \text{ for each seen symbol } s$$

For an empty context (never seen before), $D_c = 0$, so $P(\text{escape}) = 0$ — but this case is handled by falling back to lower orders.

Intuition: The escape probability is proportional to how many different symbols we've already seen in this context. If a context has shown rich diversity (high D_c), we're more likely to encounter something new. If it's been very consistent (low D_c relative to N_c), we trust it more.

Algorithm 16 PPM Prediction/Encoding Process (Simplified, for a Single Symbol)

Require: Next symbol x to encode, history buffer H, maximum order M, alphabet $\mathcal{A}$

```
k ← min(M, |H|)                                   ▷ Start with longest available context
seen ← ∅                                          ▷ Track excluded symbols (exclusion principle)
while k ≥ 0 do
    c ← last k symbols of H
    Retrieve model for c: frequencies for S_c, total N_c, distinct D_c
    S_excl ← S_c ∩ seen
    N_eff ← N_c − Σ_{s∈S_excl} Count(c, s)
    D_eff ← D_c − |S_excl|
    if x ∈ S_c \ S_excl then
        P(x | c, excl) ← Count(c, x) / (N_eff + D_eff)   ▷ PPMC with exclusion
        Encode x using this probability
        Count(c, x) ← Count(c, x) + 1
        return
    else
        P(escape | c, excl) ← D_eff / (N_eff + D_eff)
        Encode escape using this probability
        seen ← seen ∪ (S_c \ S_excl)
        k ← k − 1
    end if
end while
if k < 0 then                                     ▷ Order -1
    U ← |A| − |seen|                              ▷ Number of truly unseen symbols at this step
    P(x | unseen) ← 1/U
    Encode x using this uniform probability
    Update order-0 model with x
end if
Append x to H; truncate H to the last M symbols
```

Example

[PPMC in Action: Watching a Context Learn]
Consider a context that appears as we process text. Let's track a single context, say the order-1 context `"h"` (meaning the previous character was 'h').
Stage 1: First encounter The context `"h"` appears for the first time. It's empty initially:

$$D_{\mathtt{h}} = 0,\ N_{\mathtt{h}} = 0 \quad \Rightarrow \quad P(\text{escape}) = 0/(0+0)\ (\text{undefined})$$

But we don't use this context yet — we escape immediately to lower orders.
Stage 2: After seeing `"he"` The first time 'h' is followed by 'e', we create the context:

$$\text{Count}(\texttt{"h"}, \mathtt{e}) = 1, \quad D_{\mathtt{h}} = 1,\ N_{\mathtt{h}} = 1$$

Now if we see `"h"` again and need to predict:

$$P(\mathtt{e}) = \frac{1}{1+1} = \frac{1}{2} = 0.5, \quad P(\text{escape}) = \frac{1}{1+1} = 0.5$$

The context is equally likely to produce 'e' again or something new.
Stage 3: After seeing `"he"` and `"ha"` Later we encounter `"ha"`. Now the context has:

$$\mathtt{e} : 1,\ \mathtt{a} : 1, \quad D_{\mathtt{h}} = 2,\ N_{\mathtt{h}} = 2$$

Predictions become:

$$P(\mathtt{e}) = \frac{1}{2+2} = 0.25, \quad P(\mathtt{a}) = 0.25, \quad P(\text{escape}) = \frac{2}{4} = 0.5$$

Still a 50% chance of something new, but now two known symbols share the remaining 50% equally.
Stage 4: After seeing `"he"` twice and `"ha"` once The pattern stabilizes with more data:

$$\mathtt{e} : 2,\ \mathtt{a} : 1, \quad D_{\mathtt{h}} = 2,\ N_{\mathtt{h}} = 3$$

Now:

$$P(\mathtt{e}) = \frac{2}{3+2} = 0.4, \quad P(\mathtt{a}) = \frac{1}{5} = 0.2, \quad P(\text{escape}) = \frac{2}{5} = 0.4$$

Notice: as 'e' becomes more frequent, its probability rises, while escape probability drops slightly because the context is becoming more predictable.

Important

Why PPMC works:

- It **adapts** to context richness — diverse contexts escape more often
- It avoids zero-probability issues by assigning probability mass to unseen symbols via the escape mechanism.
- It's **computationally simple** — just need to track counts and distinct symbols
- Probabilities always sum to 1 automatically:

$$\sum_{s \in S_c} \frac{\text{Count}(c, s)}{N_c + D_c} + \frac{D_c}{N_c + D_c} = \frac{N_c + D_c}{N_c + D_c} = 1$$

Example

[Quick Comparison: Two Contexts]
Compare two different contexts to build intuition:
Context X: Very predictable. Has seen `"the"` 100 times, always followed by space.

$$D_{\texttt{the}} = 1,\ N_{\texttt{the}} = 100 \quad \Rightarrow \quad P(\text{space}) = \frac{100}{101} \approx 0.99,\ P(\text{escape}) = \frac{1}{101} \approx 0.01$$

Context Y: Very diverse. Has seen 50 different symbols, total 100 occurrences.

$$D_{\texttt{Y}} = 50,\ N_{\texttt{Y}} = 100 \quad \Rightarrow \quad P(\text{any given seen symbol}) \approx \frac{2}{150} = 0.0133,\ P(\text{escape}) = \frac{50}{150} \approx 0.333$$

Context Y is much more likely to see something new — exactly what PPMC captures!

Important

Other escape estimation methods exist (Method A with fixed escape = 1, Method B/D with more sophisticated formulas), but PPMC remains the most widely used due to its simplicity and strong performance across diverse data types. Modern compressors like PPMZ build upon PPMC with additional refinements like secondary escape estimation.

9.2.4 The Exclusion Principle

Important

The **exclusion principle** is a key optimisation in PPM: when the encoder escapes from a higher-order context to a lower-order one, all symbols that were *present* in the higher-order context (and hence already considered) are *excluded* from the probability distribution in the lower-order context. This renormalises the remaining probabilities upward, sharpening predictions for genuinely novel symbols and typically saving 5–10% of bits compared with non-excluding PPM. Excluded symbols are removed from both the symbol set and the normalization constant to ensure probabilities remain properly normalized.

Example

Quantified exclusion. Suppose order-2 context `"ab"` has seen {`r`} only, so an escape is sent. The order-1 context `"b"` has seen {`r`, `a`} with counts $[1, 1]$ ($N_{\mathtt{b}} = 2$, $D_{\mathtt{b}} = 2$).

- *Without exclusion:* $P(\mathtt{r}) = 1/4$, $P(\mathtt{a}) = 1/4$, $P(\text{esc}) = 2/4 = 0.5$.
- *With exclusion* of `r` ($S_{\text{excl}} = \{\mathtt{r}\}$, remaining $D_{\text{eff}} = 1$, $N_{\text{eff}} = 1$): $P(\mathtt{a}) = 1/2$, $P(\text{esc}) = 1/2$.

Excluding `r` doubles $P(\mathtt{a})$, saving 1 bit if `a` is the next symbol.

9.3 Step-by-Step Examples

9.3.1 Example 1: Encoding `"banana"`

This example demonstrates PPM with maximum order $M = 2$ and PPMC escape estimation. The alphabet size is $|\mathcal{A}| = 256$ (bytes). We apply the exclusion principle throughout, noting that in early steps, exclusion has no effect because there are no overlapping symbols between contexts. All bit costs are approximated to two decimal places.

Example

[PPM in Action: Encoding "banana" with Exclusion]
Step 1 — First symbol: `'b'`

- History: `""`. The longest available context is order-2, then order-1, then order-0. All are empty. In an empty context, escape has probability 1, so it contributes 0 bits (i.e., it does not change the coding interval).
- After escaping through all empty contexts, we reach Order -1.

- At Order -1: $|seen| = 0$, so $P(\texttt{b}) = 1/256$.
- Cost: $-\log_2(1/256) = 8$ bits.

After encoding, update models:

Order	**Context**	**Symbol counts**	D_c	N_c
Order-0	$\emptyset$	b:1	1	1

History: "b"

Step 2 — Second symbol: 'a'

- Current longest: order-1 context "b" — empty ⇒ escape (0 bits)
- Next: order-0. $D_\emptyset = 1$, $N_\emptyset = 1$; $P(\text{esc}) = 1/(1+1) = 1/2$ (1 bit)
- 'a' not in order-0 ⇒ encode escape; add {b} to *seen*
- Order -1: $|seen| = 1$, so $P(\texttt{a}) = 1/(256-1) = 1/255 \approx 7.99$ bits
- Total for 'a': $0 + 1 + 7.99 = 8.99$ bits

After encoding, update models:

Order	**Context**	**Symbol counts**	D_c	N_c
Order-0	$\emptyset$	b:1, a:1	2	2
Order-1	"b"	a:1	1	1

History: "ba"

Step 3 — Third symbol: 'n'

- Current longest: order-2 "ba" — empty ⇒ escape (0 bits)
- Next: order-1 "a" — empty ⇒ escape (0 bits)
- Next: order-0. $D_\emptyset = 2$, $N_\emptyset = 2$; $P(\text{esc}) = 2/(2+2) = 1/2$ (1 bit)
- 'n' not in order-0 ⇒ encode escape; add {a, b} to *seen*
- Order -1: $|seen| = 2$, so $P(\texttt{n}) = 1/(256-2) = 1/254 \approx 7.99$ bits
- Total: $0 + 0 + 1 + 7.99 = 8.99$ bits

After encoding, update models:

Order	**Context**	**Symbol counts**	D_c	N_c
Order-0	$\emptyset$	b:1, a:1, n:1	3	3
Order-1	"b"	a:1	1	1
Order-1	"a"	n:1	1	1
Order-2	"ba"	n:1	1	1

History: "ban"

Step 4 — Fourth symbol: 'a'

- Current longest: order-2 "an" — empty ⇒ escape (0 bits)
- Next: order-1 "n" — empty ⇒ escape (0 bits)
- Next: order-0. $D_\emptyset = 3$, $N_\emptyset = 3$; 'a' IS in order-0 table!
- $P(\mathtt{a}) = \text{Count}(\emptyset, \mathtt{a})/(N_\emptyset + D_\emptyset) = 1/(3+3) = 1/6 \approx 2.58$ bits
- Total: 2.58 bits (no escapes needed)

After encoding, update models:

Order	**Context**	**Symbol counts**	D_c	N_c
Order-0	$\emptyset$	b:1, a:2, n:1	3	4
Order-1	"b"	a:1	1	1
Order-1	"a"	n:1	1	1
Order-1	"n"	a:1	1	1
Order-2	"ba"	n:1	1	1
Order-2	"an"	a:1	1	1

History: "bana"

Step 5 — Fifth symbol: 'n'

- Current longest: order-2 "na" — empty ⇒ escape (0 bits)
- Next: order-1 "a" — EXISTS! Has seen n:1
- $P(\mathtt{n})$ at order-1 $= \text{Count}(\mathtt{"a"}, \mathtt{n})/(N_\mathtt{a} + D_\mathtt{a}) = 1/(1+1) = 1/2 = 1$ bit
- Total: 1 bit!

After encoding, update models:

Order	**Context**	**Symbol counts**	D_c	N_c
Order-0	$\emptyset$	b:1, a:2, n:2	3	5
Order-1	"b"	a:1	1	1
Order-1	"a"	n:2	1	2
Order-1	"n"	a:1	1	1
Order-2	"ba"	n:1	1	1
Order-2	"an"	a:1	1	1
Order-2	"na"	n:1	1	1

History: "banan"

Step 6 — Sixth symbol: 'a'

- Current longest: order-2 "an" — EXISTS! Has seen a:1
- $P(\texttt{a})$ at order-2 = Count("an", a)/$(N_{\texttt{an}} + D_{\texttt{an}}) = 1/(1+1) = 1/2 = 1$ bit
- Total: 1 bit!

After encoding, update models:

Order	Context	Symbol counts	D_c	N_c
Order-0	$\emptyset$	b:1, a:3, n:2	3	6
Order-1	"b"	a:1	1	1
Order-1	"a"	n:2	1	2
Order-1	"n"	a:2	1	2
Order-2	"ba"	n:1	1	1
Order-2	"an"	a:2	1	2
Order-2	"na"	n:1	1	1

History: "banana"

Final Summary:

Symbol	Context used	Cost (bits)
b (1)	Order -1	8.00
a (2)	Order -1 (after escapes)	8.99
n (3)	Order -1 (after escapes)	8.99
a (4)	order-0	2.58
n (5)	order-1 ("a")	1.00
a (6)	order-2 ("an")	1.00
Total		**30.56 bits**

Comparison:

- Raw ASCII: $6 \times 8 = 48$ bits
- Our cost: $8 + 8.99 + 8.99 + 2.58 + 1 + 1 = 30.56$ bits
- Savings from PPM: $\approx 36\%$

9.3.2 Example 2: Encoding "abracadabra" (PPMC, $M = 2$)

This classic example shows PPM on a more complex string. We'll use PPMC with maximum order $M = 2$ and alphabet size $|\mathcal{A}| = 256$. Models update after each symbol. We track all contexts that appear: order-0 (empty context), order-1 contexts (single symbols), and order-2 contexts (pairs).

Example

[PPM Encoding "abracadabra" Step by Step]

Symbol 1: `'a'`

- All contexts empty; escape to order -1.
- Cost: $-\log_2(1/256) = 8$ bits.

After encoding, update models:

Order	**Context**	**Symbol counts**	D_c	N_c
Order-0	$\emptyset$	a:1	1	1

History: $H =$ `"a"`

Symbol 2: `'b'`

- Longest available context: order-1 `(a)` — empty $\Rightarrow$ escape
- Order-0: $D_\emptyset = 1$, $N_\emptyset = 1$; $P(\text{esc}) = 1/(1+1) = 1/2$ (1 bit)
- `'b'` unseen in order-0 $\Rightarrow$ encode escape. Add `a` to *seen*.
- Order -1: $|seen| = 1$, so $P(\texttt{b}) = 1/255 \approx 7.99$ bits.
- Total cost: $1 + 7.99 = 8.99$ bits

After encoding, update models:

Order	**Context**	**Symbol counts**	D_c	N_c
Order-0	$\emptyset$	a:1, b:1	2	2
Order-1	`"a"`	b:1	1	1

History: $H =$ `"ab"`

Symbol 3: `'r'`

- Longest available: order-2 `(ab)` — empty $\Rightarrow$ escape
- Next: order-1 `(b)` — empty $\Rightarrow$ escape
- Order-0: $D_\emptyset = 2$, $N_\emptyset = 2$; $P(\text{esc}) = 2/(2+2) = 1/2$ (1 bit)
- `'r'` unseen in order-0 $\Rightarrow$ encode escape. Add {`a`, `b`} to *seen*.
- Order -1: $|seen| = 2$, so $P(\texttt{r}) = 1/254 \approx 7.99$ bits.
- Total cost: $1 + 7.99 = 8.99$ bits (escapes from order-2 and order-1 cost 0 bits as their models are empty)

After encoding, update models:

Order	**Context**	**Symbol counts**	D_c	N_c
Order-0	$\emptyset$	a:1, b:1, r:1	3	3
Order-1	"a"	b:1	1	1
Order-1	"b"	r:1	1	1
Order-2	"ab"	r:1	1	1

History: $H =$ "abr"

Symbol 4: 'a'

- Longest available: order-2 (br) — empty ⇒ escape
- Next: order-1 (r) — empty ⇒ escape
- Order-0: $D_\emptyset = 3$, $N_\emptyset = 3$; $P(\mathtt{a}) = \text{Count}(\emptyset, \mathtt{a})/(N_\emptyset + D_\emptyset) = 1/(3+3) = 1/6 \approx 2.58$ bits
- Symbol found at order-0!
- Total cost: 2.58 bits

After encoding, update models:

Order	**Context**	**Symbol counts**	D_c	N_c
Order-0	$\emptyset$	a:2, b:1, r:1	3	4
Order-1	"a"	b:1	1	1
Order-1	"b"	r:1	1	1
Order-1	"r"	a:1	1	1
Order-2	"ab"	r:1	1	1
Order-2	"br"	a:1	1	1

History: $H =$ "abra"

Symbol 5: 'c'

- Longest available: order-2 (ra) — empty ⇒ escape
- Next: order-1 (a) — has b:1 ($D_\mathtt{a} = 1$, $N_\mathtt{a} = 1$); $P(\text{esc}) = 1/2$ (1 bit). Add b to *seen*.
- Next: order-0. Effective model after exclusion: remaining symbols are {a, r}. $D_{\text{eff}} = 2$, $N_{\text{eff}} = 3$ (since count of b excluded). Denominator $= N_{\text{eff}} + D_{\text{eff}} = 3 + 2 = 5$.
- $P(\text{esc})$ at order-0 $= 2/5 \approx 1.32$ bits.

- `'c'` unseen ⇒ encode escape. Add {`a`, `r`} to *seen*.
- Order -1: $|seen| = |\{\texttt{a}, \texttt{b}, \texttt{r}\}| = 3$, so $P(\texttt{c}) = 1/(256-3) = 1/253 \approx 7.98$ bits.
- Total cost: $1 + 1.32 + 7.98 = 10.30$ bits

After encoding, update models:

Order	**Context**	**Symbol counts**	D_c	N_c
Order-0	$\emptyset$	a:2, b:1, r:1, c:1	4	5
Order-1	"a"	b:1, c:1	2	2
Order-1	"b"	r:1	1	1
Order-1	"r"	a:1	1	1
Order-1	"c"	(empty)	0	0
Order-2	"ab"	r:1	1	1
Order-2	"br"	a:1	1	1
Order-2	"ra"	c:1	1	1

History: $H =$ "abrac"

Symbol 6: `'a'`

- Longest available: order-2 (ac) — empty ⇒ escape
- Next: order-1 (c) — empty ⇒ escape
- Next: order-0. $D_\emptyset = 4$, $N_\emptyset = 5$; $P(\texttt{a}) = 2/(5+4) = 2/9 \approx 2.17$ bits
- Total cost: 2.17 bits

After encoding, update models:

Order	**Context**	**Symbol counts**	D_c	N_c
Order-0	$\emptyset$	a:3, b:1, r:1, c:1	4	6
Order-1	"a"	b:1, c:1	2	2
Order-1	"b"	r:1	1	1
Order-1	"r"	a:1	1	1
Order-1	"c"	a:1	1	1
Order-2	"ab"	r:1	1	1
Order-2	"br"	a:1	1	1
Order-2	"ra"	c:1	1	1
Order-2	"ac"	a:1	1	1

History: $H =$ "abraca"

Symbol 7: `'d'`

- Longest available: order-2 (ca) — empty ⇒ escape

- Next: order-1 `(a)` — has `b`:1, `c`:1 ($D_{\mathtt{a}} = 2$, $N_{\mathtt{a}} = 2$); $P(\text{esc}) = 2/4 = 1/2$ (1 bit). Add {`b`, `c`} to *seen*.
- Next: order-0. Effective model after exclusion: remaining symbols are {`a`, `r`}. $D_{\text{eff}} = 2$, $N_{\text{eff}} = 4$ (counts of `a`:3, `r`:1). Denominator $= 4 + 2 = 6$.
- $P(\text{esc})$ at order-0 $= 2/6 = 1/3 \approx 1.58$ bits.
- `'d'` unseen $\Rightarrow$ encode escape. Add {`a`, `r`} to *seen*.
- Order -1: $|seen| = |\{\mathtt{a}, \mathtt{b}, \mathtt{c}, \mathtt{r}\}| = 4$, so $P(\mathtt{d}) = 1/(256 - 4) = 1/252 \approx 7.97$ bits.
- Total cost: $1 + 1.58 + 7.97 = 10.55$ bits

After encoding, update models:

Order	**Context**	**Symbol counts**	D_c	N_c
Order-0	$\emptyset$	`a`:3, `b`:1, `r`:1, `c`:1, `d`:1	5	7
Order-1	`"a"`	`b`:1, `c`:1, `d`:1	3	3
Order-1	`"b"`	`r`:1	1	1
Order-1	`"r"`	`a`:1	1	1
Order-1	`"c"`	`a`:1	1	1
Order-1	`"d"`	(empty)	0	0
Order-2	`"ab"`	`r`:1	1	1
Order-2	`"br"`	`a`:1	1	1
Order-2	`"ra"`	`c`:1	1	1
Order-2	`"ac"`	`a`:1	1	1
Order-2	`"ca"`	`d`:1	1	1

History: $H =$ `"abracad"`

Symbol 8: `'a'`

- Longest available: order-2 `(ad)` — empty $\Rightarrow$ escape
- Next: order-1 `(d)` — empty $\Rightarrow$ escape
- Next: order-0. $D_\emptyset = 5$, $N_\emptyset = 7$; $P(\mathtt{a}) = 3/(7 + 5) = 3/12 = 0.25$ (2 bits)
- Total cost: 2 bits

After encoding, update models:

Order	Context	Symbol counts	D_c	N_c
Order-0	$\emptyset$	a:4, b:1, r:1, c:1, d:1	5	8
Order-1	"a"	b:1, c:1, d:1	3	3
Order-1	"b"	r:1	1	1
Order-1	"r"	a:1	1	1
Order-1	"c"	a:1	1	1
Order-1	"d"	a:1	1	1
Order-2	"ab"	r:1	1	1
Order-2	"br"	a:1	1	1
Order-2	"ra"	c:1	1	1
Order-2	"ac"	a:1	1	1
Order-2	"ca"	d:1	1	1
Order-2	"ad"	a:1	1	1

History: $H =$ "abracada"

Symbol 9: 'b'

- Longest available: order-2 (da) — empty $\Rightarrow$ escape
- Next: order-1 (a) — has b:1, c:1, d:1 ($D_{\mathtt{a}} = 3$, $N_{\mathtt{a}} = 3$); $P(\mathtt{b}) = 1/(3+3) = 1/6 \approx 2.58$ bits
- Symbol found at order-1!
- Total cost: 2.58 bits

After encoding, update models:

Order	Context	Symbol counts	D_c	N_c
Order-0	$\emptyset$	a:4, b:2, r:1, c:1, d:1	5	9
Order-1	"a"	b:2, c:1, d:1	3	4
Order-1	"b"	r:1	1	1
Order-1	"r"	a:1	1	1
Order-1	"c"	a:1	1	1
Order-1	"d"	a:1	1	1
Order-2	"ab"	r:1	1	1
Order-2	"br"	a:1	1	1
Order-2	"ra"	c:1	1	1
Order-2	"ac"	a:1	1	1
Order-2	"ca"	d:1	1	1
Order-2	"ad"	a:1	1	1
Order-2	"da"	b:1	1	1

History: $H =$ "abracadab"

Symbol 10: 'r'

- Longest available: order-2 (ab) — has r:1 ($D_{\texttt{ab}} = 1$, $N_{\texttt{ab}} = 1$); $P(\texttt{r}) = 1/(1+1) = 1/2 = 1$ bit
- Symbol found at order-2!
- Total cost: 1 bit

After encoding, update models:

Order	Context	Symbol counts	D_c	N_c
Order-0	$\emptyset$	a:4, b:2, r:2, c:1, d:1	5	10
Order-1	"a"	b:2, c:1, d:1	3	4
Order-1	"b"	r:2	1	2
Order-1	"r"	a:1	1	1
Order-1	"c"	a:1	1	1
Order-1	"d"	a:1	1	1
Order-2	"ab"	r:2	1	2
Order-2	"br"	a:1	1	1
Order-2	"ra"	c:1	1	1
Order-2	"ac"	a:1	1	1
Order-2	"ca"	d:1	1	1
Order-2	"ad"	a:1	1	1
Order-2	"da"	b:1	1	1

History: $H =$ "abracadabr"

Symbol 11: 'a'

- Longest available: order-2 (br) — has a:1 ($D_{\texttt{br}} = 1$, $N_{\texttt{br}} = 1$); $P(\texttt{a}) = 1/(1+1) = 1/2 = 1$ bit
- Symbol found at order-2!
- Total cost: 1 bit

After encoding, update models:

Order	Context	Symbol counts	D_c	N_c
Order-0	$\emptyset$	a:5, b:2, r:2, c:1, d:1	5	11
Order-1	"a"	b:2, c:1, d:1	3	4
Order-1	"b"	r:2	1	2
Order-1	"r"	a:2	1	2
Order-1	"c"	a:1	1	1
Order-1	"d"	a:1	1	1
Order-2	"ab"	r:2	1	2
Order-2	"br"	a:2	1	2
Order-2	"ra"	c:1	1	1
Order-2	"ac"	a:1	1	1
Order-2	"ca"	d:1	1	1
Order-2	"ad"	a:1	1	1
Order-2	"da"	b:1	1	1

History: H = "abracadabra"

Final Summary:

Symbol	Context used	Cost (bits)
a (1)	order -1	8.00
b (2)	order -1	8.99
r (3)	order -1	8.99
a (4)	order-0	2.58
c (5)	order -1	10.30
a (6)	order-0	2.17
d (7)	order -1	10.55
a (8)	order-0	2.00
b (9)	order-1 ("a")	2.58
r (10)	order-2 ("ab")	1.00
a (11)	order-2 ("br")	1.00
Total		**58.16 bits**

Raw ASCII would require $11 \times 8 = 88$ bits. PPM saves about 34%.
Notice how the final two symbols cost only 1 bit each — once the pattern establishes that "ab" is followed by 'r' and "br" is followed by 'a', PPM predicts with near-perfect accuracy!

9.3.3 Example 3: Repetitive String "abababababab" ($M = 1$, PPMC)

This example shows how order-1 PPM rapidly learns a strict alternation pattern. Alphabet size 256; $M = 1$ (order-0 and order-1 only).

Example

[PPM Learning Alternating Pattern "abababababab"]
We track the evolution of models after each symbol:

Symbol	Context used	Cost	$D_\emptyset, N_\emptyset$	$D_{\mathtt{a}}, N_{\mathtt{a}}$	$D_{\mathtt{b}}, N_{\mathtt{b}}$
1: a	order -1	8.00	(1,1)	(0,0)	(0,0)
2: b	order -1	8.99	(2,2)	(1,1)	(0,0)
3: a	order-0	2.58	(2,3)	(1,1)	(1,1)
4: b	order-1 ("a")	1.00	(2,4)	(1,2)	(1,1)
5: a	order-1 ("b")	1.00	(2,5)	(1,2)	(1,2)
6: b	order-1 ("a")	0.58	(2,6)	(1,3)	(1,2)
7: a	order-1 ("b")	0.58	(2,7)	(1,3)	(1,3)
8: b	order-1 ("a")	0.42	(2,8)	(1,4)	(1,3)
9: a	order-1 ("b")	0.42	(2,9)	(1,4)	(1,4)
10: b	order-1 ("a")	0.32	(2,10)	(1,5)	(1,4)

Observations:

- By symbol 4, order-1 context `"a"` perfectly predicts `'b'` (1 bit)
- By symbol 5, order-1 context `"b"` perfectly predicts `'a'` (1 bit)
- As counts increase, probabilities become more certain and costs decrease
- Total: $8 + 8.99 + 2.58 + 1 + 1 + 0.58 + 0.58 + 0.42 + 0.42 + 0.32 \approx 23.89$ bits (2.39 bpc)

An adaptive order-0 model would require many more symbols to reach similar efficiency.

9.3.4 Example 4: Exclusion Applied to `"abracadabra"` (Symbol 5)

We revisit symbol 5 (`'c'`) from Example 2 but now apply exclusion.

After encoding symbols 1–4 (`"abra"`), the relevant models are:

- Order-2 `(ra)`: empty.
- Order-1 `(a)`: `b`:1 ($D_{\mathtt{a}} = 1$, $N_{\mathtt{a}} = 1$).
- Order-0: `a`:2, `b`:1, `r`:1 ($D_{\emptyset} = 3$, $N_{\emptyset} = 4$).

Step 1 — Order-2 escape. `(ra)` is empty, $P(\text{esc}) = 1$. Symbols excluded from lower orders: $\emptyset$.

Step 2 — Order-1 escape with exclusion. `(a)` contains `b`:1 only. `c` is not present $\Rightarrow$ escape. $P(\text{esc}) = D_{\mathtt{a}}/(N_{\mathtt{a}} + D_{\mathtt{a}}) = 1/2$ (1 bit). After this step, *seen* $= \{\mathtt{b}\}$.

Step 3 — Order-0 with exclusion of `b`. Effective table: `a`:2, `r`:1 (exclude `b`). $D_{\text{eff}} = 2$, $N_{\text{eff}} = 3$; denom $= 3 + 2 = 5$. $P(\text{esc}) = 2/5$ (≈ 1.32 bits). `c` unseen $\Rightarrow$ escape.

Without exclusion (Example 2): $P(\text{esc at O-0}) = 3/7 \approx 1.22$ bits.
With exclusion: $P(\text{esc at O-0}) = 2/5 = 1.32$ bits.

Here exclusion slightly increases the O-0 escape cost, but crucially, if the target symbol had been `a`, exclusion would have increased $P(\mathtt{a})$ from $2/7 \approx 1.81$ bits to $2/5 = 1.32$ bits — a saving of 0.49 bits. Exclusion benefits the *average* case by concentrating probability on likely symbols.

9.4 PPM Implementation Challenges

9.4.1 Memory Management

High-order PPM can, in the worst case, require up to $|\mathcal{A}|^M$ contexts.

Example

For $M = 5$ and $|\mathcal{A}| = 256$ (byte data), the theoretical maximum is $256^5 \approx 1.1 \times 10^{12}$ contexts — clearly infeasible. However, on typical English text ($|\mathcal{A}| \approx 70$ printable characters), the actual number of observed contexts stays below 10^6 with 1 MB of RAM. PPM exploits this sparsity by allocating memory only for contexts actually encountered.

Memory is managed through:

- **Lazy allocation**: Context nodes are created on first encounter.
- **Count rescaling**: Periodically halving all counts prevents overflow and lets the model adapt to non-stationary data.
- **Memory budgets**: Some implementations cap total nodes and evict the least-recently-used contexts when full.

9.4.2 Data Structures: The Context Trie

The standard solution for efficient context lookup is a **trie** (prefix tree).

Definition

In a **context trie**, the root node represents the empty context (order-0). Each edge is labelled with a symbol, and a path of length k from the root represents a k-th order context. Each node stores:

- For the context c represented by this node:
  - D_c — number of distinct symbols seen after this context
  - N_c — total occurrences of this context
  - Frequency table for symbols following this context
- Child pointers (typically a hash map for large alphabets).
- A *suffix link* pointing to the node for the context obtained by dropping the oldest symbol, enabling O(1) fallback.

Suffix links allow PPM to traverse the escape hierarchy in $O(M)$ time without rebuilding the context string at each level. Combined with hashed children, the trie supports $O(1)$ average-case lookup and update.

9.4.3 The Zero-Frequency Problem and Modern Refinements

Several advanced techniques address the remaining inadequacies of simple escape estimation:

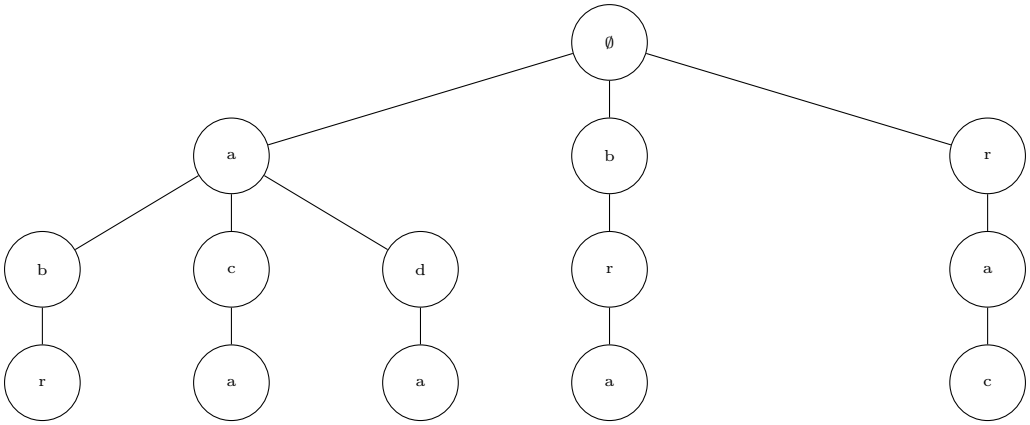


Figure 3: Partial context trie for `"abracadabra"` with $M = 3$. Each path from the root to a node at depth k represents a k-th order context; suffix links (not shown) connect each depth-k node to the depth-$(k-1)$ node for the same context suffix. This structure enables $O(1)$ escape transitions.

- **Adaptive escape**: Adjust escape weight online using running averages of how often escape was actually needed in each context class.
- **Secondary Escape Estimation (SEE)**: Use a second-level model (often itself a PPM-like structure over context metadata) to predict $P(\text{escape})$; used in PPMZ and PAQ.
- **Universal coding connections**: View escape as coding the event "novel symbol"; Bayesian methods (e.g. Krichevsky–Trofimov estimator) give principled priors.
- **Mixing**: Combine escape estimates from different orders via log-linear interpolation (foreshadowing context mixing, Section 5.3).

9.5 PPM Variants and Successors

9.5.1 PPM*: Unbounded Contexts

Definition

PPM* (Witten, Neal, and Cleary, 1987) removes the fixed maximum order M, instead finding the longest suffix of the current history that has been seen at least once in the past. This is analogous to the phrase-matching in Ziv–Lempel (LZ) schemes but operates probabilistically.

PPM* is implemented efficiently using augmented tries with *failure functions* (similar to Aho–Corasick), achieving $O(n \log n)$ compression time.

Example

On the string `"banana"`, suppose the full context `"anana"` was seen during encoding. PPM* would use this five-symbol context, whereas PPM with $M = 3$ would be limited to `"ana"` — potentially missing stronger long-range predictions.

Trade-off: PPM* excels on long-period repetitive data (DNA, source code) but may over-fit on noisy or short data, where shorter fixed contexts provide more reliable statistics (see Exercise 4.3).

9.5.2 PPMZ: Optimised Practical Variant

Building on PPMC, Charles Bloom's PPMZ introduced several production-quality innovations:

- **Dynamic order selection**: The effective maximum order adapts per-block based on measured prediction improvement.
- **Secondary Escape Estimation (SEE)**: A compact neural-like model replaces fixed heuristics for $P(\text{escape})$, updating on-the-fly.
- **Weighted novelty**: Unseen symbols at the fallback level are assigned probabilities biased toward frequently appearing symbols in the global distribution.

PPMZ held world records on the Calgary Corpus throughout the late 1990s, outperforming Gzip and BZIP2 by 20–30%.

9.5.3 PAQ: Context Mixing

Important

Context mixing represents the natural evolution of PPM's multi-order philosophy. Rather than *escaping* through a hierarchy, PAQ (Matt Mahoney, 2002) *combines* predictions from all orders simultaneously using adaptive weighting.

Key innovations in PAQ and its descendants:

- **Hundreds of models**: Order-0 through order-12 PPM models, plus word-level models, sparse contexts, and transform-based models (e.g. BWT residuals).
- **Log-linear mixing**: Each model outputs a log-odds prediction $\ell_i = \log \frac{P_i(1)}{P_i(0)}$; the combined prediction is

$$P_{\text{mix}}(1) = \sigma\left(\sum_i w_i \ell_i\right), \quad \sigma(x) = \frac{1}{1 + e^{-x}}.$$

- **Online weight update**: Weights w_i increase for models that predicted the actual symbol and decrease otherwise (gradient descent on log-loss).

PAQ8 and its successor CMIX consistently win the Hutter Prize (enwik9, 100 MB Wikipedia), achieving $\approx$ 1.1–1.3 bpc on English, within 10–30% of the estimated Shannon limit of $\approx$ 1.0 bpc.

Example

In PAQ encoding a space after `"the"`: the order-3 model assigns $P(\text{space}) = 0.40$, a word-boundary model assigns 0.65, a positional model 0.35. Mixed log-linearly with learned weights, the combined $P(\text{space}) \approx 0.55$ — better than any single model alone.

9.5.4 Limitations of PPM

Despite its theoretical elegance, PPM has practical limitations:

- **Memory consumption**: Even with trie structures, PPM with $M = 5$ can consume hundreds of megabytes on large files, limiting its use in memory-constrained environments.
- **Speed**: Context lookups and updates are slower than dictionary-based methods like LZ77, making PPM impractical for real-time or high-throughput applications.
- **Alphabet size sensitivity**: PPM performs best with small alphabets (e.g., text with 70-100 symbols). For large alphabets (e.g., 16-bit Unicode), context sparsity becomes severe even with moderate M.
- **Non-stationarity**: While adaptive, PPM's simple frequency counts may not adapt quickly enough to changing source statistics; methods like count halving help but are ad-hoc.

These limitations explain why PPM is rarely used in production compressors today, despite its theoretical importance. Modern systems prefer LZ77-derived algorithms (e.g., Deflate, Zstandard) that offer better speed/memory trade-offs, or context mixing (PAQ/CMIX) when maximum compression is required.

9.6 Why PPM Matters Today

9.6.1 Theoretical Foundations

Theorem 9.1 (Asymptotic Optimality of PPM)**.** *For a stationary ergodic source with entropy rate H (bits per symbol), PPM with maximum order M_n growing appropriately*

with input length n satisfies:

$$\frac{1}{n}\sum_{i=1}^{n}\big(-\log_2 P(x_i \mid x_1^{i-1})\big) \xrightarrow{a.s.} H \quad \text{as } n \to \infty,$$

provided $M_n \to \infty$ and $M_n = O(\log n / \log |\mathcal{A}|)$. The per-symbol redundancy is bounded by $O\Big(\frac{M_n \log |\mathcal{A}|}{n}\Big)$.

Proof sketch. By ergodicity, empirical frequencies $\mathrm{Count}(c, s)/N_c$ converge almost surely to the true conditional probabilities $P(s \mid c)$ for all contexts c that occur with positive probability. As n grows, the probability that the current context of length M_n has been seen at least once approaches 1, so escapes occur with vanishing probability. The cumulative escape overhead contributes $O(M_n \log |\mathcal{A}|/n)$ redundancy, which tends to 0 under the stated growth condition. □

9.6.2 Applications Beyond Compression

PPM's predictive power extends well beyond file compression:

- **Text and keyboard prediction**: iOS and Android keyboards use PPM-like models for next-word suggestion, reducing keystrokes by up to 20% in user studies.
- **Bioinformatics**: Order-5 PPM captures codon-usage bias and splice-site patterns in DNA/protein sequences; the Glimmer gene-finder uses Interpolated Markov Models, a close relative of PPM.
- **Intrusion and anomaly detection**: Network packet sequences or system-call traces assigned low probability by a PPM model flag anomalous behaviour; STIDE and related systems use this principle.
- **Adaptive prefetching**: OS file-system and memory prefetchers use PPM-style context models to predict the next block or memory address.
- **Spam filtering**: Character-level PPM scores give low probability to randomly generated strings or obfuscated text typical of spam.

9.6.3 Influence on Modern Compression Systems

PPM's ideas are embedded in virtually every modern lossless compression system:

- **Brotli** (Google, 2015) and **Zstandard** (Facebook, 2016) use context-adaptive Asymmetric Numeral Systems (ANS) entropy coders; the context models for literal bytes and copy lengths are PPM-inspired.
- **CMIX** and **PAQ8** achieve near-theoretical limits on the Hutter Prize through massive context mixing, a direct extension of PPM's multi-order approach.

- **Neural compressors** such as LSTM-compress and the Transformer-based models in DeepMind's language-model compressor (Deletang et al., 2023) are essentially PPM with unlimited context encoded via neural weights, replacing the trie with a deep network.

Summary

In this chapter, we have explored Prediction by Partial Matching (PPM), a sophisticated adaptive statistical compression technique that represents the pinnacle of Markov-based modeling. Key takeaways include:

- PPM maintains multiple Markov models of orders 0 through M, using an *escape* mechanism to fall back to shorter contexts when predictions fail.
- The *zero-frequency problem* is addressed through escape estimation, with PPMC being the most widely used method: $P(\text{escape} \mid c) = D_c/(N_c + D_c)$.
- The *exclusion principle* sharpens probability estimates by removing symbols already considered in higher-order contexts, and requires the use of effective counts (N_{eff}, D_{eff}) for proper normalization.
- The **order -1** model provides a final fallback for symbols that remain unseen after exclusion across all higher orders, assigning them uniform probability over the currently-unseen alphabet.
- Efficient implementation requires trie data structures with suffix links, where each node stores D_c and N_c for its context.
- Modern variants like PPM* (unbounded contexts) and PPMZ (secondary escape estimation) push compression ratios even closer to theoretical limits.
- PPM's ideas live on in context-mixing compressors (PAQ, CMIX) and influence modern systems like Brotli and Zstandard.

While rarely used in its pure form today, PPM remains the conceptual foundation for statistical modeling in lossless compression.

Exercises

Exercise 9.0

Implement PPM-1 (PPMC, $M = 1$) in Python or C for the string `"abababababab"`. Track $D_\emptyset$, $N_\emptyset$, $D_{\texttt{a}}$, $N_{\texttt{a}}$, $D_{\texttt{b}}$, $N_{\texttt{b}}$ after each symbol. Compute the exact arithmetic code lengths and verify the totals from Example 3.

Exercise 9.1

Prove that PPMC probabilities always sum to 1. For a context c with D_c distinct symbols and total count $N_c = \sum_{s \in S_c} \text{Count}(c, s)$, show that:

$$\sum_{s \in S_c} \frac{\text{Count}(c, s)}{N_c + D_c} + \frac{D_c}{N_c + D_c} = 1.$$

Also prove that $0 < P(\text{escape}) \leq 1$ for all valid context states, and identify the unique condition under which $P(\text{escape}) = 1$ and the condition under which $P(\text{escape})$ approaches 0.

Exercise 9.2

Why might PPM* underperform a fixed-order PPM (e.g. $M = 3$) on uniformly random binary data? Reason about how long contexts overfit noise and yield unreliable statistics. Simulate compression of 1 MB of pseudo-random data and 1 MB of repetitive structured data, and compare bit-per-character ratios for PPM* and PPM-3.

Exercise 9.3

Extend the `"abracadabra"` trace (Example 2) by applying the exclusion principle to *every* symbol, not just symbol 5. For each symbol, list: the contexts visited, the symbols excluded at each level, the adjusted probabilities (using D_c and N_c notation), and the resulting code-length change compared to non-excluding PPMC. Summarise the total bit savings.

References

[1] Cleary, J. G., & Witten, I. H. (1984). Data compression using adaptive coding and partial string matching. *IEEE Transactions on Communications*, **32**(4), 396–402.

[2] Witten, I. H., Neal, R. M., & Cleary, J. G. (1987). Arithmetic coding for data compression. *Communications of the ACM*, **30**(6), 520–540.

[3] Bell, T. C., Cleary, J. G., & Witten, I. H. (1990). *Text Compression*. Prentice-Hall.

[4] Bloom, C. (1998). Solving the problems of context modelling. *Technical Report*, available at `http://www.cbloom.com/papers`.

[5] Mahoney, M. (2005). Adaptive weighing of context models for lossless data compression. *Florida Tech Technical Report*.

[6] Sayood, K. (2017). *Introduction to Data Compression* (5th ed.). Morgan Kaufmann.

[7] Deletang, G., Ruoss, A., Duquenne, P.-A., et al. (2023). Language modelling is compression. *arXiv preprint arXiv:2309.10668*.

10: Lecture 10: Modern Entropy Coders — ANS and Range Coding

10.1 Introduction: Beyond Huffman's Limitations

Entropy coding is the final stage of most modern compression systems. Earlier chapters introduced two fundamental approaches:

- **Huffman Coding** (Chapter 2.8) — fast but limited to integer-bit codeword lengths.
- **Arithmetic Coding** (Chapter 3.7) — achieves fractional-bit precision and approaches the Shannon limit.

Although arithmetic coding provides near-optimal compression, it has historically faced several challenges:

1. **Patent restrictions** (expired around 2005).
2. **High computational cost**, especially multiplications and divisions.
3. **Complex implementation**, particularly renormalization and interval management.

These limitations motivated the search for faster, simpler entropy coders that still preserve the compression efficiency of arithmetic coding. This chapter presents two such techniques:

- **Range Coding** — a practical integer formulation of arithmetic coding used in LZMA, WebP, and ffmpeg.
- **Asymmetric Numeral Systems (ANS)** — a breakthrough method discovered in 2009 that combines Huffman-like speed with arithmetic-like compression.

Both techniques are fundamental to modern compressors, including Zstd, LZFSE, JPEG XL, and other industry standards.

10.2 Range Coding: Arithmetic With Integers

Range coding is best understood as a re-interpretation of arithmetic coding where the "interval" is represented using integers rather than fractions. This gives several advantages:

- simpler renormalization,
- integer-only arithmetic,
- complete avoidance of arithmetic coding patents,
- highly efficient implementations (used by LZMA, WebP, VPx, and others).

10.2.1 Core Concept

Arithmetic coding encodes a message by narrowing a fractional interval $[0, 1)$. Range coding reformulates this idea:

> *Represent the current interval as integer values* `low` *and* `range` *where the actual interval is* [`low`, `low`+ `range`) *inside a fixed-size integer space (usually 32 bits).*

For each symbol s with frequency $f(s)$ and cumulative frequency $F(s)$ (where $\sum f(s) =$ total), the interval is updated as:

Algorithm 17 Modern Range Coding Update

```
procedure EncodeSymbol(s, freq, cum, total)
    range ← range × freq[s]/total                    ▷ First narrow range
    low ← low + range × cum[s]/total                 ▷ Then adjust low
    Renormalize
end procedure
```

Most modern implementations use **scaled integer frequencies**, where `total` is a power of two (e.g., 2^{12}, 2^{15}) to replace division with bit shifts.

10.2.2 Renormalization with Byte Output

Because `low` and `range` are stored in a fixed-width integer (e.g., 32 bits), we must periodically *renormalize* to maintain precision. The industry standard (LZMA, WebP) uses:

$$\text{threshold} = 2^{24} = 16{,}777{,}216$$

Whenever range $< 2^{24}$, we output the most significant byte of `low` and shift:

Algorithm 18 Renormalization (Industry Standard)

```
procedure Renormalize
    while range < 2^24 do
        output_byte(low ≫ 24)
        low ← (low ≪ 8) & 0xFFFFFFFF
        range ← range ≪ 8
    end while
end procedure
```

This ensures at least 24 bits of precision at all times and produces a byte-aligned output stream.

10.2.3 Advantages

- No floating-point operations.
- The method is patent-free.
- Excellent performance on modern CPUs.
- Simple to integrate into block compressors.

10.2.4 Range Coding Example

Consider an alphabet $\{A, B, C\}$ with frequencies:

$$f(A) = 5, \quad f(B) = 3, \quad f(C) = 2, \quad \text{total} = 10.$$

Cumulative ranges:

$$A : [0, 5), \quad B : [5, 8), \quad C : [8, 10).$$

Encode the sequence: `A B A C`.
We begin with:

$$\texttt{low} = 0, \qquad \texttt{range} = 2^{32} - 1.$$

Encode 'A':

$$\text{range} \leftarrow \text{range} \times 5/10 = 2{,}147{,}483{,}647 \tag{1}$$
$$\text{low} \leftarrow 0 + 2{,}147{,}483{,}647 \times 0/10 = 0 \tag{2}$$

Check renormalization: range $= 2{,}147{,}483{,}647 \geq 2^{24}$, so continue.
Encode 'B':

$$\text{range} \leftarrow 2{,}147{,}483{,}647 \times 3/10 = 644{,}245{,}094 \tag{3}$$
$$\text{low} \leftarrow 0 + 644{,}245{,}094 \times 5/10 = 322{,}122{,}547 \tag{4}$$

Encode 'A':

$$\text{range} \leftarrow 644{,}245{,}094 \times 5/10 = 322{,}122{,}547 \tag{5}$$
$$\text{low} \leftarrow 322{,}122{,}547 + 322{,}122{,}547 \times 0/10 = 322{,}122{,}547 \tag{6}$$

Encode 'C':

$$\text{range} \leftarrow 322{,}122{,}547 \times 2/10 = 64{,}424{,}509 \tag{7}$$

$$\text{low} \leftarrow 322{,}122{,}547 + 64{,}424{,}509 \times 8/10 = 322{,}122{,}547 + 51{,}539{,}607 = 373{,}662{,}154 \tag{8}$$

Now range $= 64{,}424{,}509 \geq 2^{24}$, so no renormalization needed. The compressed representation is the final low value.

10.3 Asymmetric Numeral Systems (ANS): The Breakthrough

Introduced in 2009 by Jarosław Duda, ANS revolutionized entropy coding by achieving:

- compression efficiency very close to arithmetic coding,
- speeds approaching and often exceeding Huffman coding,
- simple integer arithmetic,
- elegant mathematical structure,
- ideal suitability for software and hardware implementations.

10.3.1 Historical Context

Before ANS, entropy coding was effectively split between:

- **Huffman coding**: fast but suboptimal (integer bits only).
- **Arithmetic coding**: optimal but comparatively slow (requires multiplication/division).

Duda's insight was to invert the typical relationship between symbols and intervals, encoding information into the *magnitude* of a single integer state.

10.3.2 The Core Insight

ANS is founded on a simple idea:

> *Maintain a single integer state x. Encoding a symbol transforms x into a larger value x' whose growth approximates the ideal information cost $-\log_2 p(s)$.*

If a symbol s has probability p, ANS ensures:

$$\log_2(x') - \log_2(x) \approx -\log_2 p.$$

This mirrors arithmetic coding but uses integer state transitions instead of intervals.

10.3.3 Intuitive Explanation: The "Stack" Property

Arithmetic coding emits bits from the most significant side. ANS, however, emits (and consumes) bits at the *least significant side*, which reverses the order of symbols:

- Encoding: symbols push bits onto the end of the state (like a stack).
- Decoding: symbols are read in reverse order, popping bits from the state.

This natural LIFO (Last-In-First-Out) behavior means:

- If you encode $s_1, s_2, \ldots, s_n$ in that order, decoding yields $s_n, \ldots, s_2, s_1$
- To decode in natural order, either encode in reverse or buffer and reverse the output

10.3.4 Why ANS Is Faster

- A single integer state variable (no interval maintenance).
- No divisions in the fast path for tANS (precomputed tables).
- Sequential memory access (no tree traversal like Huffman).
- Compatible with SIMD and GPU architectures.

ANS supports two major variants: rANS and tANS.

10.4 Variations of ANS

10.4.1 rANS (Range ANS)

rANS uses arithmetic-like interval partitioning:

$$x' = \left\lfloor \frac{x}{f(s)} \right\rfloor \cdot \text{total} + (x \bmod f(s)) + F(s)$$

where:

- x is the current state
- $f(s)$ is the frequency of symbol s
- $F(s)$ is the cumulative frequency of all symbols before s
- $\text{total} = \sum f(s)$ (typically a power of two)

Key properties:

- Very simple to implement.
- Excellent compression efficiency.
- Naturally streaming (push/pop bits from LSB side).

10.4.2 tANS (Table ANS)

tANS precomputes a finite-state automaton (FSA) that maps:

$$\text{state},\ \text{symbol} \longrightarrow \text{new state} + \text{emitted bits}.$$

Properties:

- Extremely fast (used in Zstandard).
- No multiplications or divisions at runtime.
- Ideal for hardware (lookup tables in ROM).
- Memory: approximately $4 \times$ states bytes (e.g., 16 KB for 4096 states).

10.4.3 Comparison

- **rANS**: easier to implement, flexible, great for streaming, $O(1)$ memory.
- **tANS**: faster, more complex, uses $O(\text{states})$ memory, ideal for small alphabets.

10.5 Detailed rANS Example

To make the mechanics of rANS fully concrete, this section walks through a complete encoding and decoding example. This includes:

- constructing normalized frequencies,
- building cumulative tables,
- understanding renormalization,
- encoding a symbol sequence step-by-step,
- decoding it back to the original sequence.

For pedagogical clarity, we use small numbers and a small symbol alphabet. Practical implementations use much larger states and ranges.

10.5.1 Normalized Frequencies

Given an alphabet $\{A, B\}$ with raw frequencies:

$$f(A) = 2, \qquad f(B) = 1, \qquad \text{total} = 3,$$

we normalize the frequencies to a power-of-two total:

$$L = 4.$$

Normalized frequencies:

$$f_A = 3, \qquad f_B = 1.$$

Cumulative frequency ranges:

$$A: \ [0, 3), \qquad B: \ [3, 4).$$

Throughout this example:

$$L = 4.$$

10.5.2 Initial State

rANS maintains a single integer state x. To keep the example small, we set:

$$x_0 = L = 4.$$

Practical implementations initialize x to a much larger value (e.g., 2^{23} for rANS32 or 2^{56} for rANS64), but using $x_0 = L$ is perfectly valid for illustrating the algorithm.

10.5.3 Encoding Equations

Given the current state x and symbol s with frequency $f(s)$ and cumulative range $F(s)$, rANS encodes using:

$$x' = \left\lfloor \frac{x}{f(s)} \right\rfloor \cdot L + (x \bmod f(s)) + F(s).$$

This increases the magnitude of x so that its growth approximates $-\log_2 p(s)$ bits of information.

10.5.4 Renormalization

Since x grows during encoding, we must periodically emit lower bits to keep it within a machine word.

Define a renormalization lower bound (for educational purposes):

$$\text{threshold} = L \cdot 256.$$

While $x \geq$ threshold:

1. Emit the lowest 8 bits of x (LSB-first).

2. Set $x \leftarrow x \gg 8$.

This keeps x at a manageable size. (Production implementations use much larger thresholds.)

10.5.5 Encoding the Sequence `B A B A`

Important: Because ANS is a stack-like coder, we encode symbols in **reverse order** so that decoding produces the original order. For sequence `B A B A` (original order), we encode:

`A, B, A, B`

Step 1: Encode A
Current state:

$$x = 4.$$

For A:

$$f_A = 3, \qquad F(A) = 0.$$

Compute:

$$x' = \left\lfloor \frac{4}{3} \right\rfloor \cdot 4 + (4 \bmod 3) + 0 = 1 \cdot 4 + 1 = 5.$$

No renormalization required.

$$x = 5.$$

Step 2: Encode B

$$x = 5.$$

For B:

$$f_B = 1, \qquad F(B) = 3.$$

Compute:

$$x' = \left\lfloor \frac{5}{1} \right\rfloor \cdot 4 + (5 \bmod 1) + 3 = 5 \cdot 4 + 0 + 3 = 23.$$

Still below the threshold:

$$x = 23.$$

Step 3: Encode A

$$x = 23.$$

For A:

$$x' = \left\lfloor \frac{23}{3} \right\rfloor \cdot 4 + (23 \bmod 3) = 7 \cdot 4 + 2 = 30.$$

$$x = 30.$$

Step 4: Encode B

$$x = 30.$$

For B:

$$x' = \left\lfloor \frac{30}{1} \right\rfloor \cdot 4 + 3 = 30 \cdot 4 + 3 = 123.$$

$$x = 123.$$

No renormalization was triggered because our threshold $= 4 \cdot 256 = 1024$.

10.5.6 Final Compressed State

At the end of encoding:

$$x_{\text{final}} = 123.$$

The output stream consists of:

- All emitted renormalization bytes (none in this example).
- The final state x_{final} (which will be read by the decoder).

10.5.7 Decoding

Decoding processes symbols in the **forward** order (starting from the final state) and will produce symbols in reverse order of encoding. Since we encoded `A, B, A, B`, decoding will produce:

`B, A, B, A`

which matches our original sequence `B A B A`.

Given current state x, we determine which symbol was encoded by computing:

$$\text{slot} = x \bmod L.$$

Then:

$$\text{symbol} = \text{lookup_by_cumulative}(\text{slot}).$$

For our cumulative table:

$$0, 1, 2 \to A, \qquad 3 \to B.$$

After identifying the symbol s, we compute the previous state:

$$x_{\text{prev}} = f(s) \cdot \left\lfloor \frac{x}{L} \right\rfloor + (x \bmod L) - F(s).$$

Finally, if $x_{\text{prev}} < L$, we pull bytes from the input stream until:

$$x_{\text{prev}} \geq L.$$

10.5.8 Decoding Step-by-Step

Start

$$x = 123.$$

Step 1

$$\text{slot} = 123 \bmod 4 = 3.$$

Slot 3 corresponds to B.
Compute:

$$x_{\text{prev}} = 1 \cdot \left\lfloor \frac{123}{4} \right\rfloor + 3 - 3 = 1 \cdot 30 + 0 = 30.$$

$$x = 30.$$

Step 2

$$\text{slot} = 30 \bmod 4 = 2.$$

Slot $2 \to A$.

$$x_{\text{prev}} = 3 \cdot \left\lfloor \frac{30}{4} \right\rfloor + (2 - 0) = 3 \cdot 7 + 2 = 23.$$

$$x = 23.$$

Step 3

$$\text{slot} = 23 \bmod 4 = 3.$$

Slot $3 \to B$.

$$x_{\text{prev}} = 1 \cdot \left\lfloor \frac{23}{4} \right\rfloor + (3 - 3) = 1 \cdot 5 + 0 = 5.$$

$$x = 5.$$

Step 4

$$\text{slot} = 5 \bmod 4 = 1.$$

Slot $1 \to A$.

$$x_{\text{prev}} = 3 \cdot \left\lfloor \frac{5}{4} \right\rfloor + (1 - 0) = 3 \cdot 1 + 1 = 4.$$

$$x = 4.$$

This matches $x_0 = 4$, confirming correct decoding.

10.5.9 Summary

The decoded symbol sequence (in order of decoding):

$$\boxed{\texttt{B A B A}}$$

matches the original. This demonstrates that:

- Encoding in reverse order: `A, B, A, B`
- Decoding in forward order: `B, A, B, A`
- Result matches original: `B A B A`

10.6 ANS in Modern Compressors

ANS is widely used in state-of-the-art compression formats and libraries due to its exceptional performance and flexibility.

10.6.1 Zstandard (Facebook / Meta)

Zstandard's entropy stage uses tANS extensively. Benefits include:

- extremely fast decoding,

- efficient table-driven implementation,
- well-suited for CPU vectorization,
- significantly better speed than arithmetic coding in practice.

tANS contributes significantly to Zstd's reputation as one of the fastest modern general-purpose compressors.

10.6.2 Apple LZFSE

LZFSE uses ANS as a core component to replace zlib-style Huffman coding. Advantages include:

- improved compression ratio over zlib,
- much higher throughput,
- optimized for modern ARM processors.

10.6.3 Google Draco

Draco uses ANS for compressing 3D geometry and mesh connectivity. Reasons include:

- compact symbol representation,
- suitability for GPU-friendly decoding,
- excellent branching behavior.

10.6.4 JPEG XL

JPEG XL employs ANS as part of its modular and VarDCT encoding pipelines. ANS allows:

- high-speed entropy coding for transforms,
- efficient modeling of image coefficients,
- extremely competitive performance compared to arithmetic coding while avoiding multipliers.

10.7 Hands-On: Implementing a Simple ANS Coder

This section provides a complete pedagogical implementation of rANS in Python. Real-world implementations use optimizations, but this version is correct and readable.

10.7.1 Educational rANS Implementation

```python
class EducationalRANS:
    """Educational rANS implementation - demonstrates core concepts"""

    def __init__(self, freqs, L=1024):
        """
        Initialize rANS coder with symbol frequencies.

        Args:
            freqs: List of frequencies for symbols 0..n-1
            L: Total frequency (power of two, >= sum(freqs))
        """
        self.L = L
        # Normalize frequencies to sum to L
        total_raw = sum(freqs)
        self.freqs = []
        for f in freqs:
            # Ensure at least 1 occurrence
            norm = max(1, int(f * L / total_raw))
            self.freqs.append(norm)

        # Adjust to exactly sum to L (add residuals to largest symbols)
        diff = L - sum(self.freqs)
        for i in range(diff):
            self.freqs[i % len(freqs)] += 1

        # Build cumulative frequencies
        self.cum = [0]
        for f in self.freqs[:-1]:
            self.cum.append(self.cum[-1] + f)

        # Build lookup table for decoding
        self.slot_to_sym = [-1] * L
        for sym, (start, end) in enumerate(zip(self.cum,
                                                [c + f for c, f in zip(
                                                    self.cum, self.freqs
                                                    )])):
            for slot in range(start, end):
                self.slot_to_sym[slot] = sym

    def encode(self, data):
        """
        Encode data to byte stream.
        Note: Encodes in reverse order due to ANS LIFO property.
        """
        x = self.L  # Initial state
        output = []
```

```python

        # Encode in reverse order so decoding gives original order
        for sym in reversed(data):
            # Renormalization: emit bytes when state is large
            while x >= self.L * 1024:  # Example threshold
                output.append(x & 0xFF)
                x >>= 8

            # rANS encoding step
            freq = self.freqs[sym]
            cum = self.cum[sym]
            q = x // freq
            r = x % freq
            x = q * self.L + cum + r

        # Output final state
        while x > 0:
            output.append(x & 0xFF)
            x >>= 8

        return output

    def decode(self, byte_stream):
        """
        Decode from byte stream.
        Returns symbols in original order.
        """
        # Initialize state from bytes (LSB-first)
        x = 0
        for b in reversed(byte_stream):
            x = (x << 8) | b

        result = []

        while x > self.L:
            # rANS decoding step
            slot = x % self.L
            sym = self.slot_to_sym[slot]

            freq = self.freqs[sym]
            cum = self.cum[sym]
            x = freq * (x // self.L) + (slot - cum)

            result.append(sym)

        # Result is already in original order because we encoded in reverse
        return result
```

```python

# Example usage
def test_rans():
    # Alphabet: A=0, B=1 with frequencies 2 and 1
    freqs = [2, 1]
    data = [1, 0, 1, 0]  # B A B A

    encoder = EducationalRANS(freqs, L=4)
    encoded = encoder.encode(data)
    decoded = encoder.decode(encoded)

    print(f"Original: {data}")
    print(f"Encoded bytes: {encoded}")
    print(f"Decoded: {decoded}")
    assert decoded == data
    print("Roundtrip successful!")

if __name__ == "__main__":
    test_rans()
```

Listing 3: Educational rANS Implementation

10.7.2 Key Implementation Notes

- **Reverse encoding**: The encode function processes symbols in reverse order so decode returns original order.
- **Normalization**: Frequencies are normalized to sum to a power of two for efficient division (using bit shifts in optimized implementations).
- **Renormalization threshold**: The example uses $L \times 1024$; production code uses much larger thresholds.
- **Slot lookup**: The decoder uses a precomputed array for $O(1)$ symbol lookup.

10.8 Summary

This chapter presented two powerful entropy coders that complement or replace traditional arithmetic coding:

- **Range Coding** — an integer-based variant of arithmetic coding, widely used in LZMA, WebP, and ffmpeg.
- **ANS (Asymmetric Numeral Systems)** — a revolutionary coding system offering arithmetic-like compression efficiency with Huffman-like speed.

Two variants of ANS were covered:

- **rANS** — streaming, simple, and ideal for software implementations.
- **tANS** — table-driven, extremely fast, ideal for high-performance applications.

Key takeaways:

1. Range coding and ANS both achieve fractional bits per symbol (unlike Huffman).
2. ANS's LIFO property is fundamental — information is pushed to the least significant side.
3. rANS uses $O(1)$ memory; tANS uses $O(\text{states})$ memory but offers higher speed.
4. Both are patent-free and widely used in modern compressors.

10.9 Exercises

1. **Range Coding Implementation:** Construct a range coder for an alphabet of size 3 using 32-bit arithmetic. Encode and decode the sequence `CABAC`. Show all intermediate steps including renormalization.
2. **rANS Bits Emitted:** Derive the exact number of bits emitted by the rANS encoding of `B A B A` when using a practical renormalization threshold of $L \times 1024$. How does this compare to the theoretical entropy?
3. **tANS Table Construction:** Build a tANS table for frequencies $\{A : 5, B : 3, C : 2\}$ with $L = 16$ states. Trace encoding and decoding of the sequence `ABCAB`.
4. **Performance Comparison:** Implement rANS32 in C or Python and benchmark it against a standard Huffman coder on:
   - Random data with uniform distribution
   - English text (e.g., a book chapter)
   - Highly skewed data ($p(A) = 0.95$, others small)

   Compare compression ratio and speed.
5. **Mathematical Analysis:** Compare arithmetic coding and rANS for a skewed distribution (e.g., $p(A) = 0.95$, $p(B) = 0.05$). Which produces more renormalization steps? Explain why.
6. **Adaptive ANS:** Extend the rANS implementation to support adaptive probabilities. Use a simple update rule (e.g., increment frequency after each symbol). Compare compression ratio against static ANS on non-stationary data.
7. **LIFO Analysis:** Explain why ANS's LIFO property is problematic for streaming applications. Propose two different solutions and discuss their trade-offs.

Additional Resources

- Duda, J. (2013). "Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding." arXiv:1311.2540.
- Giesen, F. (2014). "rANS in practice." `https://fgiesen.wordpress.com/`
- Collet, Y. (2016). "Finite State Entropy: A new breed of entropy coder." `https://github.com/Cy`
- Zstandard source code: `lib/compress/zstd_opt.c` and `lib/entropy_common.c`
- WebP source code: `https://chromium.googlesource.com/webm/libwebp/`